University of Belgrade

Faculty of Organizational Sciences

Final work

Veljko Blagojevic 2023/3031

Belgrade, 2026

University of Belgrade

Faculty of Organizational Sciences

Information systems and technologies

Information systems

Final work

Web application development using Microfrontend architecture

Veljko Blagojevic 2023/3031

Belgrade, 2026

УНИВЕРЗИТЕТ У БЕОГРАДУ
ФАКУЛТЕТ ОРГАНИЗАЦИОНИХ НАУКА

Сагласност чланова комисије
за преглед, јавну одбрану и оцену завршног рада

Комисија која је прегледала завршни рад

Студента:	Вељко Благојевић
Број индекса:	2023/3031
Под насловом	Развој веб апликације применом Microfrontend архитектуре
The Title of Master Thesis	Web application development using microfrontend architecture

У следећем саставу

Ментор:	др Слађан Бабарогић, редовни професор
Први члан:	др Иван Луковић, редовни професор
Други члан:	др Бојан Јовановић, ванредни професор

дала је електронску сагласност за јавну одбрану завршног рада.

Јове Илића 154, 11000 Београд, Србија, Тел.: (011) 3950-800
ПИБ: 100383934, Матични број: 07004044
Е пошта: dekanat@fon.bg.ac.rs; Посетите: www.fon.bg.ac.rs

УНИВЕРЗИТЕТ У БЕОГРАДУ
ФАКУЛТЕТ ОРГАНИЗАЦИОНИХ НАУКА

04-15 број 11/

Изјава о академској честитости

Име и презиме: Вељко Благојевић

Број индекса: 2023/3031

Студијски програм: Информациони системи и технологије

Модул: Информациони системи

Аутор завршног рада под насловом	Развој веб апликације применом Microfrontend архитектуре
Чија је израда одобрена на Седници Већа студијских програма мастер академских студија одржаној:	25.9.2024.

Потписивањем изјављујем:

- Да је рад искључиво резултат мог сопственог истраживачког рада;
- Да сам рад и мишљења других аутора које сам користио у овом раду назначио или цитирао у складу са Упутством;
- Да су сви радови и мишљења других аутора наведени у списку литературе/референци који су саставни део овог рада и писани су у складу са Упутством;
- Да сам довио све дозволе за коришћење ауторског дела који се у потпуносзи/целости уносе у предати рад и да сам то јасно навео;
- Да сам свестан да је плагијат коришћење туђих радова у било ком облику (као цитата, парафраза, слика, табела, дијаграма, дизајна, планова, фотографија, филма, музике, формула, веб сајтова, компјутерских програма и сл.) без навођења аутора или представљање туђих ауторских дела као мојих, кажњиво по закону (Закон о ауторским и сродним правима, Службени гласник Републике Србије, бр. 104/2009, 99/2011, 119/2012), као и других закона и одговарајућих аката Универзитета у Београду и Факултета организационих наука;
- Да сам свестан да плагијат укључује и представљање, употребу и дустрибуирање рада предавача или других студената као сопствених;
- Да сам свестан последица које код доказаног плагијата могу проузроковати на предати завшни мастер рад и мој статус;
- Да је електронска верзија завршног рада идентична штампаном примерку и пристајем на његово објављивање под условима прописаним актима Универзитета и Факултета.

Београд, 01.09.2026. године

Вељко Благојевић

Потпис студента

WORK BIOGRAPHY

Veljko Blagojević was born on August 2, 2000 in Pančevo, where he graduated from high school in electrical engineering in 2019. After that, he attended the Faculty of Organizational Sciences and graduated in 2023 with a degree in Information Systems and Technologies. In 2022, he began his internship at BlackRock, where he still works as a programmer.

Summary

The goal of the master's thesis is to understand and evaluate the implementation of microfrontend architecture on the client layer of a web application: what is gained by switching from a frontend monolith, what is lost, and under what conditions is that price justified. The research domain is the client layer of the MicroMedic web application, intended for scheduling and managing medical examinations, whose user interface is divided into functionally separable areas of responsibility for the doctor and the patient.

The work is based on the *design science research method* : knowledge is gained by building an artifact - a microfrontend architecture implemented through multiple independent parts in four different frameworks, and its evaluation according to predefined architectural characteristics. The evaluation led to four findings: the diversity of frameworks in a single interface is not an anti-pattern per se, but a consequence of the absence of management mechanisms; consistency of appearance between parts written in different frameworks is achieved by construction through a common design system; the choice of communication mechanism between parts follows from their topological relationship on the screen; and the microfrontend architecture establishes real boundaries of responsibility, but not security boundaries on the client side, which remain exclusively the responsibility of the server layer.

The main limitation of the work is the price that the microfrontend architecture imposes on the source code: the increased number of projects and the negotiation of shared dependencies. The value of the work is that the findings are derived from the system built as a whole, rather than hypothetical analysis.

Keywords: Microfrontend, microservices, software architecture, client layer user interface

The goal of this master's thesis is to understand and evaluate the application of microfrontend architecture on the client side of a web application: what is gained by transitioning from a frontend monolith, what is lost, and under what conditions the cost is justified. The research domain is the client side of the MicroMedic web application, designed for scheduling and managing medical appointments, whose user interface is divided into functionally distinct areas of responsibility for doctors and patients.

The work is based on the design science research method: knowledge is acquired by building an artifact - a microfrontend architecture implemented through multiple independent parts in four different frameworks - and evaluating it according to predetermined architectural characteristics. The evaluation led to four findings: the diversity of frameworks within a single interface is not inherently an anti-pattern but a consequence of the absence of management mechanisms; visual consistency among parts written in different frameworks is achieved through construction via a shared design system; the choice of communication mechanisms between parts follows from their topological relationship on the screen; and the microfrontend architecture establishes real boundaries of responsibility but not client-side security boundaries, which remain an exclusive obligation of the server layer.

The main limitation of the work is the cost that microfrontend architecture incurs in the source code - a multiplied number of projects and coordination of shared dependencies. The value of the work lies in the fact that the findings are derived from a fully built system rather than hypothetical analysis.

Keywords: Microfrontend, microservices, software architecture, client-side user interface

List of acronyms

Acronym	Meaning
XML	Extensible Markup Language
AJAX	Asynchronous JavaScript and XML
API	Application Programming Interface
SPA	Single Page Application
UI	User Interface
HTML	HyperText Markup Language
DOM	Document Object Model
CSS	Cascading Style Sheets
DDD	Domain-driven design
CORS	Cross-Origin Resource Sharing
DNS	Domain Name System
SSI	Server Side Includes
TTFB	Time to First Byte
ESI	Edge Side Includes
CDN	Content Delivery Network
UTF	Unicode Transformation Format
FON	Faculty of Organizational Sciences
ECMA	European Computer Manufacturers Association
ES	ECMAScript
JSON	JavaScript Object Notation
kB	kilobyte

OAuth	Open Authorization
JWT	JSON Web Token
BFF	Backend for Frontend
GraphQL	Graph Query Language
REST	Representational State Transfer
E2E	End to End
CSP	Content Security Policy
XSS	Cross-site scripting
HSTS	HTTP Strict Transport Security
HTTPS	Hypertext Transfer Protocol Secure
OWASP	Open Worldwide Application Security Project
ICD-10	International Classification of Diseases , Tenth Revision
ICD-10	International Classification of Diseases, 10th Revision
PDF	Portable Document Format
MM	MicroMedic

List of tables

List of images

Contents

1 Introduction

Architecture, as a discipline that encompasses the art and science of design and construction, has deep roots in the history of human civilization. Since the earliest times, people have used architecture as a means of expressing their cultural values, technological achievements, and social needs. Over the centuries, architecture has evolved, adapting to changes in society, technology, and aesthetics, while simultaneously shaping the way we live, work, and interact with our environment.(Collins, Gowans, Ackerman, & Scruton, 2026)

The word "architecture" comes from the ancient Greek word " *arkhitéktōn* ", which translates to "chief builder" and denotes the one who leads the construction, not the one who carries it out. In this sense, architecture is not the technical details of construction, but a series of decisions about what is built and according to what priorities: decisions that are made before construction begins and that are all the more difficult to change the longer the construction is underway.

These decisions were never merely technical. The Egyptian pyramids and Greek temples perform a function for which much simpler structures would suffice, and their form and scale testify to priorities—durability, symmetry, visibility — that outlived the immediate purpose of the building (Münster, 2024). It is precisely such priorities, not functions, that correspond to what in software architecture are called *architectural features* : properties of a system that cannot be read from a list of its functions and are therefore decided by compromise rather than choice (Richards & Ford, 2025).

The development of web application user interfaces has undergone a similar order of magnitude change. With the introduction of asynchronous data exchange with the server, described as *Ajax* (Asynchronous *JavaScript and XML*) (Garrett, 2005), the page stopped being a document that reloaded with each interaction and became an application that runs in the browser (*web browser*). Frameworks that *emerged in the* following years made this way of development the default, and the single- *page application (SPA)* became the *application*) in the standard form of the client layer (Mikowski & Powell, 2013).

This form, however, has a consequence that only becomes apparent with growth. A single-page application is typically one project, one build *,* and one delivery *:* any change, however local, requires a recompilation and release of the whole, and the number of people who can work on it at the same time is limited by the number of people who can agree on a single version of each external library and a single delivery point.

, this consequence ceases to be a property of a single application and becomes an imbalance between layers. The server layer is broken down into microservices that are delivered separately by independent teams, and the team that delivers its own A service can publish several times a day, waiting for a shared frontend delivery to make that service visible to the user . The frontend monolith is therefore not a problem because of its size, but because it is a bottleneck in a delivery flow that is distributed everywhere except within it (Geers, 2020).

Microfrontend architecture (English *Micro frontend*) emerged as a response to this imbalance: the user interface is divided into independently developed, delivered and executed parts, the boundary between them is not technical but business, and the place of its assembly as a whole is the browser (Geers, 2020) (Mezzalira, Building Micro-Frontends, 2025). The division brings autonomy of teams and independent delivery, but also a cost - the browser, when assembling, has to solve the problem of routing, communication between parts, consistency of appearance, the amount of downloaded code and trust boundaries between parts, and none of these problems existed in a monolithic application, because the framework solved them implicitly, in the background without requiring any effort from the developer.

The most controversial consequence of this division is the possibility that parts of the interface can be implemented in different frameworks. The literature is therefore very cautious and most often describes such use as an anti-pattern, citing increased code duplication, inconsistent appearance, and difficulty in maintenance (Mezzalira, Building Micro-Frontends, 2025) (Rufus, 2023). The caution, however, is directed at the absence of control mechanisms, not at diversity per se, and the difference between the two statements is not resolved by principled reasoning, but by analysis of the built system.

The topic of this paper belongs to the field of information systems architecture, and the research domain is the client layer of web applications with multiple functionally separable areas. The subject of the paper is microfrontend architecture as a way of organizing that layer, and the goal is to examine on a specific, fully built system what is gained by switching from a monolithic frontend to a microfrontend architecture, what is lost, and under what conditions the use of multiple frameworks remains a controlled decision, not an anti-pattern. The system on which the examination is performed is "MicroMedic", an information system for scheduling and recording medical examinations, chosen because its domain contains clearly separable subdomains, a workflow that requires multiple forms of interface division and communication methods on a single screen.

design . science research), in which knowledge is acquired by building an artifact and evaluating it according to predetermined criteria (Vaishnavi & Kuechler, 2015). An artifact is the aforementioned

system, the client layer of which is implemented as a set of independently deliverable microfrontends consciously implemented in several different frameworks, while the server layer is intentionally left monolithic so that the variable under investigation is limited to the client layer.

The second chapter introduces the concept of software architecture, architectural characteristics as a criterion by which architectures are compared, monolithic and distributed styles, and the development from multi-page to single-page applications, to front-end monoliths as a problem.

The third chapter presents the microfrontend architecture: definition and principles, advantages and disadvantages, defining boundaries using domains, its type space, and a decision framework that is used as an analytical tool in the rest of the paper.

Chapter 4 covers composition, integration, and routing, from transitions via links, through server-side and client-side composition, to the *Module mechanism. Federation* , and the impact of the chosen technique on resource loading.

The fifth chapter discusses communication and state management: three directions of communication, the event bus, global context and authentication, and communication with the server layer.

The sixth chapter is dedicated to the user interface and design system , how *to* maintain consistency of appearance when the interface is created by multiple independent teams.

Chapter 7 describes delivery, automation, and testing: continuous integration, the relationship between mono-repositories and poly-repositories, versioning, delivery metrics, and test strategy layers.

Chapter eight covers security and isolation of microfrontends: web platform standards, the question of why client-side isolation is not solvable, authentication, and token storage.

Chapter 9 systematizes the anti-patterns of microfrontend architectures, exposed after all the mechanisms they criticize, and among them singles out anarchy, the use of multiple frameworks without management mechanisms, as the case that is directly the subject of this paper.

Chapter 10 analyzes the research problem in a specific domain: the workflow of a doctor when examining a patient, functional requirements, non-functional requirements expressed as architectural features, identified subdomains, and confidentiality and traceability requirements for medical data.

Chapter 11 presents the architecture of the MicroMedic solution , explains the decisions made in designing the client layer, and describes the server layer.

Chapter 12 describes in detail the implementation of the client layer: the shell application , the topology of microfrontends, both forms of interface partitioning used, shared state, the design system with intermediary packages, the use of multiple frameworks as a controlled experiment, and implemented and unimplemented security measures.

Chapter thirteen summarizes the work's findings through four findings, lists the contributions of the work, and outlines directions for further research.

2 Software architectures

Software architecture encompasses the structure of a system, its parts, and the relationships between them, along with the decisions and principles by which that structure was derived (Clements, et al., 2010).

Designing a structure involves two activities: understanding the business domain, which determines what the system does, and determining the properties that the system must have to be successful, which determine how and under what conditions it does it (Richards & Ford, 2025). The first describes the behavior of the system, the second its capabilities. These properties are called architectural features, and the ways in which a particular set of them can be supported are called architectural styles.

The organizing principle of both activities is modularity, the grouping of related code into parts that are understood and modified separately, where the separation is logical, not necessarily physical (Richards & Ford, 2025). A set of classes can be neatly divided by responsibilities, while still being delivered as a single unit.

2.1 Architectural features and compromises

Properties that a system must possess that cannot be read from a list of its functions are called *non-functional requirements* , *quality attributes* , or *architectural characteristics* . (Richards & Ford, 2025)The first term is rejected because it devalues what it describes, and the second because quality refers to the subsequent evaluation of the finished system, not the object of design; therefore, the third is used here.

For a requirement to be an architectural feature, it must meet three conditions: it must not belong to the business domain, it must affect the structure of the solution, and it must be critical to the success of the system (Richards & Ford, 2025). The third condition is restrictive, because each supported feature introduces complexity: the goal is to choose the fewest of them.

Characteristics are classified as operational, structural, and cross-cutting. Some of them are explicit and stated in the requirements, and some are implicit: availability, reliability, and security are implied in almost every application, so the architect derives them from domain knowledge.

Table 2-1 Architectural features

Characteristic	Group	Determination
Performance	operational	Response time and initial loading time under expected load.
Scalability	operational	The ability of a system to maintain responsiveness as the number of users or requests increases.
Availability	operational	The portion of time during which the system is usable.
Modularity	structural	The degree of decomposition of the system into parts of high cohesion and low coupling.
Delivery	structural	The frequency, ease, and risk of delivering a single change to production.
Testability	structural	Ease of fully verifying the system with automated tests.
Extensibility	structural	Ease of adding new functionality without modifying existing parts.
Security	cross	System resistance to unauthorized access and malicious use.
Confidentiality	cross	Protection of personal data from unauthorized access.
Authorization	cross	Verify that the participant has the right to perform the action requested.
Accessibility	cross	Usability of the interface with assistive technologies.
Agility	complex	A complex feature: deliverability, modularity, and testability together.

The relationship between features and structure shows a comparison of security and scalability: security in a monolithic system can be ensured by design, encryption, hashing , and discipline in writing code, while scalability beyond a certain limit is not achievable by any design, but requires a transition to a distributed style (Richards & Ford, 2025). The first is, therefore, a feature that the structure can change, and the second that the structure must change.

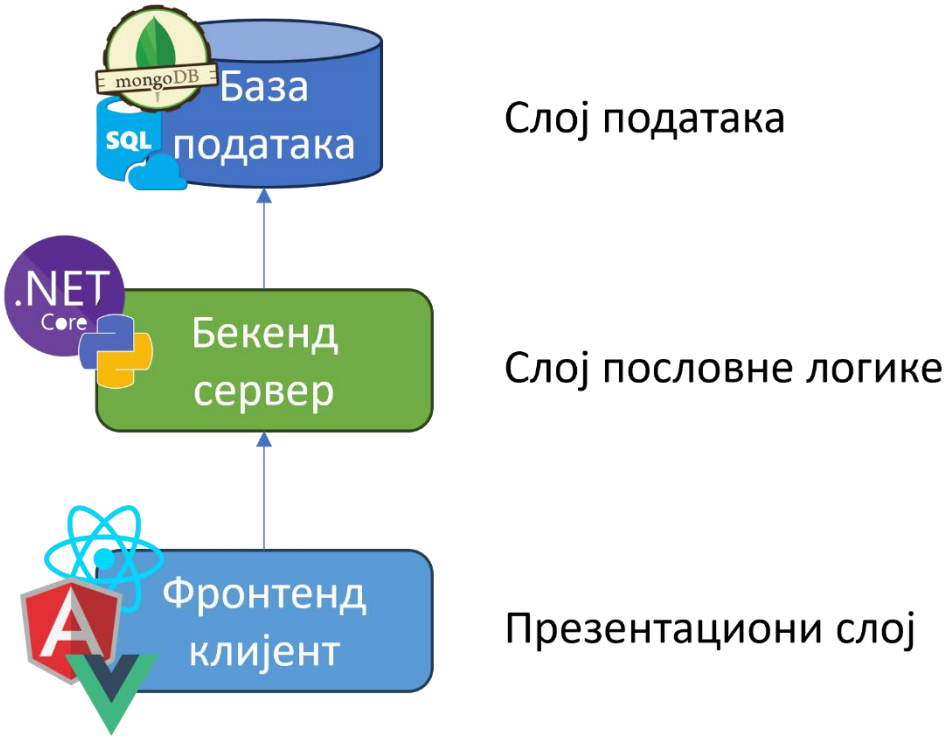
No feature comes without a price, and features interact with each other, with increased security almost always taking a toll on performance. Hence the principle that one should not look for the best architecture, but the least bad one (Richards & Ford, 2025), and hence architectures are compared by feature, one by one.

2.2 Architectural styles

An architectural style is a named set of constraints on the structure of a system, whereby certain features are supported while others are sacrificed. Styles are classified into two families: *monolithic* , in which the entire system is delivered as a single unit, and *distributed* , in which parts are delivered independently (Richards & Ford, 2025).

Monolithic architecture is based on a single codebase *that* encompasses all functionality (Mishra, Kunde, & Nambiar, 2018). This same feature brings its advantages: ease of development, testing, and delivery, and its challenges: a change anywhere requires delivery of the whole, and the system scales only by replicating the whole.

Within a monolithic family, there are two types of division, called **technical** and **domain** . A layered architecture is technically divided: the system is divided horizontally, according to the technical role of the parts, most often into user interface, business logic, data access, and storage (Влајић, 2020). The layers are closed, so that a change in one does not reach the others as long as the contract between them does not change, a property called *layer isolation* . Since the division is technical, one business domain intersects all layers, which in an organization divided by layers means that multiple teams participate in the development of one functionality. A modular monolith is domain-divided: the same system is divided vertically, into modules that correspond to business domains and are connected by well-defined interfaces, most often over a single database (Gandhi, Richards, & Ford, 2024). Such a division facilitates parallel work of teams organized by domain, but the flexibility of delivery practically does not change, the modules are still delivered as a single unit, and even the modular monolith remains a monolith in the capacity that is most important for this work.



Picture 2-1 Three-tier software architecture

Distributed styles push that boundary: the system is divided into units that are delivered separately and work together through standardized contracts, allowing each to use technology tailored to its task. In a microservices architecture, services are modeled around business domains, and the principles underlying it are independent delivery, the business domain as the basis for division, and a separate data domain per service, so that isolation ceases to be a property of the application layer alone (Newman, 2021). The price, however, is the same in all distributed styles: the network becomes a point of failure, consistency weakens, and complexity is shifted from the code base to delivery and monitoring.

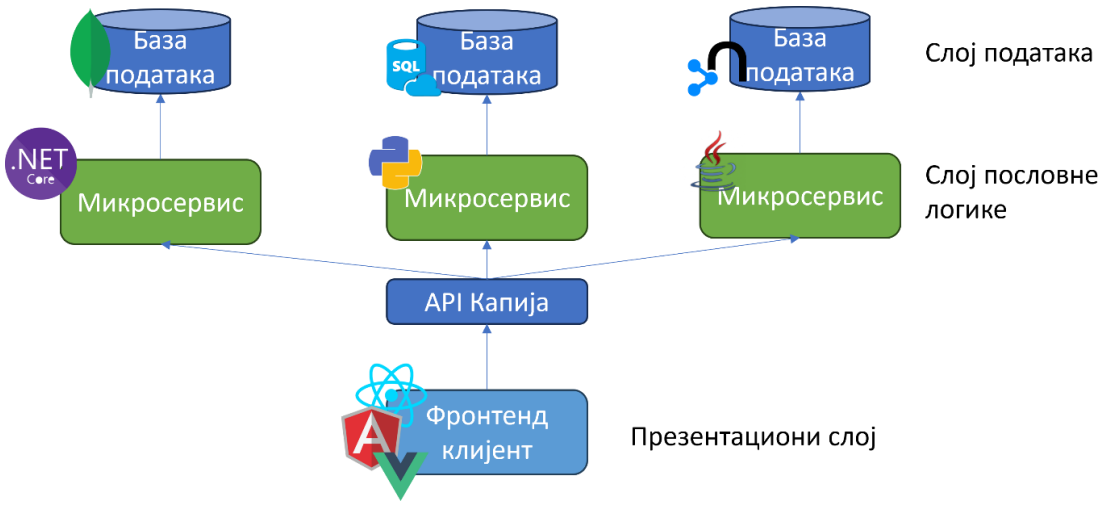


Figure 2-2Microservice software architecture

2.3 From multi-page to single-page applications

In multi-page applications, the document is compiled by the server, and each interaction that changes the view is a new request and reloads the entire page. The development pattern described as *Ajax* (*Asynchronous JavaScript and XML*) breaks this connection: a proxy mechanism in the browser exchanges data with the server asynchronously, so user interaction no longer waits for a network cycle (Garrett, 2005).

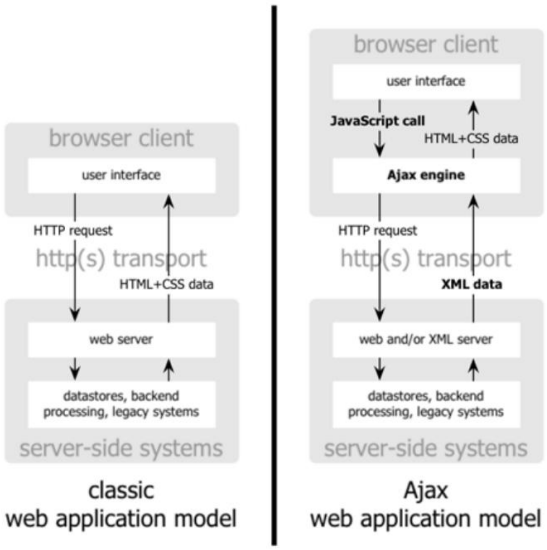


Figure 2-3Traditional web application model (left) compared to Ajax model (right)(Garrett, 2005)

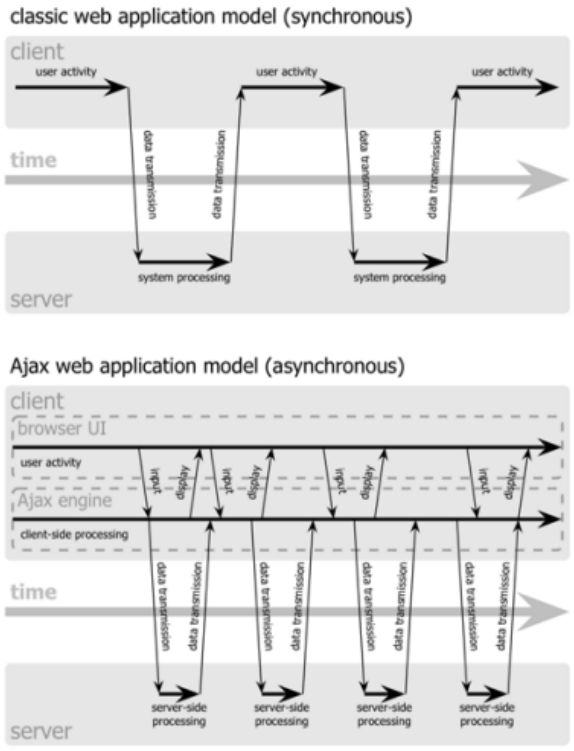


Figure 2-4The synchronous interaction pattern of a traditional web application (above) compared to the asynchronous communication pattern of the Ajax model (below)(Garrett, 2005)

This pattern gave rise **to single-page applications** : the entire frontend is downloaded in advance, in the form of *JavaScript* files, and then only parts of the display are changed, while the application in the background retrieves data from server access points.(Monteiro, 2014) (Yadav, 2025)The client thus takes over the responsibilities previously held by the server, routing, state management, and view composition, thus maintaining the division into layers, but shifting the boundary between them, and making the client "fat" (Mikowski & Powell, 2013).

The resulting user experience is close to that of desktop applications, but not without a price: the initial load is longer, indexing of content by search engines requires additional steps, and, most importantly for this work, the entire client layer becomes one project, one build, and one delivery.

2.4 Frontend monolith as a problem

The consequence described at the end of the previous section does not depend on the style in which the rest of the system is implemented. Monolithic architecture, the division into frontend *and* backend , and microservice architecture have one thing in common: in all of them the frontend is monolithic, that is, it comes from a single code base on which only one team can meaningfully work.

This leads to two limitations. The first is organizational: if teams are divided into layers, multiple teams participate in the development of a single feature, and each boundary between them introduces negotiation, specification, and waiting , and reducing this waiting is the primary goal of microfrontend

architecture. The second is technical: a single codebase means a single version of the framework and a single upgrade decision for the entire interface, so new technology cannot be introduced and tested in a limited part of the application without a migration plan for the whole (Geers, 2020).

2.5 Frontend modularity

Modularity is measured by cohesion, which shows how much the parts of a module belong together, and coupling , which shows how much a module depends on others; coupling is measurable in two ways, by the number of modules that depend on a given one and by the number of modules on which it depends (*affernt* and *efferent) . coupling*) (Richards & Ford, 2025). At the client layer, the unit of this grouping is **a component** : a whole that unifies the display structure, style, and behavior, and exposes a narrow interface to the environment, the properties it receives, and the events it publishes.

The need for components to go to *the* platform level, outside the framework, has led to the creation of the Web Components standard , *composed* of **three** technologies (Ottaviani Aragão, 2024): custom *elements elements*), which introduce new tags into HTML (*HyperText Markup Language). Markup Language*) along with lifecycle methods; *HTML templates* , document fragments that are not rendered until used; and *the " shadow" DOM* (English *Shadow Document Object Model*), a separate DOM tree whose structure and styles are closed to the rest of the document, while events still cross its boundary (Glazkov, 2011).

3 Microfrontend architecture

Section 2. 4 showed that the frontend remains uncomplicated even when the rest of the system is already distributed. A microfrontend architecture arises by extending the domain partitioning (2. 2), already applied to the server through microservices, to the presentation layer: the interface is divided into independently developed, tested, and deliverable parts, not just the logic underneath (Mezzalira, Building Micro-Frontends, 2025).

A microfrontend is, therefore, an independently developed, tested, and delivered slice of the user interface that represents a single business subdomain, and is combined with other microfrontends only at runtime, in the browser, into a complete application (Mezzalira, Building Micro-Frontends, 2025). The application that combines these parts is called **a wrapper** (*application) . shell*): it does not carry business logic itself, but distributes parts, provides them with a common loading space and, most often, manages authentication and shared processes.

There are two distinct forms in which a microfrontend enters the screen: as **a page** , when the entire view is occupied by a single microfrontend, or as **a fragment** , a self-contained part that fits into another view. This paper uses both terms in the following: page for the form corresponding to the vertical division (3.3), fragment for the form corresponding to the horizontal one.

The seven principles of microservice architecture also apply to the frontend:

- **Modeled around business domains** , the boundary of a microfrontend follows the domain, not the technical role.
- **A culture of automation** , a rapid feedback cycle precedes the adoption of architecture.
- **Hiding implementation details** , contract (*Application) Programming Interface - API*) separates the interface from the implementation.
- **Decentralization** , the team chooses the technology and pace of work for its own domain.
- **Independent delivery** , each microfrontend is published without waiting for the others.
- **Fault isolation** , the failure of one part does not bring down the whole, but is hidden or replaced.
- **High observability** , the error must be traceable through a chain of independent parts.

The difference between a microfrontend and a component is not in size, but in the dividing criterion: a component solves a technical reuse problem and exposes an interface that the container (*host*)

knows in detail, while a microfrontend fully owns the business domain and exposes a reduced set of data needed to understand the context.

Microfrontend architecture also introduces challenges that the component model did not have: shared state, routing, performance monitoring, and appearance consistency. Hence the caution: the architecture is not a universal solution, but is justified only when iterative long-term development, a large development team, a multi -*tenant* application, or a gradual replacement of an existing system require it (Mezzalira, Building Micro-Frontends, 2025). The formulation that applies to this choice, and not only to the choice of architectural style: there is no best architecture, but only the least bad combination of compromises.

3.1 Advantages, disadvantages and application criteria

The autonomy that microfrontend architecture brings is not free and four consequences stand out.

The first is **redundancy** : each team maintains its own transpiling and delivery process , so the same problem is sometimes solved more than once; a critical bug in a library used by multiple teams is not fixed in one place, but by each team separately.

The second is **consistency** : when one team needs data that another has, replication introduces latency, so data can be briefly inconsistent between parts of the same view, a trade-off that yields robustness instead of guaranteed consistency.

The third is **the heterogeneity of technologies** : the freedom to choose a framework makes it difficult for developers to move internally between teams and share practices.

The fourth is **more frontend code** : the isolation that makes each fragment stand-alone necessarily introduces some redundancy *in JavaScript* and *Cascading* Style Sheets (*CSS)* . *Style Sheets*) of code, although this amount does not grow linearly with the number of teams.

These four consequences also determine when architecture makes sense. It comes down to a threshold of team size: the *Two - Pizza Team Rule* , a team that cannot be fed by two large pizzas, about a dozen people, becomes a candidate for splitting. Below that threshold, communication within a team is not a problem to be solved by architecture, and the cost of setup, common namespace rules , and knowledge sharing between teams outweighs the benefit.(Geers, 2020)

Microfrontend architecture, moreover, works best on the web: a natively open platform where parts are downloaded and modified independently, unlike native mobile applications , which are published as a single whole. It is also not a good choice when the domain changes shape frequently and substantially (business model pivot), because then the reorganization of microfrontend boundaries

introduces more friction than the architecture saves. The absence of a communication problem in a small team and the frequent change of domain are, therefore, two separate reasons for the same conclusion, that a microfrontend architecture is not suitable here.

3.2 Defining a limited context

Before deciding how to build microfrontends, it is necessary to decide what a microfrontend *is* , where the business domain it supports begins and ends. For this reason, it is useful to apply domain- *driven development* . *design , DDD*), although *DDD* is not originally a frontend technique.

DDD distinguishes three types of subdomains: **key** (e.g. for a video streaming platform , content catalog), supporting **(** e.g., voting or rating system) and *generic* **(** e.g., login and registration), according to how directly they bring value, an important distinction because a key subdomain deserves a more experienced team and quick feedback, while a generic one is often solved with a ready-made solution.

Above the subdomain is the *bounded* **context** . *context*): a logical boundary that hides the implementation and exposes only the contract through which the domain is used. In the new system, the subdomain and the bounded context most often coincide, thus the vertical division (3.3) is directly mapped to one bounded context per team.

The rule for recognizing a well-placed boundary is the same rule from Chapter 3, expressed in reverse: if a microfrontend requires more than a session token and a context identifier to function, the boundary is probably not well-placed, not that the container needs to provide it with more data. The second rule is temporal: the boundary should be delayed until the last moment. development when there is enough data about how users actually use the system, rather than presuming a division (Mezzalira, Building Micro-Frontends, 2025).

This section defines the instrument; its application to a specific domain, identifying subdomains and bounded contexts, is the subject of section 10.4.

3.3 Microfrontend architecture type space

The first axis is **compilation time** : static, at compile time, or dynamic, at run time. The static approach gives a faster application at the cost that any change to the microfrontend requires a recompilation of the whole; the dynamic approach (example: *Module Federation*) allows independent delivery at the cost of greater fragility and more complex version control.

The second axis is **the place of assembly** : on the server or on the client. Server-side assembly gives the user a finished document without the cost of the browser, but requires

a complex infrastructure for scaling; client-side assembly (example: *single-spa*) gives the most flexibility in technology choice, at the cost of the time it takes the browser to assemble the pieces.

The third axis is **the type of division** : horizontal, multiple microfrontends appear on the screen, vertical, one microfrontend per screen.(Mezzalira, Micro-frontends in context, 2020)

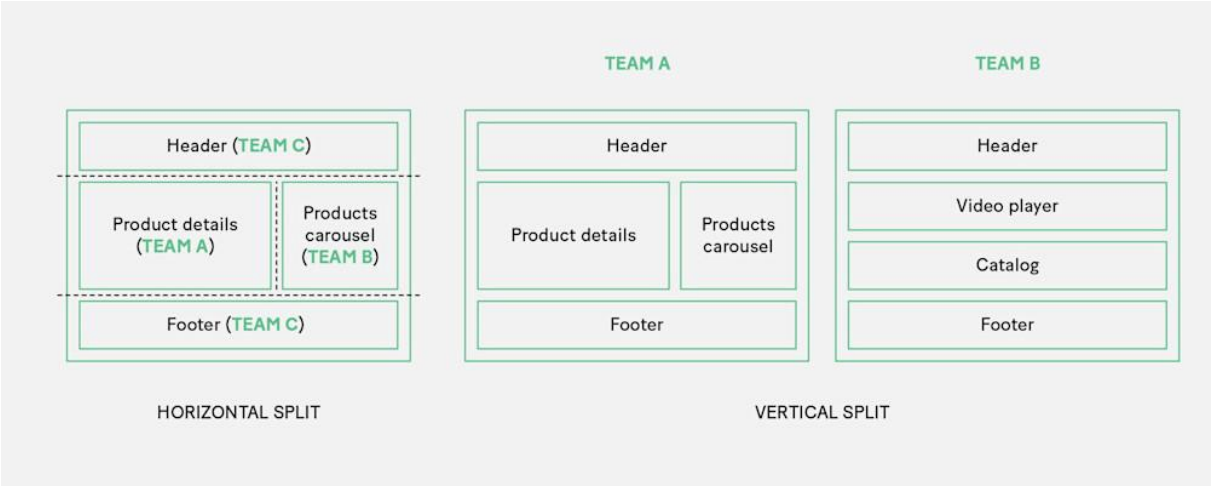
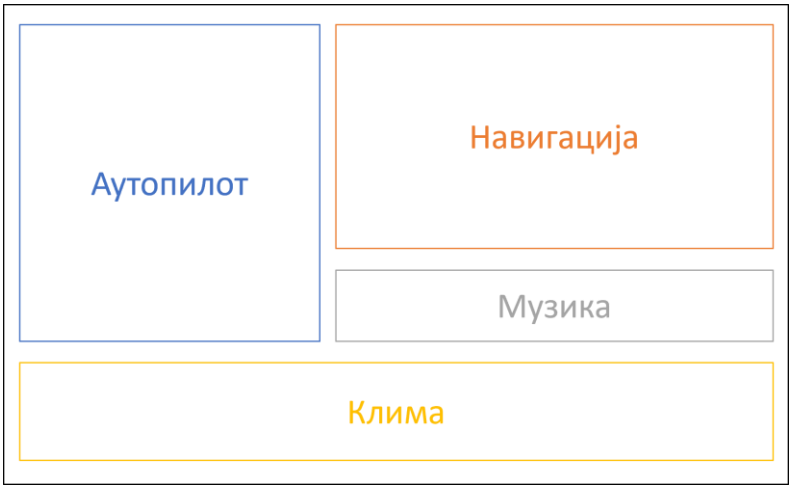


Figure 3-1Horizontal versus vertical type of division(Mezzalira, Micro-frontends in context, 2020)

Horizontal partitioning is a style in which multiple different microfrontend applications are displayed on the same view/screen. This approach allows multiple development teams to influence the same screen by making changes to their respective applications. This also adds flexibility, considering that a single microfrontend can appear on multiple screens.

An example of horizontal division is given through the car's multimedia interface where on the home screen you can see most of the functions the system provides, each on its own part of the screen.



Picture 3-2 Example of horizontal division on a car multimedia system

Vertical partitioning is a partitioning in which the user can see only one microfrontend on the screen. This allows one view to have only one application, which is practical when dividing into business domains (within *Domain Driven Design - DDD*). An example is also given through the image of a

car multimedia system, but this time with an older system where pressing a button on the dashboard can open only one functionality at a time.

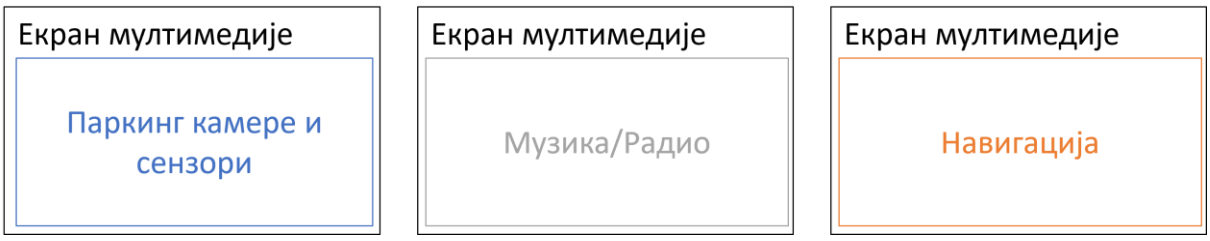


Figure 3-3Example of vertical division on a car multimedia system

3.4 Team boundaries and Conway's Law

Microfrontend architecture assumes teams that need to be demarcated, and Conway *'s law* states that the structure of a system will copy the communication structure of the organization that designs it (Conway, 1968). For vertical division, this means: team boundaries should follow application boundaries, not vice versa.

Three processes help to find the boundary: domain-driven development (3.2), user focus (user journey stages, "what the user is looking to do"), and a review of the existing screen structure. None of these are definitive, so it is advisable to move the boundary only when user behavior data confirms it, not in advance.

4 Composition, integration and routing

Chapter 3 answered the question of what a microfrontend is; this chapter answers the other two questions: how the parts are assembled into a single screen and how the user is guided between them. Three concepts are distinguished that are often confused in practice: **transition technique** (how the user moves from one page to another), **composition technique** (how multiple microfrontends appear on the same screen at the same time), and **high-level architecture** (the combination of the first two with the decision on where to build the view).

4.1 Link transition and server-side routing

The simplest form of integration is a hyperlink , and its value is measured by the definition of coupling that (Geers, 2020)*it* introduces for relationships between teams: coupling is the amount of knowledge that one team must have about the other. With a link, that amount is reduced to a single address, the team that places the link knows neither the framework in which the target application is written nor how it delivers its files, and the browser does not establish any connection between the two applications, so the technical isolation is complete without anyone's effort.

The price of this independence is the accepted redundancy: each application supplies its own styles, thus avoiding a central file that would bind both teams together (Geers, 2020). The address is a contract, and the parameters from the address (*query) parameters*) becomes richer and more fragile, because every change to it must be respected by both sides.

In the example of a fictitious system, we can see that the first application was delivered to the domain *www.pocetna.fon.bg.ac.rs* .

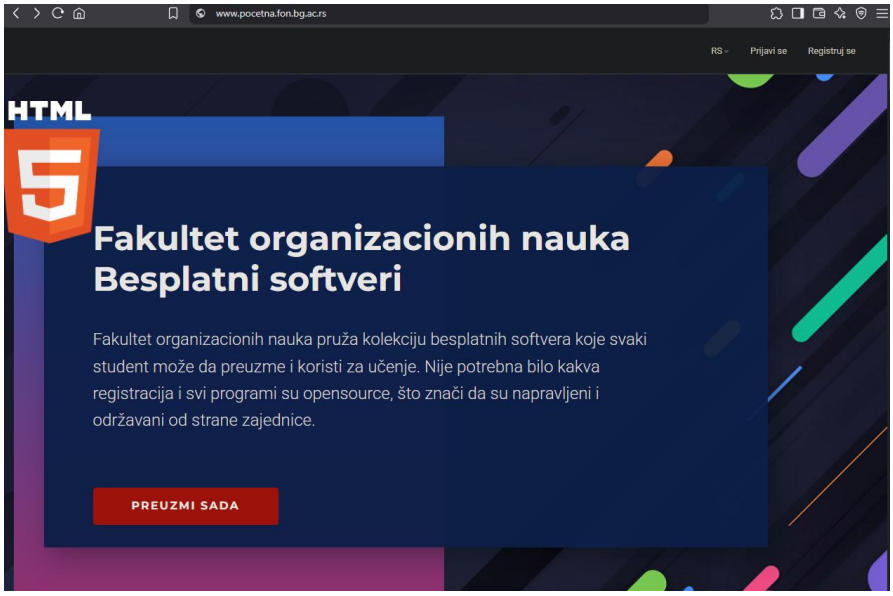


Figure 4-1Home page of the fictitious FON application

If the user clicks on the button labeled "DOWNLOAD NOW", the internet browser will take them to the next page, which is under a different domain (for example, the fictitious domain *www.preuzimanja.fon.bg.ac.rs*). Each of these applications separately fetches all the necessary materials, files and loads them completely independently. Therefore, the performance of this approach is not great. Also, the user will see the page reload and this approach loses all the benefits of single-page applications and the advantages that this modern architecture brought. However, this achieves the perfect vertical division, which was previously mentioned in the microfrontend architecture.

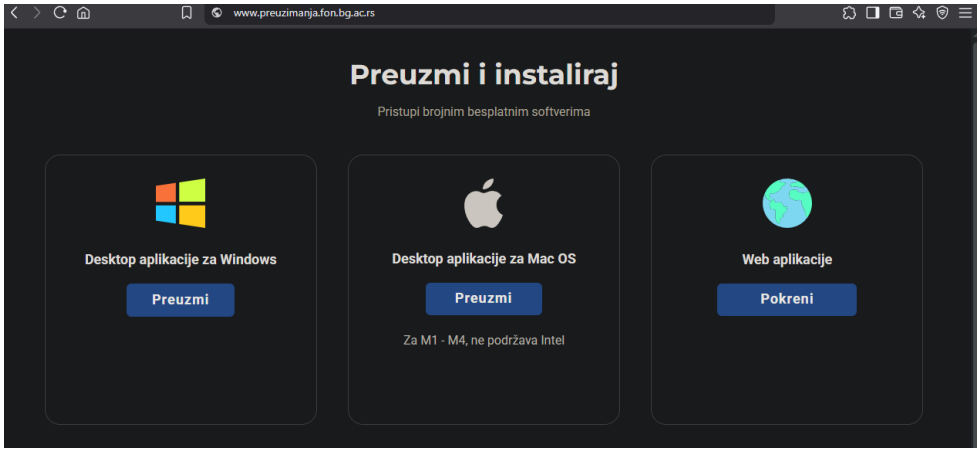


Figure 4-2FON fictitious application download page

The code to implement this approach is extremely simple. You just need to set *an href attribute on the a* tag (or any tag that supports hyperlinks) that will lead to another application.

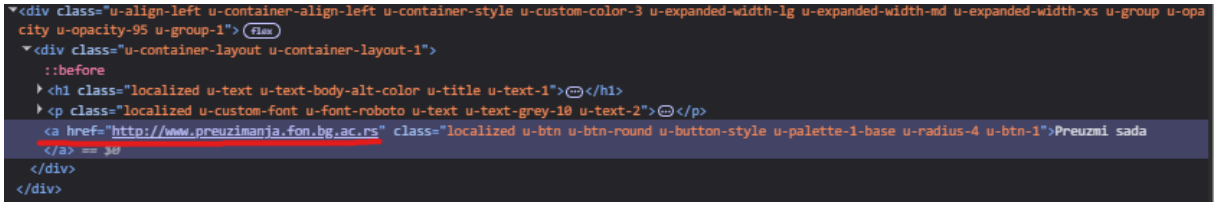


Figure 4-3Redirecting pages via a href sadness

What a link can't hide is a transition: every transition is **a hard navigation** (hard *navigation*) , in which the browser discards the document, state, and downloaded files, thus losing the advantages of single-page applications described in 2. 3 . What it achieves for free is perfect vertical division, each part being an entire page owned by one team.

First hope of construction Such a system would not change the transition technique, but its visibility: to make all applications available under the same domain, a frontend *proxy* without business logic is introduced, whose execution either assigns path prefixes to teams or reads a table of rules published by the applications themselves. One domain resolves cookies, the restriction of sharing resources between origins (*Cross - Origin Resources Sharing - CORS*) and the need for multiple records within

the Domain Name System (*DNS*) and certificates, introduces a place for caching, and improves search engine indexing. Both drawbacks stem from the same property: the proxy is a new shared infrastructure that someone has to maintain, and **a single critical point** whose failure brings down all teams at once, which is repeated in 4.5 for the client-side wrapper.

4.2 Server-side composition

The previous section brought multiple applications under a single domain, but not onto a single screen, because the broker assigns a single owner to a single request. **Server-side composition** takes it a step further: the document is assembled from fragments returned by different teams before it reaches the browser (Geers, 2020).

The simplest implementation is *server - side* **inclusions** . *Includes - SSI*), a standard feature of most web servers: the team owner writes an include directive in their tag, and the server replaces it in a second step with content pulled from the fragment address. The consequence of this two-step assembly is that the time to the first byte (*time that first byte - TTFB*) is no longer a property of a single application, it is the sum of the time it takes for the owner to return its own tag and the time of the slowest fragment in it (Geers, 2020). . Edge **connections** *Side Includes - ESI*) move the same idea to a content delivery network *(CDN*) , bringing the compilation closer to the user, but the standard wait limit per fragment is unknown, as it depends on the service provider.(Mezzalira, Micro-frontends in context, 2020)

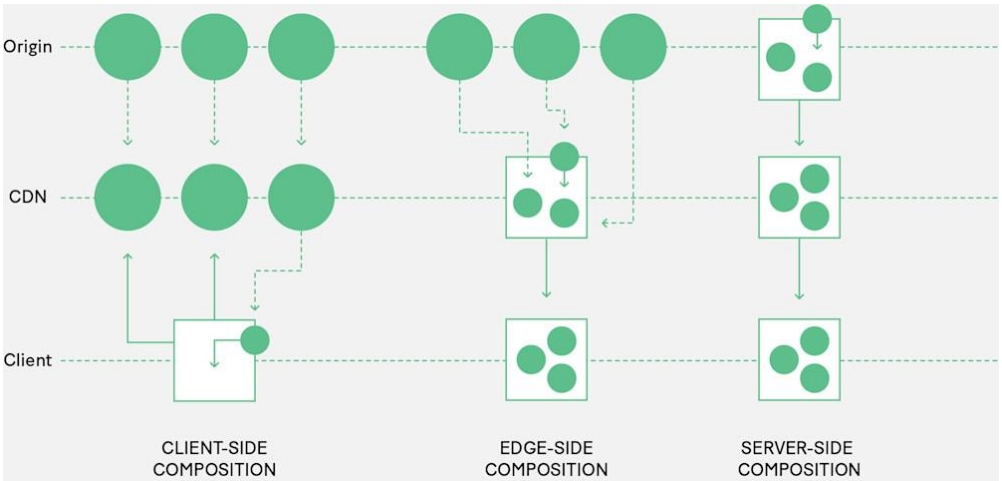


Figure 4-4Comparison of three types of composition(Mezzalira, Micro-frontends in context, 2020)

4.3 Client-side composition

In this type of composition, the client loads multiple microfrontends directly from *a CDN* or server point, and thus the client layer is responsible for triggering the access points of each of the fragments, usually through JavaScript or HTML files.

The three techniques in this section lie on one axis: the more isolation they gain, the more interactivity they pay, and vice versa. What they all have in common is that the compilation is done by the browser, thus occupying client-side and dynamic time (3.3).

The iframe element solves isolation radically: it opens a completely new context in the existing browser context, with its own global objects, DOM tree, and file loading, so there is no overlap in any way (Wright, 2022). The isolation is the highest among the techniques in this chapter, the implementation is the simplest, and the browser support is practically perfect. However, the price is not only in performance: since the content in the new context cannot affect the height of the element in the parent document, the parent must know the height in advance (Geers, 2020); before that, teams only had to know the address, but now they must also know the height of its content, which extends the contract where it is not expected. In addition, each instance carries the cost of a new context, restricts layout, makes accessibility difficult, and is not indexed as part of the parent page, which is why the justified cases are those in which isolation is a requirement, not a consequence.

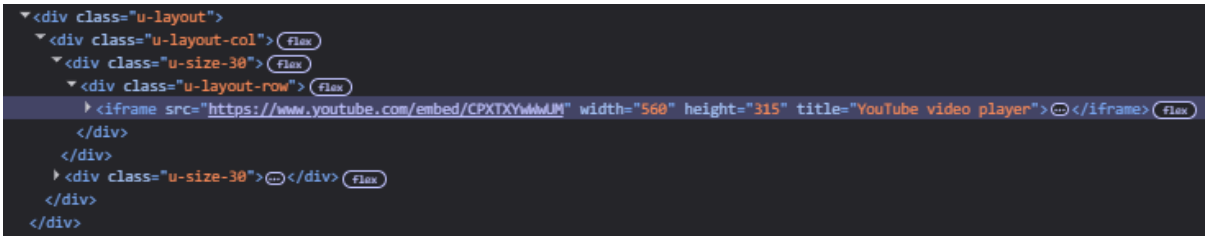


Figure 4-5Using iframe tag in an HTML document

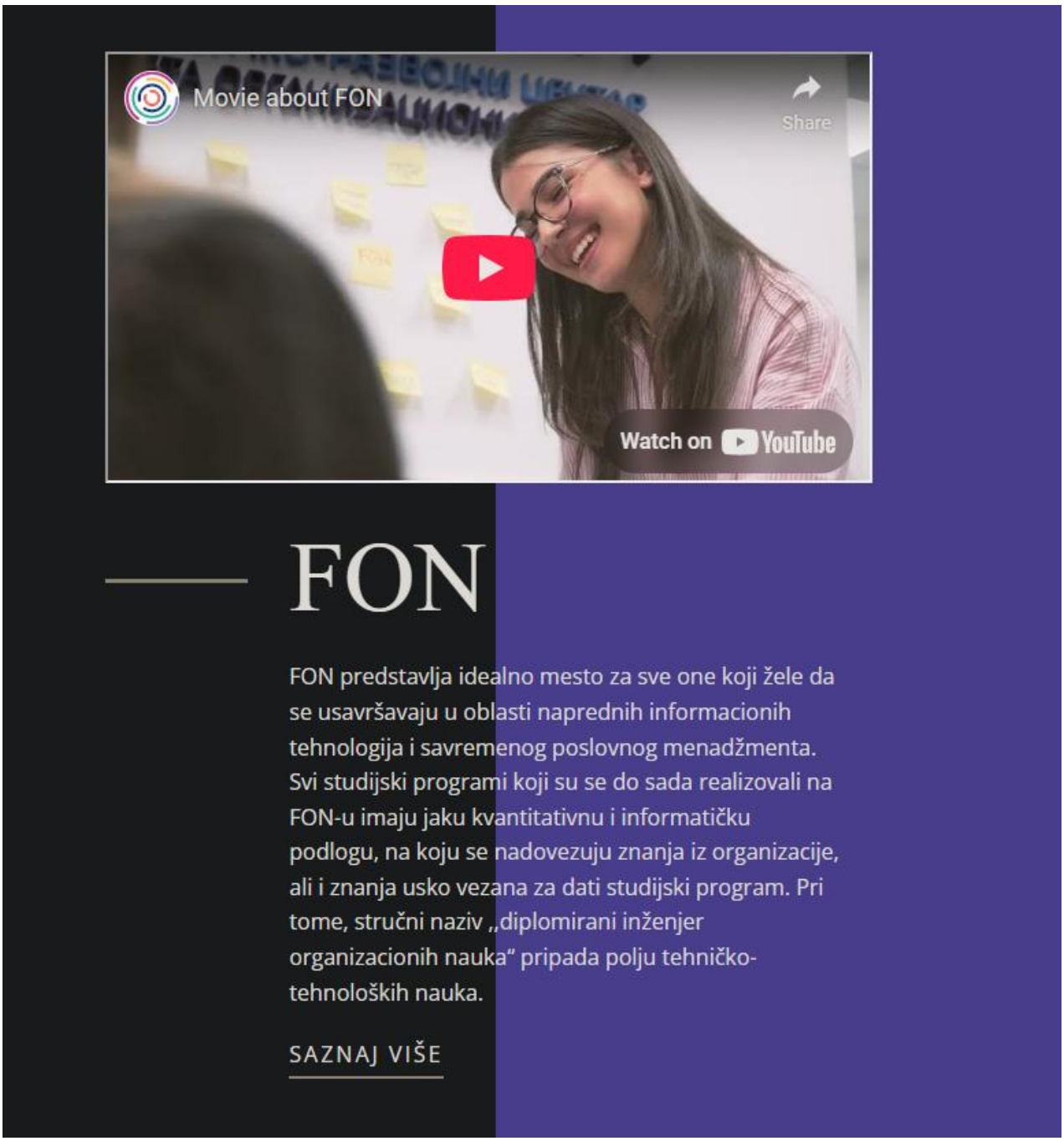


Figure 4-6iframe display components on the page

Figure 4-7 DOM tree inside an iframe sadness

Web components , whose three technologies are defined in 2. 5 , do not pay this price, and they retain some of the isolation. As a composition technique, their contribution is not in the component model, but in that the fragment contract becomes declarative and complete and reduced to three pieces of information: the tag name, the attributes that are passed to the fragment to provide context, and the addresses of the files it requires (Geers, 2020). The mandatory hyphen in the custom element name serves a dual purpose, as the standard uses it to separate custom elements from built-in elements, and the [team]-[fragment] convention also carries ownership. Lifecycle methods become an integration interface, the same for all teams regardless of the underlying framework, thus reducing the pressure on inter-team agreements; the limitation is that they are synchronous, so they do not allow the fragment to delay initialization as long as it wants. The isolation is asymmetric, the DOM "shadows" the structure and styles, while there is no equivalent for *JavaScript , which is the subject of 8.* 2 . Two points prevail: it is a web standard that does not change backwards, which is no small matter for long-lived systems, while the disadvantage remains that the fragment requires *JavaScript* on the client, since there is no standard declarative form of display on the server (Geers, 2020).

```
class MyElement extends HTMLElement {
    static get observedAttributes ( ) {
        return [ ' my-attribute ' ]; // Specifying attributes that will to be
followed
}

    constructor ( ) {
        great ( );
        // Initialization element
}

    connectedCallback ( ) {
        // Code to be executed when an element is added to the DOM
}

    disconnectedCallback ( ) {
        // Code to be executed when the element is removed from DOM
}

    attributeChangedCallback ( name : string , oldValue : string | null , newValue
: string | null ) {
        if ( name === ' my-attribute ' ) {
            // Code to be executed when the attribute " my-attribute " changes
}
}
}

customElements . define ( 'my-element' , MyElement );
```

```
<! DOCTYPE html >
< html lang = " en " >
  <head>
    < target charset = "UTF-8" />
    < title > Fon Konkurs </ title >
  </head>

  <body>
    < phone -button label = " Sign up for FON " >
      < span slot = "icon" > ★ </ span >
    </ background -button >

    <script>
      class PhoneButton extends HTMLElement {
        constructor ( ) {
          great ( );

          this . attachShadow ({ mode : "open " } );

          this . shadowRoot . innerHTML = `
<button>
<slot name="icon"></slot>
<span id="label"></span>
</button>
` ;
}

        static get observedAttributes ( ) {
          return [ "label" ];
}

        attributeChangedCallback ( name , oldValue , newValue ) {
          if ( name === "label" ) {
            this . shadowRoot . getElementById ( "label" ). textContent = newValue
;
}
}

        connectedCallback ( ) {
          this . clickHandler = () => {
            this . dispatchEvent (
              new CustomEvent ( " fon -click" , {
details : { message : " You have registered for FON ! " } ,
bubbles : true ,
```

```
composed : true
})
);
};

          this . shadowRoot
            . querySelector ( "button" )
            . addEventListener ( "click" , this .clickHandler );
}

        disconnectedCallback ( ) {
          this . shadowRoot
            . querySelector ( "button" )
            . removeEventListener ( "click" , this .clickHandler );
}
}

      customElements . define ( " fon -button" , FonButton );

      document
        . querySelector ( " fon -button" )
        . addEventListener ( " fon -click" , ( e ) => {
          alert ( e .detail .message );
} );
    </script>
  </body>
</html>
```

Figure 4-8Custom button inside DOM "shadowed"

single-spa takes the same idea from the fragment level to the entire application level: it doesn't assemble parts of a single screen, but rather multiple single-page applications into one. The contract here is *a JavaScript* object with three asynchronous functions, *bootstrap , mount* and *unmount* , and the wrapper registers each application by calling *registerApplication* , passing it a name, a function to load the code, and a function activation that decides from the address whether an application should be rendered (Geers, 2020). Three features have consequences for 4.5: code loading is on-demand, not in advance; each application gets its own DOM element to render into; and **multiple applications can be active at the same time** , which is what a persistent header or navigation microfrontend makes feasible. Asynchronous lifecycle functions are an advantage over synchronous Web Component

methods, but also a liability: since all applications share a single document, forgotten event listeners, timers , and global variables become someone else's problem.

```
export async function bootstrap ( props ) { ... }
export async function mount ( props ) { ... }
export async function unmount ( props ) { ... }

singleSpa . registerApplication ({
  name : "app1" ,
  activeWhen ,
  app ,
  customProps : ( name , location ) => { authToken : "d83jD63UdZ6RS6f70D0 " }
});

singleSpa.start ( ) ;
```

4.4 Modules Federation and dependency sharing

DOM "shadows" styles; there is no *JavaScript* equivalent. Given that *JavaScript* in a single document is shared by default, it can be concluded that sharing dependencies ceases to be a decision that can be postponed: libraries will either be split or shared, and both outcomes have a cost. (Rappl, The Art of Micro Frontends, 2024)

There are three different models:

- In a **shared-less model** , each microfrontend includes everything it needs in its own package, which gives the greatest independence with the greatest throughput, the same compromise of accepted redundancy from 4.1.
- In the **central sharing model** , libraries are provided by the wrapper, and microfrontends become small; the cost is more serious than it seems, because the team that maintains the wrapper, therefore, takes on the technical decisions of all microfrontends, and the model scales poorly with their number.
- In the **distributed sharing model** , each microfrontend carries its own copy, but at runtime it negotiates with the others and uses the one that is already loaded (Rappl, The Art of Micro Frontends, 2024).

Modules Federation is a third model implementation. It is configured on both sides: the microfrontend publishes the container name and the list of modules it exposes, the wrapper lists the containers it loads, and both sides list the libraries they offer for sharing (Rappl, The Art of Micro Frontends, 2024). The module names thus published have the nature of a contract and must not be renamed without prior coordination between the teams, the same form of silent failure as the address in 4.1, only moved from the network to the build phase.

Since containers can be nested, and a microfrontend can be a wrapper for others, the line between a wrapper and a loaded part becomes thin, which is why it is recommended to keep the sharing **one-way** : the wrapper does not expose its own modules to the loaded part, because two-way sharing creates a circular dependency in which a change in one microfrontend can break the other, thus breaking the autonomy for which the sharing was introduced, the same notion of coupling from 2. 5 , expressed over dependencies. A standardized alternative exists and is based on ECMAScript modules and import maps (engl. *import map*) instead of a single compiler mechanism.

```
/ root-config
├── src /
│ ├── index.ejs           # HTML entry
│ ├── root-config.js # Single -SPA configuration
│ └── import - map .json    # Shared dependency map
└── package.json
```

```
{
"imports": {
"@ mfe / navbar":"http ://localhost:8500/navbar.js",
"@ mfe /dashboard": "http://localhost:8600/dashboard.js",
"react": "https://cdn.jsdelivr.net/npm/react@18.2.0/umd/react.production.min.js",
" react - dom ": "https://cdn.jsdelivr.net/npm/react-dom@18.2.0/umd/react-
dom.production.min.js"
}
}
< script type = " importmap " >
{
"imports": {
" lodash ": "https://cdn.jsdelivr.net/ npm /lodash@4.17.21/lodash.min.js"
}
}
</script>
```

The mechanism, however, does not answer the question of *what* should be shared, and practice suggests a default decision not to share anything. The rationale is the asymmetry of consequences: wrong non-sharing hurts less than wrong sharing. This basis is only deviated from if the following criteria are met:

- the dependency is **of considerable size** , approximately at least 15 to 30 kB , whereby the measure should be weighed against the transfer speed available to the average target user;
- it is used **by at least two** microfrontends;

- **there are no more widely used versions of it** that introduce incompatible changes among themselves;
- plays **an important role in displaying** components;
- **It's technical, not domain specific** .

The last criterion is marked as controversial: the need to share a domain module is an indicator of insufficient domain separation. Business code should not be shared, for auxiliary functions it is recommended separation is required, and if sharing must still be practiced, then it should be separated as a versioned package at build time, not as a dependency at runtime.

4.5 Wrapper application and client-side routing

Application wrapper (engl. *application) shell*) is the parent application of all microfrontends: every request comes to it, it picks up the part the user is looking for and displays it in the body of the document(Geers, 2020) (Chen, 2021). Since this is shared code, it should be kept as simple as possible and without business logic. The wrapper has four essential jobs: to provide a shared HTML document, to translate addresses into team pages, to display the corresponding page, and to deinitialize the previous one and initialize the next one when navigating.

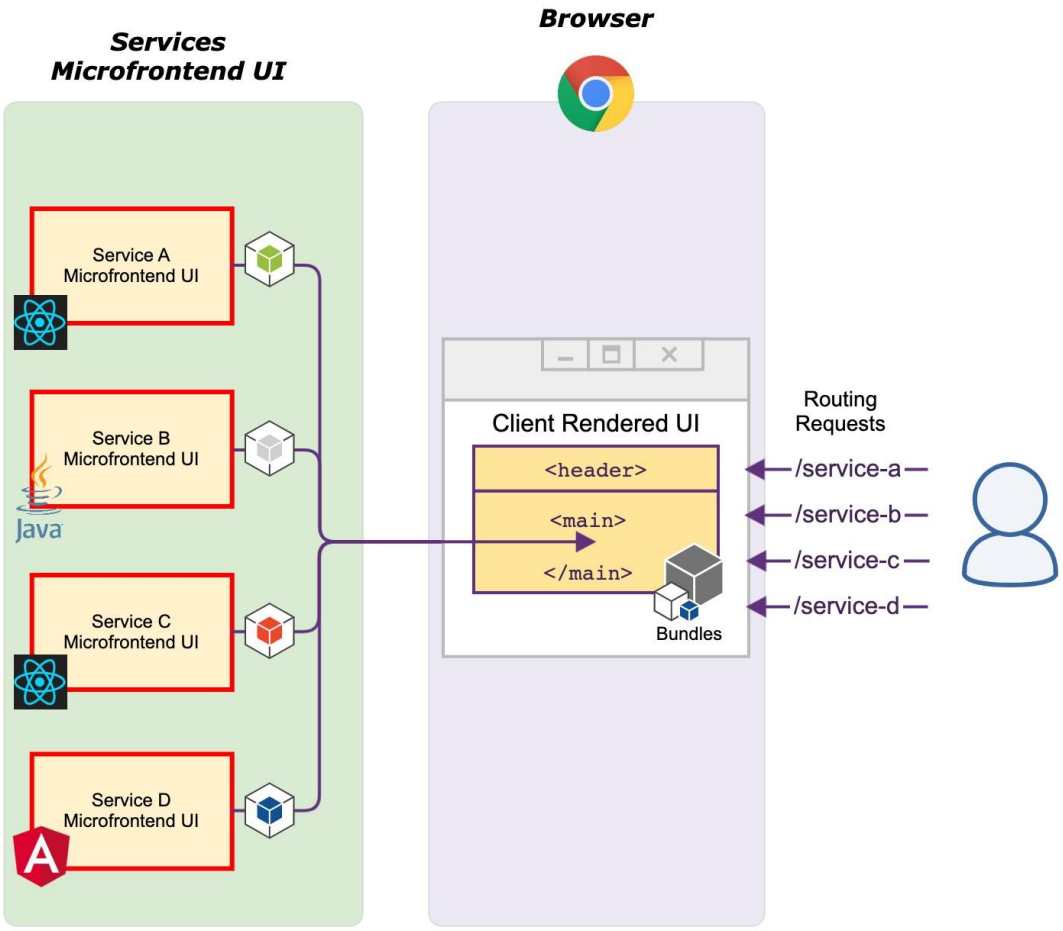


Figure 4-9 The wrapper application pulls microfrontends based on requests(Chen, 2021)

Routing at one level is the simplest implementation: the wrapper keeps a table that assigns a responsible team component to each address, and by generally listening for clicks on the document, it downloads the links and writes the new address to the browser history. The disadvantage is structural: the wrapper must know all the application addresses, so a team that wants to change an existing one or add a new one must change and re-deliver the wrapper, which puts the coupling at its worst, because the wrapper should be as neutral as possible from an infrastructure perspective(Geers, 2020) (Chen, 2021).

Routing This shortcoming is eliminated **on two levels** : the wrapper reads only the prefix from the address and decides which team is responsible, and the router of the team itself determines which page to display from the entire address. using soft **navigation** . This results in multiple single-page applications combined into one, with the wrapper only changing if a new team is introduced or if the prefix of an existing one is changed (*Geers* , 2020).

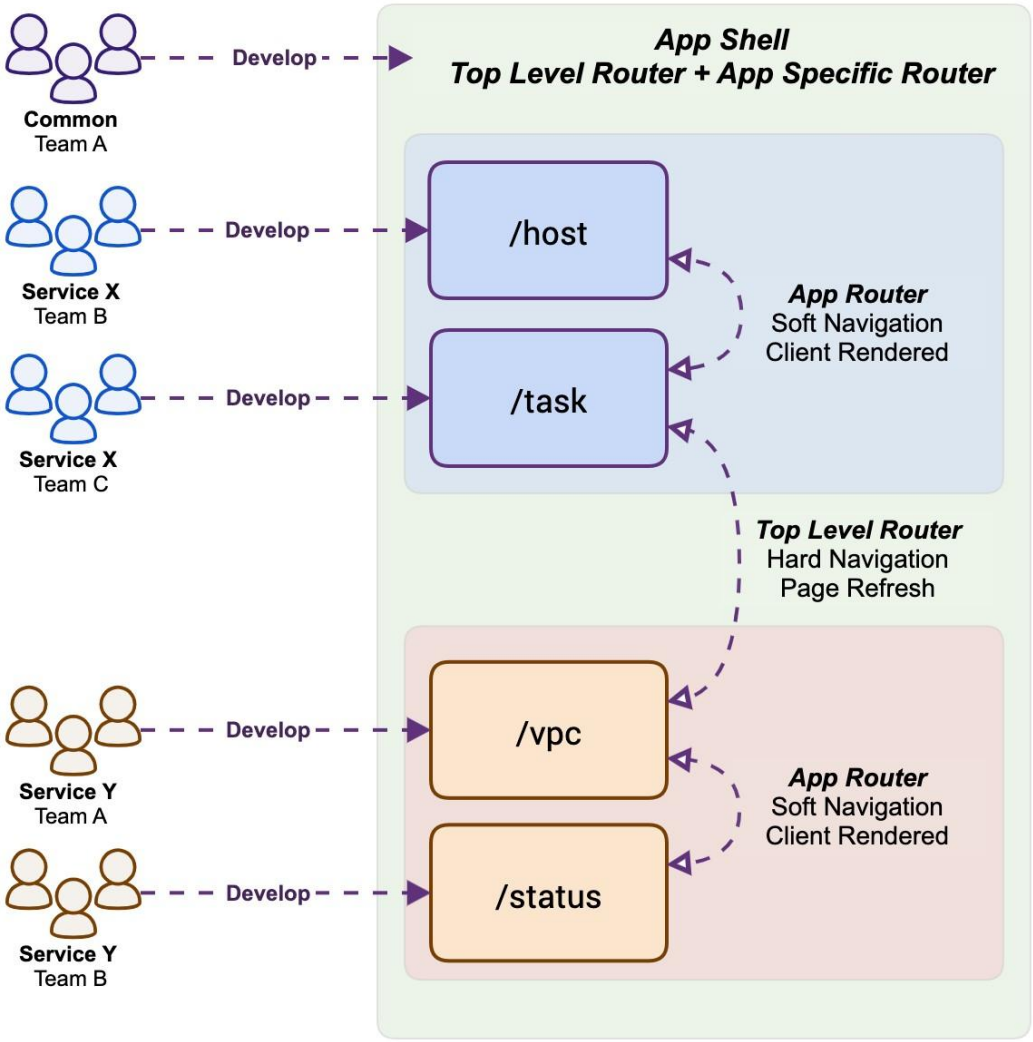


Figure 4-10One- and two-level routing(Chen, 2021)

Routing on two levels where the wrapper resolves only the prefix:

```
const routes = {
  "/product/" : "decide-pages" ,
  "/checkout/" : "checkout-pages" ,
  "/" : "inspire-pages" ,
};

function findComponentName ( pathname : string ) : string | undefined {
  const prefix = Object . keys ( routes ). find (( key ) => pathname . startsWith (
key ));
  return routes [ prefix ];
}
```

The contract between the wrapper and the teams thus gains a normative rule: business logic belongs in the teams' code, and a reliable indicator that the coupling is too tight is that the delivery of one functionality by one team requires a change to the wrapper (Geers, 2020). The most explicit form of this rule is that **state should not be moved to the wrapper** : a shared state container equals a shared

database, and this is precisely the coupling for which the distributed style was introduced (2. 2). Mechanisms that solve this without shared state are the subject of Chapter 5.

5 Communication and status management

How to achieve communication between microfrontends without tight coupling is a major issue in distributed system design. With well-defined team boundaries, the need for extensive browser-based communication should be minimal, since the user completes the task in a single team's interface, and communication is required mainly at the drop-off points, and most of this communication between teams occurs over addresses. The volume of communication is therefore a measure of the quality of the boundaries: when new functionality requires two teams to send data back and forth to each other, it is a reliable indicator that the boundaries are not well-defined, and a fix is no better channel than shifting responsibility or expanding the scope of one of the domains (Geers, 2020).

Here, the mechanisms are organized along two axes: by **the topology of the participants** (containment relationship, adjacency relationship, or no relationship at all) and by **the concurrency condition** , whether both parties are alive in the same document at the same time. The chapter concludes with Table 5-1, which brings these axes together into a decision process.

5.1 Three directions: parent→fragment , fragment→parent , fragment↔fragment

When multiple microfrontends share a single screen, three directions of exchange are distinguished, and each has its own natural mechanism.

Parent → fragment. The parent page sends data to the fragment, which receives it as **an attribute** . The fragment responds by pre-registering which attributes it observes and processing any changes to them. The flow is unidirectional and follows a pattern known as "props down, events up" (*props up). down , events up*): the changed state of the parent is passed down by attributes, and the notification is returned by an event.

Picture 5-1 Parent and fragment communication

Fragment → parent. The mechanism is not studied here, but the prohibition that precedes it. The team that owns the fragment technically has access to the entire document and can edit the part of the page that does not belong to it, and that is exactly what it is not allowed to do. The pure form is a contract: the fragment publishes **a custom event** (English *Custom Event*), and the page owner reacts to it in his part.

Fragment ↔ fragment. When two fragments are not in a containing relationship, i.e. not within each other but at the same level of the hierarchy, then there are three ways of communication.

- The first, to find the fragment itself, the second in the document and **directly** call its function, and is explicitly rejected: direct reliance on another element introduces a tight coupling, the screen composition can no longer be changed, and removing a fragment has consequences that no one has foreseen.
- The second is **mediation through parents** , merging the first two directions: the solution is clean and the flow of exchange is explicitly modeled, but any change in communication then has to be implemented by two teams.
- The third is **the event highway** , where fragments publish to a common channel and subscribers react, so the parent does not even need to know that the exchange exists, which minimizes coupling and is the subject of Section 5.2.

5.2 Event Bus: Custom Events and *Broadcast Channel API*

The browser event bus does not require any libraries. **A custom event** is an interface used to attach custom data to an event generated by an application. and are supported by all browsers. The event is posted **on the element** , rather than directly on the global window object, for two reasons: the origin of the event is preserved, which is necessary when there are multiple instances of the same fragment on the screen. The specific element then registers and listens for the selected events. The posting and listening elements can be in different fragments, allowing communication between microfrontends in a horizontal split. (Mezzalira, Micro-frontends in context, 2020)

```
// create custom events
const foundcat = new CustomEvent ( " Pet Found " , {
  details : { name : " cat " } ,
});
const foundDog = new CustomEvent ( " Pet Found " , {
  details : { name : "dog " } ,
});

const element = document . createElement ( "div" ); // create element

element . addEventListener ( // event listener registration
```

```
" foundAnimal " ,
( e ) => console . log ( e . detail . name )
);

// broadcast event
element . dispatchEvent ( foundMac );
element . dispatchEvent ( foundDog );
// " cat " and "dog" are printed
```

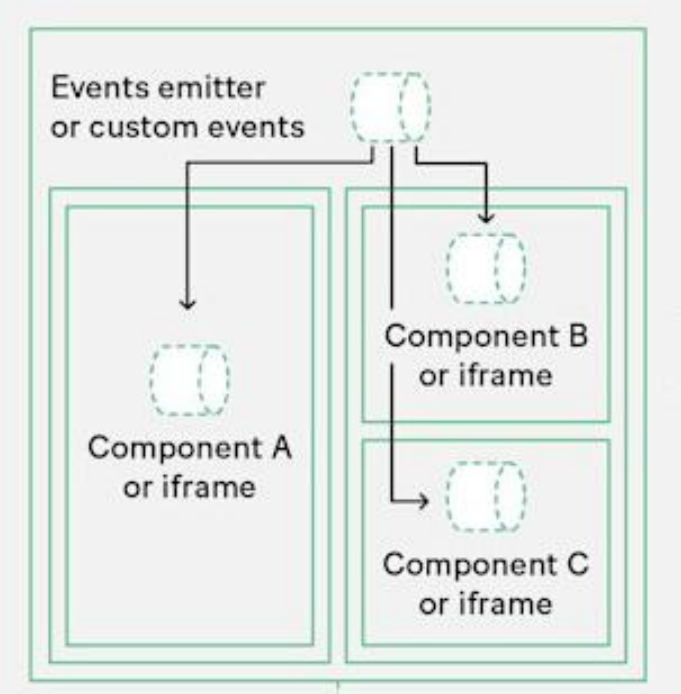


Figure 5-2Fragment communication using events(Mezzalira, Micro-frontends in context, 2020)

Such a mechanism is considered the most important communication pattern among microfrontends, also because of its resilience: events are loosely coupled by nature and do not break if the handler is not registered or if the one who usually publishes them is not there, and their very form indicates that their delivery is not guaranteed in time. The most reliable and fastest way to introduce them is precisely the standard DOM mechanism.

Exchanging data structures via events introduces additional coupling, the payload should be kept to a minimum, and the event should be used for notification rather than data transfer; models and domain objects remain within the boundaries of a single team. Only the minimum amount needed for integration should be exchanged. The practical consequence for contract design is that the event carries an identifier, and the receiver itself retrieves what it needs from its server layer (5.6).

Broadcast The Channel API is another standardized form of publish-subscribe, but with a different reach: a channel is opened by name, messages are sent and received using the methods of that channel,

and it reaches **other documents of the same origin** (Geers , 2020) (Nwamba, 2024). This is its decisive advantage over DOM events, which do not leave their own document, and the reason why it is not chosen between two fragments on the same screen, except when the same user has multiple tabs open. It is preferable to use named channels as boundaries: a general channel for exchange between teams and a separate channel for internal exchange within a team, where a richer structure is also allowed, a separation that reduces the risk of unintentional coupling.

```
const channel = new BroadcastChannel ( 'auth' ); // create channel

channel . postMessage ({ // send message
  type : 'AUTH_UPDATE' ,
  payload : { user : { name : 'Veljko Blagojević ' } , isAuthenticated : true } ,
});

channel . addEventListener ( // event listener registration
  'message' ,
  event => console . log ( event . data ) // whatever was passed in postMessage
);

channel . addEventListener ( // register event listener
  ' messageerror ' ,
  event => console . error ( 'Error while deserialization messages :' , event )
);

channel . close (); // close the channel
```

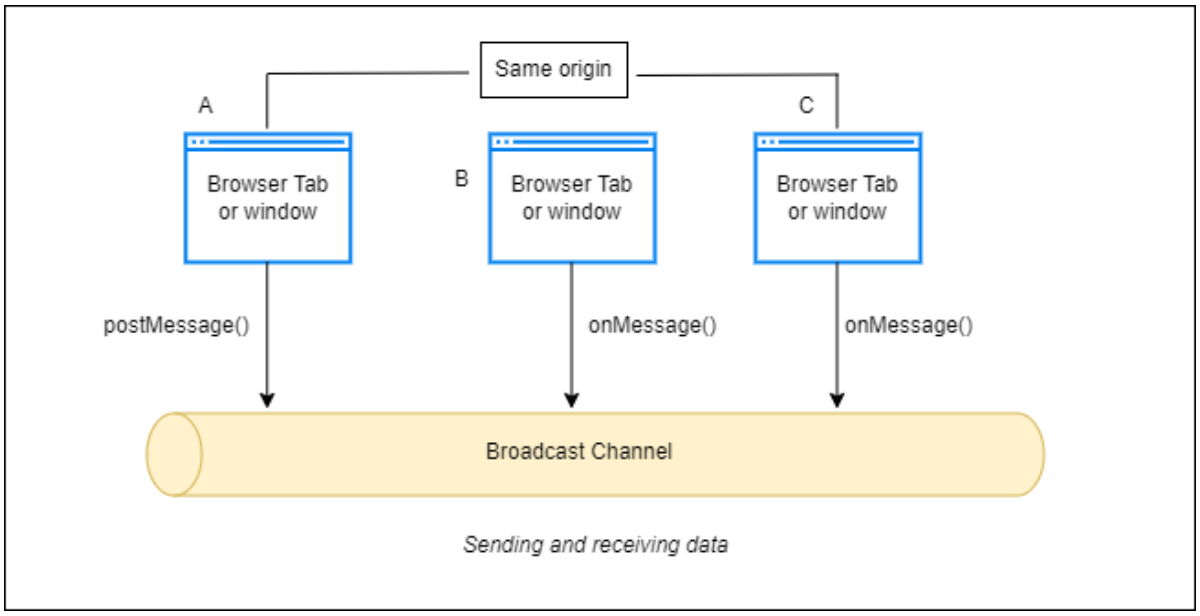


Figure 5-3Broadcast Channel

5.3 Transfer via address

The introductory section of the chapter already announced that most of the exchange between teams takes place via addresses, at handover points. This mechanism is distinguished from all the ones

presented so far by the requirement of concurrency. Element attributes, the custom event that is raised, and the event bus assume that both parties are mounted in the same document at the same time, and the Broadcast Channel API assumes that both documents are open. There is, however, a handover where none of these assumptions apply: a cross-boarder handover, where the sender leaves the screen before the receiver arrives. In this case, the address (*Uniform Resource Locator - URL*) , i.e. data stored in the path or in the parameters of the address. Its defining property is that it does not require concurrency, since the recipient can read it whenever it mounts itself.

The rule that Section 4.2 already sets for events, that the payload is the identifier and the recipient retrieves what he needs, applies to the address as well, for two independent reasons. The first is duration: since the address survives the exchange, everything written to it remains in the browser history, so nothing that requires confidentiality must enter the address. The second is form: the address is of limited length and is entirely visible to the user, which excludes large and complex objects from it as transmitted data. What remains is the domain object identifier, never the object itself. The approach where only a simple identifier is sent, and then the recipient is expected to extract a large object based on that unique data is a variation of the Claim Check pattern (Mosyan, 2023).

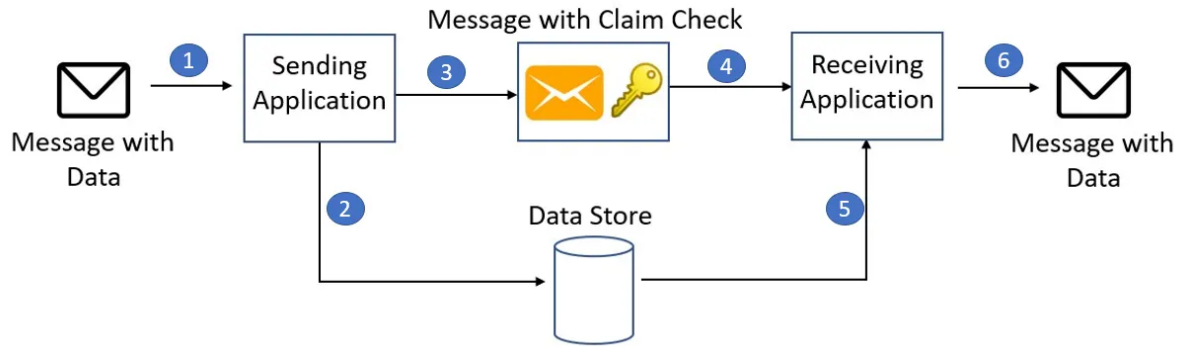


Figure 5-4Claim Check pattern(Mosyan, 2023)

5.4 Global context and authentication

A piece of data is neither a message nor a domain state, but **a context** : the user's language and country, the selected currency, the login status, the environment in which the system is running (Geers, 2020). It stands out because it has three properties that separate it from everything else presented so far: it is needed by everyone, it is read-only by nature, and it represents an infrastructure job that should be solved once for all teams and not repeated in every fragment.

Authentication is a more difficult case because it is not only infrastructural: it also carries business logic, so the question is not technical but a question of boundaries. (Geers, 2020)The answer is derived from the teams' missions, the team in whose domain the login naturally resides becomes the

authentication bearer, publishes a login page or fragment to which the others refer the user, and the login status is then transmitted through established mechanisms such as OAuth and JWT.

State, after all, is what is **not** shared. It is set up so that its team holds its state, because sharing it tightly couples it, makes the system more sensitive, and, more importantly, is regularly abused as a communication channel, with one part writing a value in the hope that the other will notice the change.

5.5 Local storage and cookies

The criterion by which this section is separated from the previous ones is time: it is about what the document must survive.

Web Storage *distinguishes* between two lifespans: local storage survives browser closure, while *session storage Storage*) lasts as long as the tab it was created in. Cookies are an older mechanism with a different cost: they are sent with each request to their origin, which makes them a natural context bearer *when* a page is compiled on the server, but also adds a load to each request.

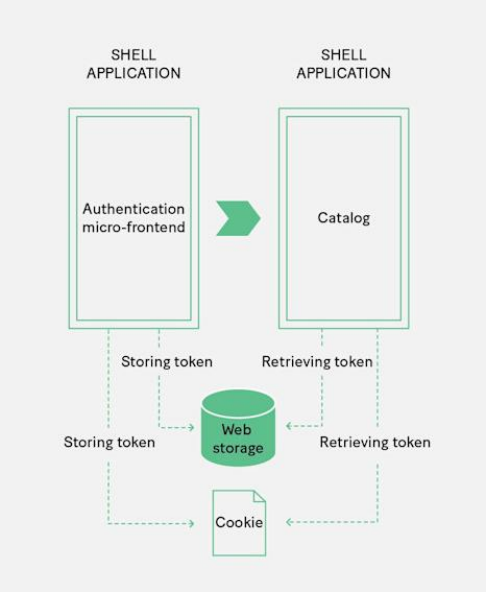


Figure 5-5Web storage and cookies

The discipline that follows from all this is that the repository is **a medium, not a channel** : entering a value does not notify anyone. The one who wants another fragment to react must still announce the event, so the repository and the highway are not substitutes for each other but complement each other, one carrying duration, the other notification.

5.6 Communication with the server layer

There are several approaches to exposing API access points on the server side so that the client can read them. In distributed systems and microfrontend architectures, one of the following three patterns is most often used:

An API **gateway** introduces a single entry point in front of a large number of services and performs routing, verification, and response composition on it, as well as what its real value is: the so-called edge functions, token verification, authorization, call throttling, caching , metering , and logging, in one place, instead of each service repeating them (Mezzalira, Building Micro-Frontends, 2025). The gateway can also hide the internal protocol, so the client remains on the contracts and not on the service technology. The disadvantages are: a single point of failure, which is why the gateway must be run in multiple copies; a bottleneck in management, when each contract change is waiting for the team that owns the gateway; and an additional step in the request path, so the response time is somewhat longer (Ozkaya, 2021).

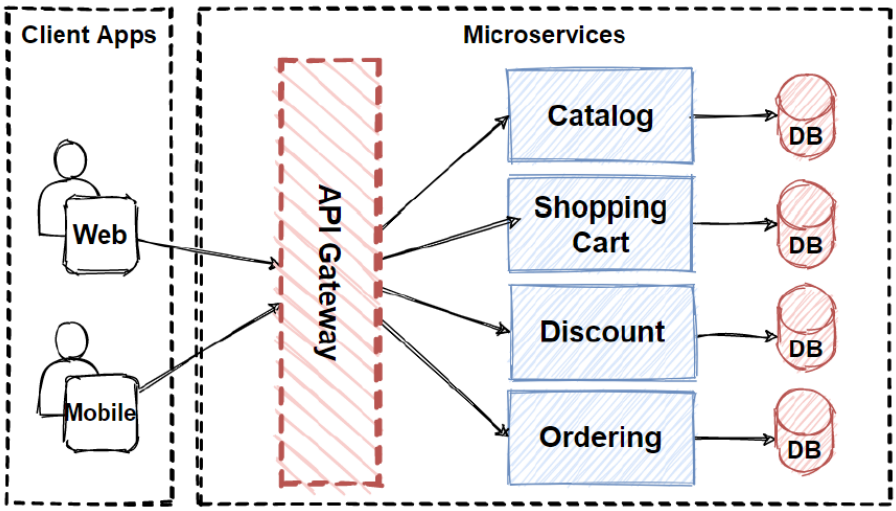


Figure 5-6API gateways in a microservice architecture(Ozkaya, 2021)

Backend for Frontend (BFF), extends the gateway by introducing one entry point **per client type** (Mezzalira, Building Micro-Frontends, 2025)The gain is twofold: the response is assembled from multiple internal calls and returned in a form that is suitable for the display, thus reducing both the number of calls and the amount of unused data, and different platforms get what they really need instead of the lowest common denominator. (Das, 2024). BFF is also a natural center of gradual replacement of monoliths in the *strangler fig pattern. fig*), since it redirects calls to new services in one place as they arise, and can be divided by subdomain instead of by client type, which allows for different scaling of parts (Villavicencio, 2023).

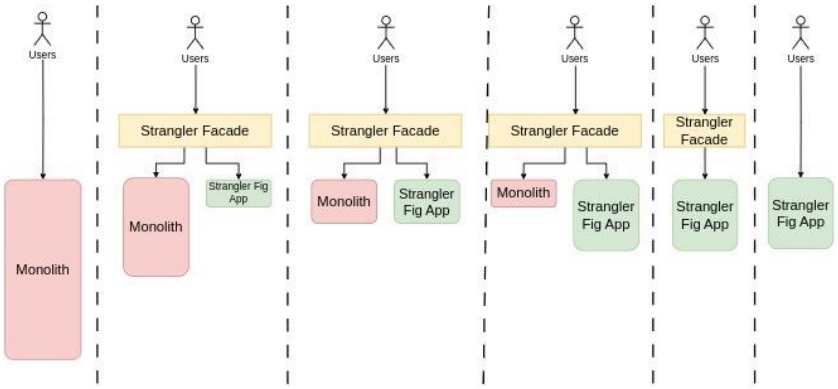


Figure 5-7 Monolith migration using the strangler fig pattern(Villavicencio, 2023)

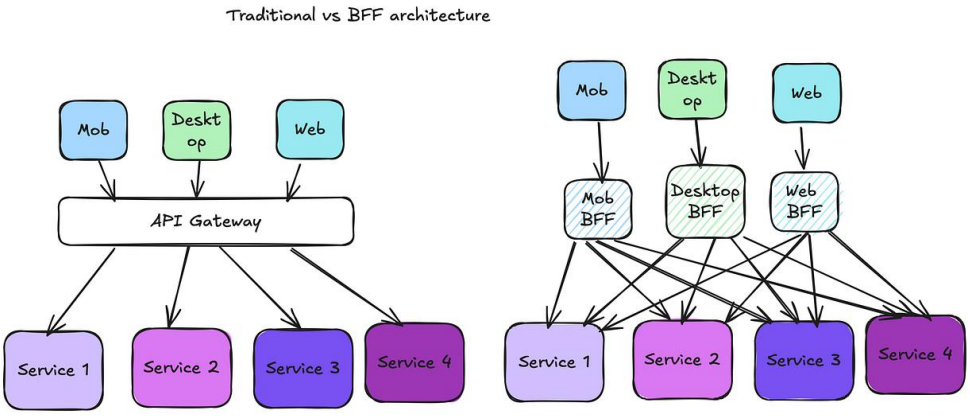


Figure 5-8 Comparison of API Gateway and BFF Pattern (Das, 2024)

GraphQL , released in 2015, is a server-side query language with a typed schema and microfrontends. is mentioned for one feature: the schema should be designed **from a view perspective** , not from a backend resource perspective, which is the opposite of the usual REST API design and is suitable for fragments with localized and diverse data needs (Mukhadin, 2026)It has a single entry point, is not usually released in versions, and is intentionally not shared across domains; team independence is instead maintained **by schema federation** , which assembles teams' schemas into a single graph. (Mezzalira, Building Micro-Frontends, 2025).

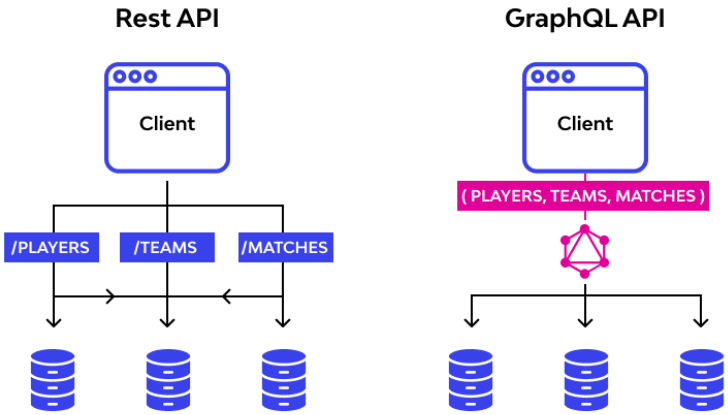


Figure 5-9REST API vs. GraphQL API(Mukhadin, 2026)

5.7 Data replication between teams

The last form of exchange does not take place in the browser and is therefore easily missed, and it is the one that decides whether the autonomy of the teams is real. The previous section assumes a rule: a microfrontend talks **only to its own team's server layer** , never to another team's endpoint. The reasoning is not stylish. Calling someone else's API introduces coupling, creates runtime dependencies between teams, and, most expensively, breaks isolation, since one's own part can no longer be run or tested without the other's system. The solution to this problem is **to replicate** the required subset, the owner publishes the change stream, and the consumer maintains its own copy of only those fields it needs and refreshes it in the background.

Table 5-1. Exchange mechanisms between microfrontends by topology and availability condition

Mechanism	**Participant topology**	**Availability condition**	**Contract holder**	**Example of data that is recommended to travel**
Element attributes	parent → fragment (container)	same document, fragment attached	tag name and attribute list	context values
Custom event elevation	fragment → parent (container)	the same document	event name	notification of user action
Event Highway	adjacent fragments, without containing	same document, both mounted at the same time	event name and load capacity type	notification with identifier
Broadcast Channel API	documents of the same origin (cards)	both documents open	channel name and message format	change session or shared state
Address (paths and parameters from the address)	crossing the mounting boundary	does not require concurrency	address format	domain object identifier
Web storage and cookies	same origin, through time	does not require concurrency; does not notify	key name and record format	the context that the document must survive
Shared balance with owner	adjacent fragments	the same document	separate read and write interfaces	the smallest set required for an API call
API Gateway, BFF	client ↔ server layer	first request on load	dictionary form and API contracts	endpoint addresses and domain information
Replication across the change stream	server layer ↔ server layer	asynchronously, in the background	shape of the flow of changes	subset of master data

Read from top to bottom, the table also shows the cost: the lower the mechanism, the greater the reach, but it comes at the cost of either a contract that multiple parties must honor, or a dependency that the browser does not provide on its own. The first two rows require two teams to agree on a

name; the middle rows add a form of portability; the last three also require an agreement on the infrastructure. Hence the rule with which Chapter 5 ends: the mechanism is not chosen by what is most expressive, but by what is **least satisfying to the topology** and availability conditions of the given case.

6 User interface and system design

The previous chapters have shown how microfrontends are defined, how they are assembled, how they are routed , and how they communicate. All four answers concern **the relationships between parts of a system** , and display consistency is a property that is not visible in any such relationship, it is visible on the screen, and only when two fragments are next to each other. If the boundaries are well-defined, assembly is done in the browser, routing is owned by the wrapper, and communication is kept to a minimum, the interface of the resulting system can still look as if it were assembled by four different manufacturers, and as a rule it does, because each of those four correct decisions increases the independence of the parts, and independence is precisely what consistency degrades.

User interface consistency across microfrontends implemented in different frameworks, and at what cost. The question has two parts, and they differ in what can be answered. The first part, is it possible, has an answer in the literature and it is affirmative, with conditions; that part is the subject of this chapter. The second part, at what cost, requires a measured artifact, and is therefore answered in 1 2 .8.

The presentation follows a single thread. First, it is shown that the design system is the only place in this architecture where the autonomy of teams is **intentionally limited** .

6.1 Design system versus component library

A design system is defined as the shared visual language of an organization, broader than a component library: it encompasses principles, patterns, rules, and terminology, and a component library is its derived, executable form. The difference is not a terminological pedantry, but has consequences for everything that follows.

Brand recognition, consistent user experience, speed of teams that don't reinvent the wheel, and easier collaboration between designers and developers. The first two are **product properties** and are visible on the screen, so they can be verified on the artifact. The second two are **properties of an organization** with multiple teams .

6.2 Exception to the sharing principle

The principle of this architecture is **the removal of commonality** : commonality is an anti-pattern (9.2), shared dependency is a cost that is paid and hard to recover (4.4), directly importing someone else's code across a boundary is the abolition of the boundary (4.3), and the default answer to the question of what to share is, nothing (Rappl, The Art of Micro Frontends, 2024). The design system is the only place where the same problem is answered **by adding** commonality. It is, therefore, the only intentional exception to the principles that underpin the entire architecture.

The argument can be shown, not just stated, because four independent sources do it in four different ways and arrive at the same final conclusion. (Geers, 2020)In the chapter on teams and boundaries, he advocates the rule *"don't be afraid of a copy"* : code duplication is usually a smaller cost than the coupling that sharing introduces, and immediately afterwards he cites the pattern library as an example to which this rule **cannot** be applied. (Mezzalira, Building Micro-Frontends, 2025)He sets the same boundary more sharply and from the opposite end: his warning that *"wrong abstraction is far more expensive than duplicate code"* stands alongside the claim that *"core components are the perfect place for centralization"* , so the author who warns most strongly against premature sharing claims for this one case that sharing is correct. shares the entire chapter prohibits excessive use of shared code among microfrontends, because of (Rufus, 2023)*dependency* hell *hell*) and the distributed monolith that follow from it, and cites one exception: *"one thing that can ideally be shared between microfrontends is the library of interface components"* . (Rappl, The Art of Micro Frontends, 2024)It sets out five criteria for what is shared and a default rule of not sharing anything, and within the same chapter it lists *user interface design* as one of the nine areas to be managed, thus an area where the absence of commonality is taken as a defect rather than a default.

These four statements are a verifiable finding, not a series of matching quotes, because all four are **exceptions to the same authors' own rules** . The design system is not exempt because it is useful, but because it **solves a different kind of problem than the one that sharing creates** . The connectivity that sharing introduces is a technical cost, and it is paid for in speed of delivery and in the freedom to upgrade. The inconsistency that sharing removes is a cost paid by the user, and it is not reflected in any delivery metric.

6.3 Sharing criteria

The design system satisfies all five of Rappl 's criteria for and against dependency sharing, making it not an exception, but **the only case that satisfies them all simultaneously** .

(Geers, 2020)lists three axes for that decision and the third is the one that decides.

- The first is **complexity** .
- The second is **the value of reuse** , where his example is worth keeping: The product card seems like an obvious candidate for a shared library, appearing in many places, but each page it appears on wants something different from it, so the central component soon carries a dozen or so switches with which its consumers choose behavior. Its solution is more interesting than the warning itself: the card remains shared, but what is different about it becomes not a switch, but **a place to embed** content provided by the consumer.

- The third is **domain specificity** , and it is checked by asking a single question: which team has an interest in changing this component and why. If the answer is "everyone, because of the appearance," the component is shared; if it is "one, because of its job," it is not, no matter what it looks like.

7 Delivery, automation and testing

A microfrontend architecture is no different from a modular monolith in terms of the appearance of the source code, and both have clear boundaries, named contracts, and mutually independent parts. The difference is in **what is delivered separately** , and this is not a property of the code but a property of the delivery flow. A fragment that cannot be deployed without the others is a module, not a microfrontend, no matter how clean its boundary in the code may be. Hence the practical consequence: **the characteristic properties of this architecture are in the delivery flow, not in the source code** .

7.1 Mono-repository, poly-repository, and branching strategy

A mono-repository is an approach in which multiple applications, services, or microfrontends are developed and maintained within a single shared repository, while a poly-repository implies that each application or microfrontend has its own repository. The choice between these two approaches affects the way dependency management, code sharing, team organization, and delivery independence are handled. A mono-repository facilitates code sharing, dependency management, and local development, but can increase project coupling, while a poly-repository provides greater isolation and team independence, at the cost of code duplication and difficulty in managing common standards. Therefore, as a general rule, **the repository topology should follow the team topology, not the number of microfrontends** : if multiple teams independently develop and deliver their parts of the system, a poly-repository is a more natural choice, while for a single team or a closely coordinated organization, a mono-repository is more rational.

Regardless of the choice, a mono-repository must not violate the independent deliverability of microfrontends. In this case, independence is not ensured by the repository structure itself, but by the delivery process, which recognizes changed fragments and deploys only them. In terms of working with branches, the essential principle is to use short-lived branches and frequent integration, thus avoiding long-lived branches and complex merge conflicts.

7.2 Versioning, rollbacks, and feature switches

In a microfrontend architecture, versioning and release management require the ability to clearly name different versions of individual fragments and to determine at runtime which version the user is getting. Therefore, each fragment should have its own version, while the fragment discovery mechanism should contain their versions, addresses, and other necessary metadata . This mechanism allows traffic to be directed to a specific fragment version and is the basis for strategies such as *Blue-Green* and *Canary* .

Blue-Green allows for quick rollback by switching traffic to the previous environment. It requires an additional set of all the hardware components that the second version will be delivered to, and then routes traffic to one of those two sets via a router. (Maruthi, 2022)

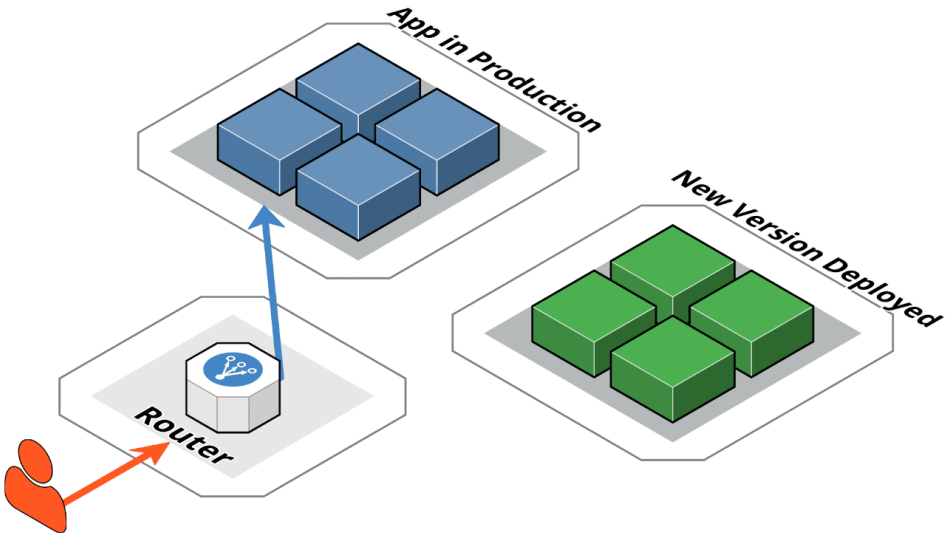


Figure 7-1Blue-Green delivery(Maruthi, 2022)

Canary allows for a gradual rollout of a new version to a limited subset of users. The approach is similar to Blue-Green, however the main difference is that only a specific subset of users is redirected to a specific subset of new functionality (Rastogi, 2022).

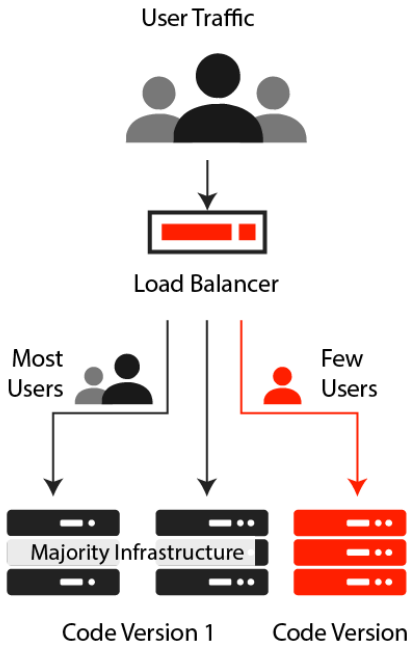


Figure 7-2Canary delivery(Rastogi, 2022)

Reverting to a previous version can be accomplished by redeploying the previous version or redirecting traffic, but it is necessary to check compatibility with the current state of the rest of the system.

Feature switches *flag / toggle*) have a different role: they control behavior within a single version, while traffic distribution determines which version of the fragment the user receives.

7.3 Migration from a monolithic frontend

Migration from a monolithic frontend to a microfrontend architecture can be accomplished by gradually breaking out parts of an existing application, building a new frontend on top of an existing backend system, or replacing the system entirely at once. The most common approach is a gradual migration, in which new fragments take over individual areas of the application as the monolith gradually loses its role. Before migrating the entire system, it is useful to implement the approach on a smaller, realistic part of the application to test the technical and organizational assumptions.

The choice of migration strategy primarily determines how long it is necessary to maintain the old and new systems simultaneously. Gradual migration reduces the scope for individual failures and allows for more small deliveries, but extends the period of dual maintenance. A complete replacement is faster in the final transition, but carries the highest risk because there is no gradual transition or simple return to the previous state. Therefore, the migration strategy should be viewed primarily as a decision on the acceptable duration of dual maintenance, and not as a choice of technology itself.

7.4 Testing strategy and suitability functions

The testing strategy in a microfrontend architecture retains the usual pyramid of tests, but most of the testing is focused on individual fragments and their immediate contracts. There should be more unit and integration tests, while comprehensive tests The number of *end-to-end (E2E*) tests should be smaller and carefully selected, as testing the entire system across a large number of boundaries becomes complex, slow, and prone to failed tests due to false *negatives* . (Fowler, 2012)A fragment should test its own contract, while tests of the overall system should check that multiple fragments work together correctly.

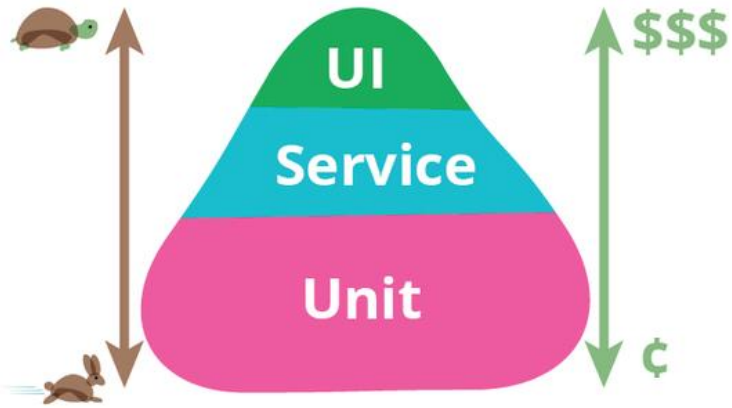


Figure 7-3 Pyramid of tests (Fowler, 2012)

In addition to functional tests, it is also necessary to automatically verify the architectural properties of the system. Suitability functions are executable checks that can, for example, control package size, performance, static analysis, test coverage, dependency safety, and rules specific to microfrontend architecture. They are especially important in projects without a production environment because they allow certain architectural properties to be objectively verified directly on the artifact.

7.5 Local development and isolation of fragments

Local development of microfrontends requires a way to develop a fragment without having to run all the other parts of the system at the same time. One approach is to isolate the fragment using mock neighbors or a special development page that allows you to view different states of the fragment. The mock neighbors should be based on the fragment contract, as a large number of complex mock implementations can indicate poorly defined boundaries between fragments. Another approach is to locally connect to real, already deployed neighboring fragments.

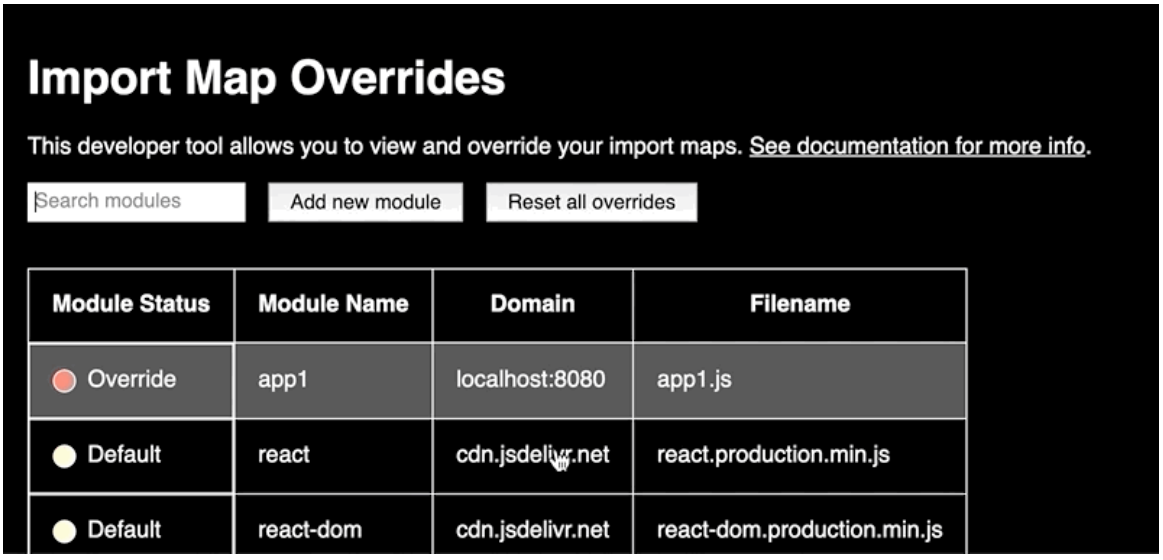


Figure 7-4 Changing the microfrontend domain during local development

Isolation and working with real neighbors are not substitutes for each other, as they check different aspects of the system. Isolation checks whether a fragment fulfills its own contract, while working with real neighbors checks whether that contract is used correctly in the actual assembly. Both approaches require that fragment addresses can change during execution, that is, that there is a mechanism for detecting and replacing them. In this way, local development becomes another consequence of the need for indirection between the wrapper application and a specific version of the fragment.

7.6 Development experience

The development experience is an important aspect of microfrontend architecture because the complexity introduced by the architecture is felt most by the developers themselves. A good development setup should make it easy to create new fragments, understand their contracts, test them, and work locally with the entire assembly. Therefore, the basic prerequisites are support in the usual development environment and a mechanism for automatically creating a new fragment from a predefined template.

A higher level of development experience includes centralized documentation, uniform rules for static analysis and code formatting, complete type descriptions, prepared testing, mock neighbors, and development environments for fragment isolation. Additional capabilities include offline development, tools for viewing fragment composition, and a dedicated development portal. These capabilities should not be viewed as strictly defined stages of maturity, but rather as different mechanisms for solving specific development problems.

8 Security and isolation of microfrontends

The security of a microfrontend architecture differs from the security of a monolithic application primarily due to the distribution of responsibilities across multiple fragments and teams, as well as the fact that fragments in a client composition execute in a shared environment. Security measures can have a document scope , when they apply to the entire composed application, or a for a single fragment , when referring to a single delivery.

8.1 Web platform standards as a boundary

The basis of security is the mechanisms provided by the web platform itself. *Content* Security **Policy** *Security Policy - CSP)* represents a list of allowed resource sources and can restrict, among other things, the loading of scripts, thus representing an important protection against certain forms *Cross-site scripting - XSS* attacks (Rappl, The Art of Micro Frontends, 2024).

the Report - Only mode is particularly important , as it allows for the discovery of actual resources and endpoints used by different fragments before imposing a restrictive policy.

HTTP Strict Transportation Security - HSTS forces the browser to communicate with a specific host over HTTPS (Hypertext *Transfer Protocol Secure*) , which reduces the risk of man-in-the-middle attacks and interception of sensitive (Rappl, The Art of Micro Frontends, 2024)data .

Cross - origin **resource** *retrieval resources sharing - CORS)* determines which origins a server allows access to its resources. In this way, access control to a fragment's resources is enforced at its origin, while CSP controls which resources the composed page itself trusts. These two policies have different scopes and must be consistent with each other (Rappl, The Art of Micro Frontends, 2024).

8.2 Script limits

Unlike *CSS* , there is no direct mechanism for *JavaScript* within client-side composition to provide separate execution environments for different microfrontends (Rappl, The Art of Micro Frontends, 2024). The web platform's primary isolation mechanism is the origin *,* and fragments executing in the same document share the same environment. As a result, one fragment can access the DOM of other fragments, share web storage, use common objects, and communicate over a common event bus.

An iframe can provide complete isolation on the client , as it gives the fragment a separate origin, and additional *sandbox* restrictions can control its capabilities. (Rappl, The Art of Micro Frontends, 2024) (Mezzalira, Building Micro-Frontends, 2025).

Server-side composition represents a different architectural approach where isolation can be implemented at the execution point rather than in the shared browser environment.

This leads to an important conclusion: **a microfrontend is not a security boundary in a client composition** . It is primarily a unit of delivery and organizational responsibility, while real isolation must be provided by the web platform mechanisms and delivery processes.

8.3 Access token storage

The choice of how to store the access token is particularly important in a microfrontend architecture. With web storage (*localStorage* or *sessionStorage*) , *JavaScript* can read the token directly, which in client-side composition means that any other fragment in the same origin and domain can read it. In addition, it is possible to use web storage and cookies as ways to share the token.

In the context of client-side composition, the advantage of cookies is that they reduce the system reach of a potential XSS attack: a compromised fragment cannot read the access token directly. However, where the token is stored does not determine the user's rights; authorization must still be performed on the server side (Mezzalira, Building Micro-Frontends, 2025).

8.4 Shared dependencies

The security of microfrontend architecture is not only a technical issue, but also an organizational one. Shared dependencies represent a common attack surface: when multiple fragments use the same dependency, its vulnerability potentially affects all users of that dependency, and its upgrade requires coordination across multiple teams. Therefore, it is recommended that dependency sharing is not assumed, but justified on a case-by-case basis with specific criteria.

Special attention is required for **monitoring and event logging** . OWASP highlights vulnerable dependencies and logging and monitoring deficiencies among relevant threats, but supply chain control and a contribution review process are also needed. In a microfrontend system, monitoring must cover both the entire document level and the level of individual fragments, in order to be able to determine what was delivered, when, and by whom.

9 Anti-pattern microfrontend architecture

An anti-pattern is a solution that is often chosen because it seems convenient at first, but in the long run it does more harm than good, and there is a recognizable way to fix it. Therefore, a single mistake is not an anti-pattern, but a repeating pattern. In the context of microfrontend architecture, it is important to distinguish an anti-pattern from a compromise: a decision whose cost is known and for which there is a mechanism to limit it does not necessarily constitute an anti-pattern (Neill, Laplante, & DeFranco, 2011).

A common characteristic of the anti-patterns discussed in this chapter is that they represent the application of practices that were useful in monolithic systems, but are applied in places where the boundaries of independent delivery are violated. Thus, avoiding code repetition can lead to premature abstraction, a single point of truth can lead to shared state, component thinking to incorrect granularity, and freedom of technology choice to lack of governance.

9.1 Incorrect granularity: microfrontend or component

One of the most important architectural choices is determining the boundaries of the microfrontend. Size alone is not a sufficient criterion: a microfrontend should represent **a complete business entity** and be able to be delivered independently, while a component is primarily a user interface element and needs to be given context to be useful in a business sense (Mezzalira, Building Micro-Frontends, 2025). As a rough guideline, the literature suggests that more than five microfrontends in a single view may indicate overly granularity.

However, the number of fragments should be seen as **a symptom, not a criterion** . What is more important is how much communication the partition creates. Too fine a partition leads to a large number of connections between fragments, thus losing the benefit of independence. A similar problem is described in microservices as *Grains of Sand* and *Big Ball of Distributed Mud* : too small units require subsequent connection to do useful work. It follows that good granularity is more appropriately assessed by cohesion and the amount of communication than by a fixed number of fragments.(Richards & Ford, 2025) (Rappl, Top 10 Micro Frontend Anti-Patterns, 2024).

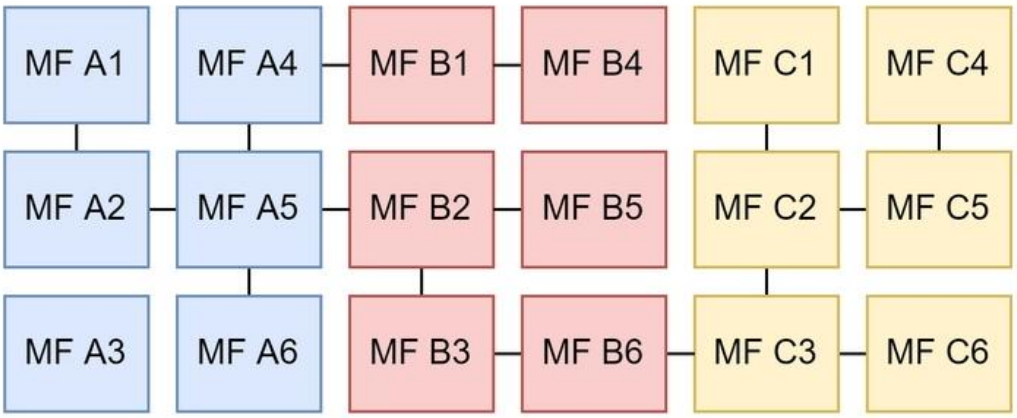


Figure 9-1 Microfrontends subdivision (Rappl, Top 10 Micro Frontend Anti-Patterns, 2024)

However, it is better to set a limit so that the fragments are still larger. Too small a division requires merging fragments and creates communication that was previously internal, while separating a too large fragment requires changes in domain, state, and ownership. Therefore, as a rule of thumb, a microfrontend should follow business capability and cohesion, not the size of the user interface.

9.2 State sharing and two-way dependency

Shared state and two-way dependency can be viewed as two forms of the same problem: the loss of clear direction of dependency between fragments. With shared state, multiple fragments read and modify the same data, while two-way dependency creates a circular dependency in the code. In both cases, fragments cease to be truly independent (Mezzalira, Building Micro-Frontends, 2025).

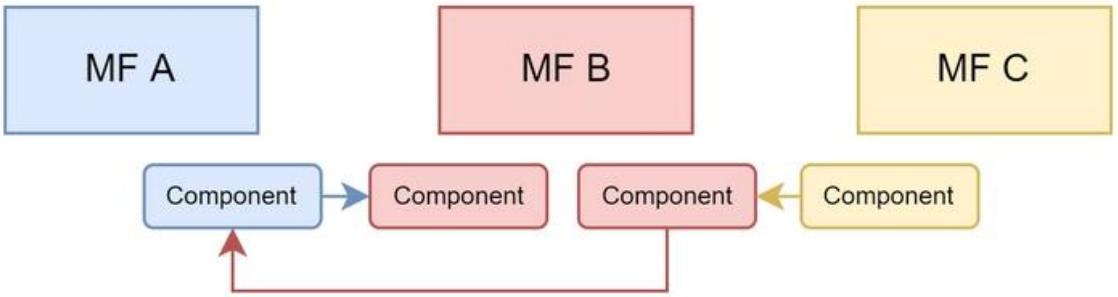


Figure 9-2Tight coupling of microfrontends (Rappl, Top 10 Micro Frontend Anti-Patterns, 2024)

The recommended approach is to keep state management within the fragment, and only pass necessary messages between fragments. via events. However, the transition from shared state to event communication does not remove the coupling in itself. If the entire domain model is transmitted through an event message, the fragments must still share its form and meaning. Therefore, it is more convenient to keep messages as small as possible, that is, to contain only the information necessary for the recipient, such as the event identifier and name.

A shared repository, an event bus, or a context that the wrapper application passes to fragments are different forms of mediation. The choice between them is therefore not a choice between coupling and its absence, but a choice **of what the fragment will depend on** . The mediator with the smallest and most stable contract is the most suitable. If two fragments must share the same business state or have strong interdependencies, this may be a sign that the fragment boundaries are misaligned and need to be re-evaluated (Mezzalira, Building Micro-Frontends, 2025).

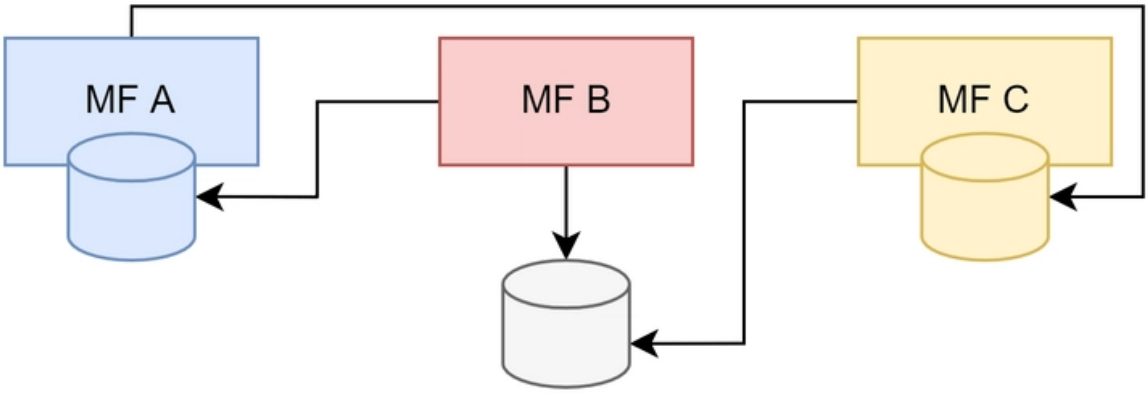


Figure 9-3 Distributed data inconsistency (Rappl, Top 10 Micro Frontend Anti-Patterns, 2024)

9.3 Anarchy: multiple frameworks without governance mechanisms

Anarchy is not just about using multiple frameworks, but also **about the absence of clear guidelines, standards, and governance mechanisms** . This leads to silos of knowledge and a lack of opportunities for developers to quickly move to another team. Therefore, using multiple technologies alone is not necessarily an anti-pattern.

There are several drawbacks to this approach. Some are organizational, such as increased cognitive load and different ways of solving problems, while one is directly technical: each framework can bring its own dependencies and packages, which increases the amount of code the user downloads and reduces the ability to share caches . This latter cost cannot be solved by organizational agreement alone, as it is directly paid by the user.(Rappl, Top 10 Micro Frontend Anti-Patterns, 2024)

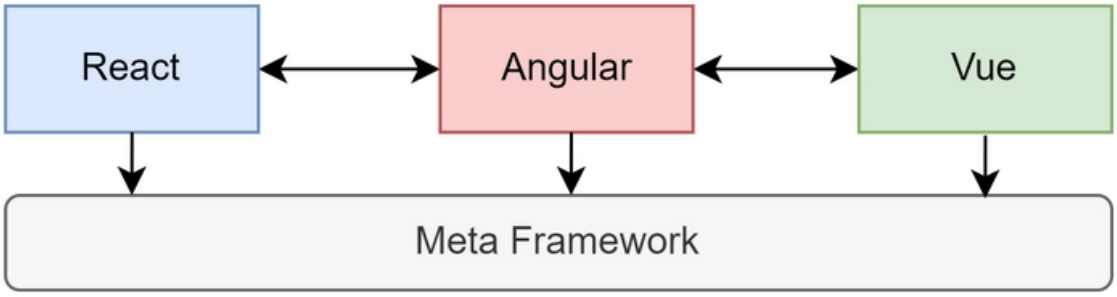


Figure 9-4A large number of different working frameworks (Rappl, Top 10 Micro Frontend Anti-Patterns, 2024)

To keep a multi-framework architecture manageable, a clearly defined set of allowed frameworks, a common design system, a single contract for fragment inclusion, automated validation of those contracts, and a process for changing common rules are needed. Sometimes, it is necessary to use multiple frameworks as the system is in the process of gradually migrating to a microfrontend architecture. In case the multi-framework approach is introduced as a temporary solution, it is necessary to define both the deadline and the responsibility for its removal. Otherwise, temporary heterogeneity easily becomes a permanent state and degenerates into anarchy.

9.4 Lack of anti-corruption layer and premature abstraction

anti-corruption layer *Layer*) and premature abstraction are two different cases of the same problem: improper management of data form and logic at the interface between systems. **The anti-corruption layer** allows a legacy or external system to conform to the contract of the microfrontend architecture without spreading its specifics to the rest of the system. In contrast, premature abstraction exposes shared code outside of the fragment before it has been confirmed that that pattern is truly stable (Mezzalira, Building Micro-Frontends, 2025).

Shared libraries that contain business logic pose a particular risk. Code reuse can violate limited context as knowledge about a single business crosses boundaries. This leads to the so-called *"Dependency Hell"* and/or the concept of *"distributed monolith"* , in which multiple teams become dependent on changes to a single library.

Shared code is not absolutely forbidden. User-defined component libraries and generic helper function code can be justified, but sharing business knowledge between fragments should be avoided. The pace of change can also be considered as a practical criterion: a shared library should change less frequently than the fragments that use it, otherwise it becomes a source of hidden coupling and imposes its release rhythm on other fragments.

The anti-corruption layer should be seen as a temporary mechanism that allows for gradual integration and whose removal should be planned. If left without a deadline and owner, it can itself become a permanent dependency.

9.5 Multiple calls to the backend

The increase in the number of calls to the backend is specific in that it is not necessarily the result of a wrong decision by the developer, but can occur as a natural consequence of breaking a monolithic page into multiple independent fragments.

The problem can occur in the form of repeated requests for the same data or in the form of a large number of different requests that increase the number of network calls. In such cases, mechanisms that do not require changes to the fragments themselves are suitable, such as short-term response caching and intermediary request aggregation. Such mechanisms maintain the independence of fragments because they do not need to know about each other.

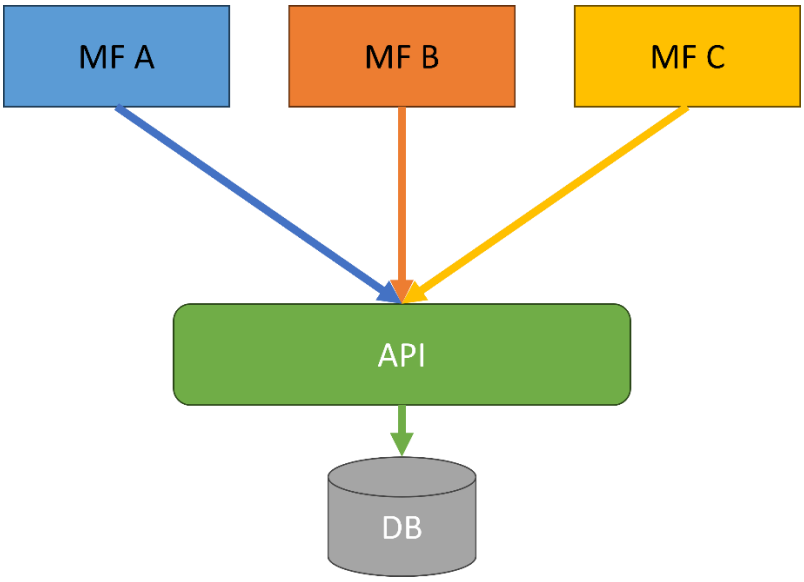


Figure 9-5 Multiple microfrontends calling the same API

Conversely, a wrapper that handles backend communication for multiple fragments can reintroduce the coupling that a microfrontend architecture is supposed to remove. Introducing such a mediator means that each module then becomes dependent on the mediator, while the mediator itself can represent *Architecture Sinkhole* if it just passes data without its own logic. Therefore, it is better to first accept an increased number of independent requests, and introduce optimization only when it is really needed and when it can be implemented without violating fragment boundaries.

10 Analysis of the research problem

The previous eight chapters described mechanisms without an object. This chapter introduces an object, based on which the application of the mechanisms will be described in the following chapters.

The discussion flows from domain to requirement and from requirement to structure. Section 10.1 describes the domain, participants, and workflow, and from these derives a domain property that gives the whole paper its subject matter. Section 10.2 states the functional requirements and indicates which of them have been met. Section 10.3 translates the domain concerns into architectural features. Section 10.4 identifies subdomains and bounded contexts, thus obtaining a list of *parts* . Section 10.5 shows in this domain what 2. 4 claimed in general, why a monolithic client layer does not fit this problem, and shows this as a claim that could be refuted. Section 10.6 states the confidentiality and access trace requirements.

10.1 Domain, users, and physician workflow

The domain is **an outpatient specialist examination and the medical documentation that results from it** . The system does not cover hospital treatment, laboratory diagnostics, billing for services, or data exchange with other institutions; it covers a single, completed event, a meeting between a doctor and a patient at a scheduled appointment, and a permanent record that remains after that meeting.

A doctor's time is the scarcest resource in the system. An appointment is of a predetermined duration, during which professional and documentation work is performed, waiting for a response, searching for data on another screen, re-entering what the system already knows, and all of this is paid for with the patient's time for the examination. The existing text says it vividly *("the doctor is the bottleneck of the system"*); here it is also expressed as a requirement: **an interruption in the workflow is not a user interface inconvenience, but a loss of clinical time** .

Table 10-1 Participants and their responsibilities in the domain

Participant	Domain role	What is it that drives?	What can we see?

Doctor	primary participant; professional decision-maker and author of the record	appointment, examination, diagnosis, therapy, report	data of patients treated
Patient	secondary participant; data subject	canceling your own appointment	exclusively your own data
Administrator	systemic participant, outside the clinical course	loading reference data, access trace monitoring	meta - access data, not record content

The first statement that follows from the table is: **The jurisdiction check must not depend on which screen is open.** The doctor's right to see a patient does not depend on which screen is open, but on whether there is an appointment between the two; the patient's right depends on identity, not on the path.

Workflow. Figure 10-1 shows the flow in the form of an activity diagram:

1. the doctor logs into the system;
2. receives a list of their scheduled appointments for the day;
3. calls the patient and begins the examination at the selected appointment;
4. While the patient describes the symptoms, the doctor writes down the medical history;
5. In parallel with the registration, it searches the International Classification of Diseases (ICD-10) and selects a diagnosis;
6. prescribes therapy, one or more medications, each with a dosage, frequency and instructions for use;
7. generates an examination report as needed and delivers it to the patient, in paper form or as a PDF;
8. If necessary, he schedules a follow-up appointment and returns to the appointment list.

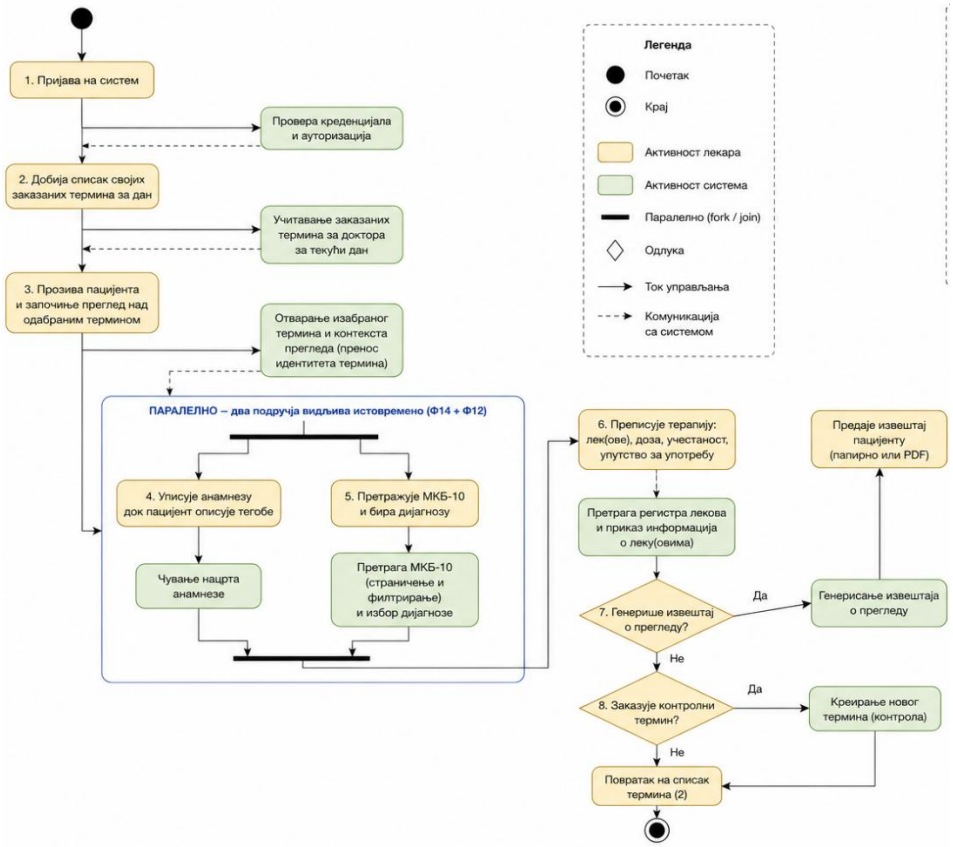


Figure 10-1 Diagram of the activity of the working doctor

Three features of that flow are not obvious from its description, and each of them is borne out in the following text.

First, in one appointment, the doctor uses four data sets of different lifespans and different ownership. The list of terms is transactional: it changes every day and is changed by the institution itself. The examination record is a clinical document: it is created once, is not subsequently revised and has a legal and professional addressee outside the system. ICD-10 is an external classification: it is not changed by either the doctor or the institution, it is revised using someone else's scheme, it is used only for reading and it is large; in the built system, the reference set of codes is of the order of several million characters, which is the data that also determines the search method. The drug register is also external and is also used only for reading, but it changes more often than the classification.

Second, the flow contains a moment in which two areas must be visible simultaneously. Step 5 is not a step after step 4, but rather alongside it: the diagnosis is selected *while* the history is visible, because the history is the basis of the diagnosis. If selecting a diagnosis requires going to another screen and coming back, the clinician must hold in mind what he or she has just written down, and this is precisely the kind of interruption that the domain's initial care prohibits. The resulting requirement is expressed in 10.2 as F12 and F14 together, not separately.

Third, the flow also contains a transition that is inverse in nature. Between steps 2 and 3, the subject of the work changes: from a list of terms, one moves to a single term, and from that point on, the list is no longer needed. Here, the areas **do not** need to be visible at the same time; it is necessary that the identity of the selected term survives the transition.

Finally, a common assumption is explicitly rejected here. Just because the areas are separable **does not** mean that the user perceives them as separate: for the doctor, an examination is one job, not four. The requirement that follows from this belongs to 10.3 and is stated there as interface consistency; it is important here for the sake of performance, because any division that 10.4 suggests must remain invisible in the user experience even when it is real in delivery.

The domain is suitable for testing microfrontend architectures because it simultaneously contains functionally separated entities, different rhythms of change, common user flows, the need for communication between parts, and requirements for independent loading and access control. In this way, the demonstration application serves not only as an example of implementation but also identifies specific architectural situations in which the advantages and limitations of the microfrontend approach can be experimentally verified.

10.2 Functional requirements

The functional requirements are expressed as capabilities per participant and are designated by the identifiers F1–F24, which are referenced in subsequent chapters. Table 10-2 lists them along with the participant and domain area.

Table 10-2Functional requirements

Eid	Request	Participant	Area
F1	Registration of a doctor, with selection of a specialist department	doctor	approach
F2	Patient registration	patient	approach
F3	Login and logout	everyone	approach
F4	View and edit your own profile and password	everyone	approach
F5	List of specialist departments	doctor	approach
F6	View your own appointments in the time view	doctor, patient	scheduling
F7	Scheduling an appointment for a patient, with overlapping rejection	doctor	scheduling
F8	Canceling an appointment	doctor, patient	scheduling
F9	Rescheduling an appointment to another time	doctor	scheduling
F10	Patient search when scheduling	doctor	scheduling
F11	Search and filter doctors by department and specialization	patient	scheduling
F12	ICD-10 catalog search	doctor	diagnosis
F13	Search the drug registry and view the drug form	doctor	therapy

F14	Recording of examinations: history, diagnosis, time of onset	doctor	review
F15	Prescribing therapy: drug, dosage, frequency, instructions	doctor	therapy
F16	Viewing a patient's examination history	doctor	review
F17	Search for examinations by patient, doctor, diagnosis and period	doctor	review
F18	Summary of the patient (appointments, examinations, therapies)	doctor	review
F19	Generate a review report	doctor	documentation
F20	Download the report as a PDF document	doctor, patient	documentation
F21	View your own reports and search for reports	patient, doctor	documentation
F22	View review statistics and diagnoses	doctor	supervision
F23	View medical data access trace	administrator	supervision
F24	Loading reference data (ICD-10, drugs, departments)	administrator	supervision

One request requires two areas at the same time, and two requests require a transfer between screens. F12 and F14 must be satisfied at the same time; F6 and F14 must be satisfied one after the other, with the transfer of term identity. Neither of the two obligations follows from a single request, both follow from **a pair** of requests, and this is why a list of requests by itself is not sufficient as a description of the problem.

10.3 Non-functional requirements

The functional requirements in 10.2 state what the system does; they do not distinguish between a system that does it in half a second and a system that does it in five, nor between a system that ships weekly and a system that ships once every six months. That distinction is the subject of architectural features, for which 2. 1 set three conditions, that the feature be non-domain-specific, that it affect the structure, and that it be critical to the success of the system, and provided Table 2-1 as a glossary.

The method and its limitation. Characteristics are derived from three sources: from **explicit domain concerns** that the client names, from **project requirements** that imply them, and from **tacit domain knowledge** that no one speaks out because it is taken for granted in a given domain.

An additional recommendation and constraint that system design should respect is: **as few features as possible should be chosen** . The reason is not to save on the document, but rather that each feature introduces a structural obligation, and the obligations cancel each other out. The list that follows therefore has seven items, and among them the first three are marked as **leading** , in accordance with the recommendation to choose the top three.

Table 10-3Architectural features

Characteristic	**Source**	**What does the domain require?**

Delivery	project request	the four data sets from 10.1 change at different rates; the non-changing area must not be re-delivered to modify the changing area
Performance	explicit domain care	interruption in an appointment is a waste of clinical time; searching the catalog should not mean downloading the entire catalog, and the initial loading should not wait for all areas
Confidentiality	tacit domain knowledge	The right to access patient data does not depend on which screen is open (10.1, Table 10-1); it must be verifiable upon request and leave a trace (10.6)
Extensibility	project request	nine capabilities from 10.2 do not yet have a client implementation; adding areas must not mean changing existing ones
Interface consistency	explicit domain care	The doctor has one job to do, so the division of the area should not be visible as a difference in the appearance or behavior of the controls.
Testability	project request	a clinical decision is recorded once and cannot be revised, meaning that a recording error has a permanent consequence
Availability	tacit domain knowledge	The cancellation of one area must not disable the entire appointment; an incorrect catalog search must not prevent the entry of anamnesis

Why are deliverability, performance, and confidentiality the top priorities? The choice follows from which failure would make the system unusable in the domain, not just worse. Deliverability is the top priority because it is the only feature required by the 10.1 *data feature* , regardless of the number of users and network speed: since ICD-10 is changed by an external institution and the term list by the practice itself, a system in which these two changes have the same delivery cycle permanently pays for the other's rhythm. Performance is the top priority because it is required by the most expensive resource in the domain: participant time. Confidentiality is the leading one because it is the only characteristic whose non-fulfillment is not a bad product but a harm to the patient himself. The other four are no less real, but all four are consequences that can be fixed by subsequent change, while none of the first three can.

10.4 Identification of subdomains and restricted contexts

Section 3.2 set up the instrument: fragment boundaries are in the domain, across subdomains and bounded contexts, not in the technology layer, and it is left there to apply the instrument when there is a domain where the difference is seen. This section is that application.

The recommended procedure for separating subdomains is *event storming* , a workshop involving product managers, testers, and developers, because subdomains are revealed from **the differences** in how people in different roles see the same job.

- driven design divides it into **core** , **supporting** , and generic subdomains, where the core carries what the system provides value in, the supporting is needed by the system and is not a difference in

itself, and the generic is the problem being solved that is being taken over. Applied to the domain from 10.1, the classification is unambiguous and useful: **examination** is *the core* **subdomain** , because medical documentation is what the system exists for; **scheduling** is supporting, because the term is a condition for the examination and not its purpose; **the ICD-10 catalog** and **the drug registry** are generic, because the system does not produce them but takes them over in a closed form; **access** (login, identity, authority) is generic.

A part can be separated **functionally** , by area of work, or **technically** , by role in building a page. Its criterion for the correctness of a boundary is the locality of change. A boundary is good if a change to a single requirement is contained within that single part, and the independence check is what is described as *"turn off one part"* : if one part is turned off, the rest should remain usable, just with fewer features.

Applied to the list in 10.2, both tests give the same result and it is listed in Table 10-4.

Table 10-4Subdomains, restricted contexts and boundary type

Subdomain	**Type**	**Requirements**	**The rhythm of change**	**"Turn off one part" test**
Overview and documentation	basic, functional	F14, F15, F16–F21	changes with professional practice and with the form of the document	without it, the term has no outcome; the rest of the system remains usable
Scheduling	supportive, functional	F6–F11	changes with the organization of the practice's work	without it, there are no new appointments, but the existing review is completed
Disease catalog	general, functional	F12	it is changed by an external institution, rarely	without it, the history is recorded, the diagnosis is not selected
Drug registry	general, functional	F13	changes more often, but still on the outside	without it, the examination is recorded without therapy
Access and identity	general, functional	F1–F5	the most stable part of the system	without it nothing else is useful
Notification, navigation, session	cross-disciplinary, technical	— (v. 10.2, fourth finding)	changes with each area a little bit	without them, each individual request works, and the system is not usable as a whole

10.5 Limitations of a monolithic frontend in the observed domain

What would really be lost in the 10.1 problem if the client layer remained a single, indivisible part.

First, one delivery cycle for four different rhythms of change. It follows from 10.1 and Table 10-4 that a change in the diagnosis area is initiated by an external institution, a change in the appointment by the practice itself, and a change in the examination record by the professional practice. In an indivisible client layer, each of these changes enters the same construction process and the same act of delivery, which means that a revision of the disease classification carries a risk for the screen on which the physician enters the history, even though it has nothing in common with it. Deliverability is the leading feature in 10.3 precisely because of this property of the data, and it is the first that cannot be satisfied in a monolithic client, not because delivery is difficult, but because it is **indivisible** .

Second, one technological choice for areas of different requirements. Searching a catalog of several million characters and entering a clinical record with the strictest accuracy requirements are not the same problem, and in an indivisible client they must be solved by the same means, because in a single project there is one version of each dependency. The consequence is not aesthetic: it means that no area can change a means before all areas have changed it.

Third, a single failure domain. Availability is listed in Table 10-3 as an implicit domain requirement: a failed catalog lookup must not prevent a history entry. In a non-shared client layer, this cannot be guaranteed, because a failure to load any part can crash the document containing all parts. In a shared client, this requirement becomes easier to satisfy; one microfrontend can fail, but **the others will continue to work unhindered** .

A monolithic client layer would be the correct answer to this problem under three conditions: that all areas have the same pace of change and the same change driver; that no area carries a performance or accuracy requirement that is significantly different from the others; and that the total amount of functionality remains such that one team can keep it in mind.

10.6 Confidentiality requirements and access trail of medical data

Confidentiality is listed in Table 10-3 as a leading characteristic and as tacit domain knowledge: in the observed domain, confidentiality of medical data represents a fundamental system constraint, not an optional functionality.

Access rights do not follow from place in the system, but from relationships between people. It follows from Table 10-1 that a doctor may see the data of the patients he treats, not all patients, and that a patient may only see own data, not other people's patients. This right does not depend on the open screen, the path, or the sequence of actions, but on the existence of an appointment between the two participants, which means that the check must be performed **every time the data is accessed** .

Every access to medical data must leave a trace. The reason is a property of the domain, not a technical precaution: the patient cannot perceive that his data has been read, so a trace is the only way for unauthorized access to become detectable after the act. It follows that what the trace must contain, who accessed what, of what type and when, and, importantly, what it must not contain. The usage record must not become a second copy of the data itself: **a trace of access to a medical record containing the content of that record duplicates the confidential data and exposes another vulnerability instead of surveillance.** The requirement is therefore expressed in a limited form, metadata - access data, never clinical content (Rappl, The Art of Micro Frontends, 2024).

After logout or session expiration, access to protected resources must be disabled in all parts of the client system. In a microfrontend architecture, this requires unified authentication state management or a mechanism by which session state changes are propagated to all active parts of the system.

11 Solution architecture

When designing the architecture of this system, four decisions need to be made: what constitutes a fragment, how fragments are assembled, how they are routed and communicated, and how to approach the design of the server layer.

11.1 Applying the decision framework: defining fragment boundaries

First decision, what constitutes a microfrontend: Table 10-4 lists five subdomains and one cross-domain, with the rate of change and the outcome of the *"turn off one part" test* for each. This input, however, is not the answer, and 10.4 makes this explicit: **not every bounded context needs to become its own microfrontend** (3.2).

Table 11-1From limited context to fragment

Limited context (10.4)	**It becomes**	**Framework**	**Basis of decision**
Overview and documentation	independent fragment	Angular	basic subdomain, own rhythm of change, own screen
Scheduling	independent fragment	React	supporting subdomain on its own screen, independent rhythm
Disease catalog	independent fragment	View	external change agent, own performance requirement, share screen with preview
Access and identity	fragment **and** library	React (screens)	screens are a vertical split; session state has no display
Notification and navigation	two fragments	without frame	different route agreement (5.1, 3.1)

11.2 Composition

The second framing decision has two parts that are often mixed into one: **where** the view is composed and **with what** .

The 4.2 framework offers three places, server, edge, and client. The decision to build microfrontends on the client is based on the nature of the application itself: the system is an interactive tool, not a page whose content needs to be readable before the executable code is loaded.

Module Federation , a *webpack* builder mechanism , is used. described in 4.4.

Dependency sharing is achieved using *Module Federation and single-spa* libraries that provide the ability to place certain dependencies as singletons . *Imported* dependencies such as framework libraries are shared across multiple microfrontends and are perfect candidates to be marked with the singleton tag , which guarantees that only one version is pulled from the web at a time and used by all fragments and pages that require it.

On the three axes of type space (3.3), the decisions of this section yield the following values: assembly is **dynamic** (at runtime), place is **client** , and the type of division is a combination of both horizontal and vertical approaches.

11.3 Routing

The third decision follows the first and second without any new freedom: where the view is compiled, it is also routed , so routing is on the client.

What the wrapper had to build itself is worth mentioning, because it answers the question of how much coverage the existing library provides. In this solution, the wrapper builds three things that the library does not provide: a route table as data, an activation function that mounts a route based on the path, and a gateway that is not mounted by the fragment until a session token is entered (except for the authorization and notification fragment, which must also function without a session token). The first two are the subject of 12.2.

11.4 Communication: typed event bus

The fourth framework decision is most often posed as a single question, “how do microfrontends talk,” and that is the wrong question. The decision of this solution is not a single mechanism but **a rule by which the mechanism is chosen** , and the rule is: the mechanism is determined by *the topology* .

The basis of this rule is in the literature, but broken down into two places that together give it. (Rappl, The Art of Micro Frontends, 2024)In composing applications, it distinguishes three types of communication, events, shared data, and **component provisioning** , thus distinguishing them by *what* is being passed. (Geers, 2020)It distinguishes the same types by *where* the participants stand: parent to fragment, fragment to parent, fragment to fragment, and parts that are not on the same screen.

Table 11-2Mechanism according to topology, and what is transmitted by it

Participant relationship	**Selected mechanism**	**What travels?**	**Why not a highway?**

parent → fragment	element attributes	three scalars (session state, name, role)	the content already carries the relationship; the fragment is a pure function of its attributes
fragment → parent	custom event	only type of intent ("logout")	the parent is the only subscriber who can exist by position
fragment ↔ fragment on the same screen	typed event bus	selected diagnosis identifier	(highway *is* the choice here)
beyond the mounting limit	page address (parameters from the address)	appointment identifier	the bus doesn't remember, and the receiver isn't mounted yet
between browser tabs	Broadcast Channel API	only the type of change ("checked in"/"checked out")	the highway does not cross the document boundary

The wrapper passes the session context **to each** application, uniformly, as a composition property upon registration; this is exactly the parent channel. → fragment, and it was handed out to everyone because the wrapper who knows who needs what would be the participant who is most difficult to re-deliver.

The shared storage in this solution **does not contain domain data** , no exam, no patient, no diagnosis code passes through it; each fragment of its data reads itself through the client to the server layer. It holds the session state and nothing else. It should also be emphasized that the session does not belong to any subdomain, and any partition that would place it in one makes that part mandatory for all the others.

11.5 Decision framework

The obligation of this section is to fill in each of the decision questions studied so far, with filling in here meaning three things: what value the solution took, what the framework offered and the solution rejected, and **where in the source code that decision actually lives** . The fifth column is what makes a filled-in framework verifiable: a decision that cannot be shown in any file is not a decision but an intention.

Table 11-3Decision Framework Completed for MicroMedic

Decision	**The framework allows**	**Selected**	**Rejected, with a price**	**Carrier in code**

Type of division	horizontal or vertical	**both, in the same solution**	/	wrapper route table (*layout* , *exceptRoutes*)
Composition	client, edge, server	**client** , via *Module Federation*	server (loses first coloring), edge (does not exist)	wrapper build configuration (origin list)
Routing	client, edge, server	**client,** and only in a wrapper which makes it **first level routing**	route announcement from fragment (composition unreadable in one place)	route table and functions activations
Communication	event emitter, custom events, web storage, query string	**all four, by topology**	/	Table 12-2

The solution took both types of division. The artifact mounts a vertically divided part (the application takes up the entire field of view) and a horizontally divided screen (the overview and the catalog side by side) in the same document.

The communication selection framework is filled in completely, which is unusual. The framework lists four mechanisms as a list to choose from; the solution uses all four, because its topologies among the four parts are different.

11.6 Server layer

The server layer remains a single program with a single database. **The alternative existed and was obvious.** Section 10.4 had already separated the subdomains, so the server offered the same decomposition as the client: one self-deliverable program each for scheduling, review, catalog, medication registry, access, and reporting. The subject of this paper is **the client** partition. Splitting the server layer would introduce another independent variable into the study, so the difference in delivered behavior could no longer be attributed to what the paper is testing.

The solution has a triple access check and the layers vary in size. First are the path matches in the security setting, which are intentionally large: they divide the world into public (three login requests and three reads of reference data), what requires the doctor role (booking an appointment and book an appointment), and everything else, which requires a logged-in user. Second are the requested right markings **on the service methods** , there are eleven of them in the solution. Third is the data queue access guard, which enforces one rule: a doctor is allowed to read a patient only if there is a scheduled appointment between them, and a patient is allowed to read only his own. It is this rule that explains why the object graph in the next paragraph is the way it is.

This three-way arrangement is what makes the entire client-side partitioning secure, and it's worth explaining why. Six independently delivered fragments mean six places that can make a request, and none of them is the place where the decision is made. The right to each individual piece of data is decided on the server, on each request, regardless of which fragment made the request. If any of this were passed to the client, the solution would be less secure than the monolith it is being compared to.

The access trail, and what should never be included in it. The obligation in 10.6 is fulfilled by a separate *audit trail table. log*), whose content is deliberately reduced: who accessed, in what role, what type of resource, which resource, which patient and when. Six columns and nothing more, **no name, no diagnosis, no history, no free text.** The basis of this cut is the one that 10.6 sets as a requirement: the access trace is read by people who have no rights over the record itself, because their job is to monitor and not treat. A trace that would cite a record would, therefore, be another copy of the same record under weaker access rules, i.e. precisely what it is supposed to prevent. The rule is expressed in one line: **the trace records that access was made, never what was seen.**

Domain and relational model. There are thirteen mapped classes; user is abstract, with inheritance across joined tables and three subtypes, and **role is a subtype function rather than a stored column** , meaning that no entry can produce a user whose role does not exist. The four classes that carry medical records have “ soft deletion ” (engl. *soft delete*) and monitoring fields, so such a record is not removed but marked. The key decision, however, is in the form of the graph : the examination **does not carry** either the patient or the doctor directly, but reaches them only through the scheduled examination. This is what makes the access guard rule from the previous paragraph feasible as **a single** query over a single connection, instead of a set of rules per record type .

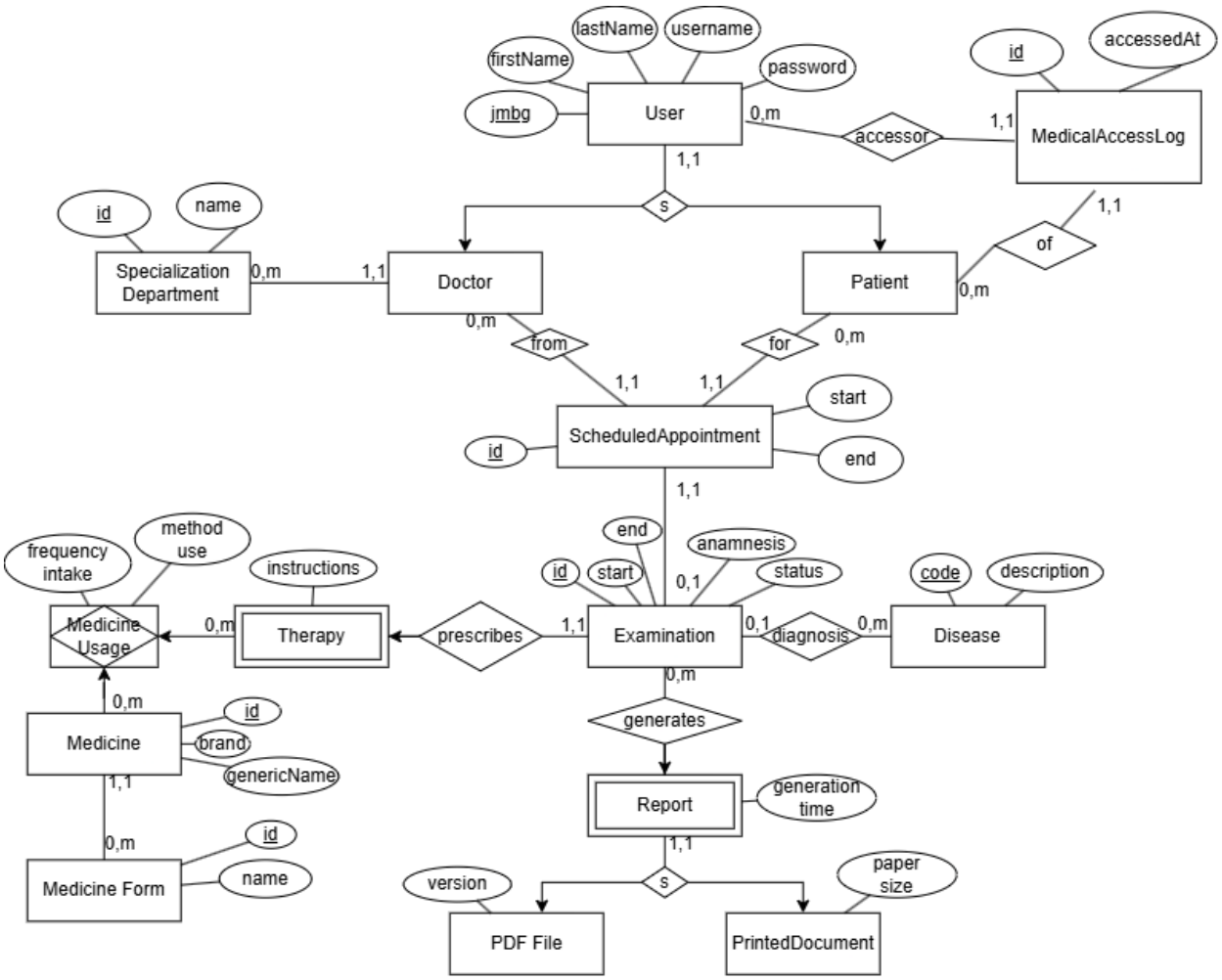


Figure 11-1 Extended Model of MicroMedic Connection Objects

12 Client layer implementation

The previous chapter outlined the decisions; this shows what was built from them. The subject is the client layer of the MicroMedic system: one repository, seven self-deliverable fragments in four different frameworks, and a wrapper that does not display anything of its own.

12.1 Mono-repository structure and workspaces

The client layer is **a single** mono-repository with thirteen packages under the *packages /* directory* , the root directory holding only the commands that are passed to the packages, a list of the checker dependencies, and a common base of *TypeScript* compiler settings. The claim that fragments are not coupled must therefore be proven by something else, the import graph between packages, and not by the fact that they can be shipped from different repositories, which this artifact does not show.

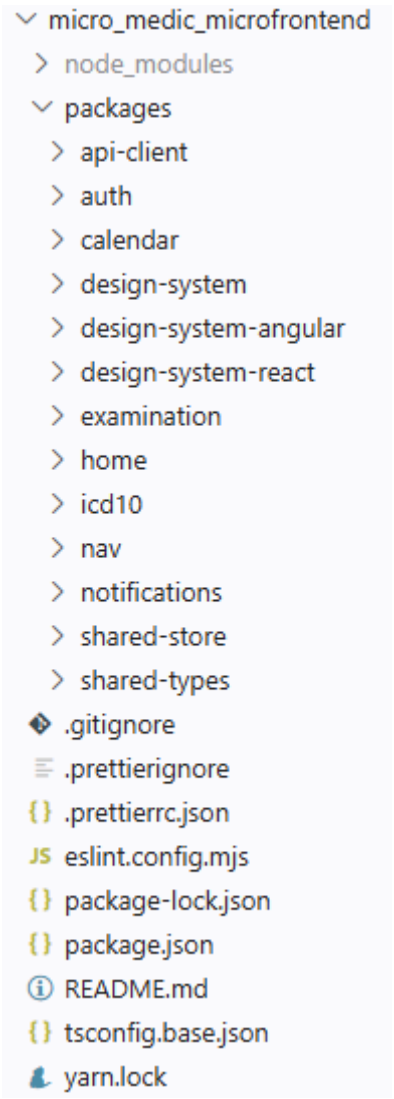


Figure 12-1 Client-layer package list

The thirteen packages fall into two types, which differ in what they produce. Seven **of them build their own artifact** : each has its own *webpack.config.js* and outputs a container named *remoteEntry.js*

. **Six do not build anything** , they are libraries (type contract, client-server, shared repository, and three design system packages), and their build command is *tsc -- noEmit* , which is just a check. Consumers do not import them as published packages, but as source code, via path mapping in *tsconfig.base.json* .

The libraries in this repository do not have their own deliverable artifact, but are built into each consumer, so there are so many copies of it at build time.

One detail shows what the mono repository actually shares, and it's not code. The *wsrun --parallel command serve* starts a static server in every workspace that has such a command.

12.2 Wrapper application: routing, activation functions, authentication gateway

The wrapper is the *home package* and it **doesn't expose anything of its own** : all its source is three *TypeScript files* , a route table, an activation function, and a launcher.

The route table is represented by the following code (packages / home / src / routes.ts):

```
const CHROME_HIDDEN_ROUTES = [ '/login , '/register' ] as const ;
export const ROUTES : read-only RouteEntry [ ] = [
  { name : 'header' , load : () => import ( 'nav/Header' ) ,
    routes : [ '*' ] , exceptRoutes : CHROME_HIDDEN_ ROUTES } ,
  { name : 'footer' , load : () => import ( 'nav/Footer' ) ,
    routes : [ '*' ] , exceptRoutes : CHROME_HIDDEN_ ROUTES } ,
  { name : 'notifications' ,
    load : () => import ( 'notifications/Notifications' ) ,
    routes : [ '*' ] , public : true } ,

  /*
* ---- Vertical split -----
  { name : 'auth', load: () => import('auth/Auth'),
routes: [LOGIN_ROUTE, '/register'], layout: 'full', public: true } ,

/*
* ---- Horizontal split ----
  { name : 'examination', load: () => import('examination/Examination'),
routes: ['/examination' ] } ,
  { name : 'icd10', load: () => import('icd10/ICD10'),
routes: ['/examination' ] } ,
  { name : 'calendar', load: () => import('calendar/Calendar'),
routes: [DEFAULT_ROUTE ] } ];
```

The four fields of the route table carry decisions that have consequences elsewhere. The application **name** is also the mount point, because the library mounts in the #single-spa-application:<name> element; this means that these names are a contract with the wrapper document, and that *the header*

and *footer are* named by their positions, not by the *nav package* they come from. **Loading is a function** , not a pre-loaded module, and this allows for lazy *loading* .

```
<! DOCTYPE html >
< html lang = " en " >
  <head>
  </head>
  <body>
    < giant id = " single-spa- application:auth " ></ div >

    < giant class = "mm-container" >
      < giant id = " single-spa- application:header " ></ div >
      < main id = "mm-main" tabindex = "-1" >
        < giant class = "mm-split mm-split --primary mm-region" >
          < giant id = " single-spa- application:examination " ></ div >
          < giant id = "single-spa- application:icd 10" ></ div >
        </div>

        < giant id = " single-spa-application:calendar " class = "mm - region" >
</div>
      </main>

      < giant id = " single-spa- application:footer " ></ div >
    </div>
    < giant id = " single-spa- application:notifications " ></ div >
  </body>
</html>
```

Matching is done on entire segments and login validation is used as a callback (packages / home / src / activity.ts , abbreviated):

```
export function matchesRoute ( pathname : string , route : string ) : boolean {
  if ( route === '*' ) return true ;
  const normalized = route . length > 1 && route.endsWith ( ' / ' ) ? route.slice (
0 , -1 ) : route ;
  return pathname === normalized || pathname . startsWith ( ` ${ normalized } /` );
}

export function createActivityFn ( entry : RouteEntry , isAuthenticated : () =>
boolean ) : ( location : Location ) => boolean {
  return ( location : Location : boolean => {
    if ( ! entry . routes. some ( route => matchesRoute ( location . pathname ,
route ))) return false ;
    if ( entry . exceptRoutes ?. some ( route => matchesRoute ( location . pathname
, route ))) return false ;
    return entry .public === true || isAuthenticated ( );
}; }
```

First: the comparison is by **entire path segments** , not by the start of the string. A mere match of the startsWith method would make / calendar match / calendar-archive , and / log match / login , thus

mounting the application on a page it knows nothing about. Second, and more importantly: isAuthenticated is passed as **a callback, not a boolean** . The library compiles this function once, and calls it on every route change; the captured boolean would, therefore, be what the session was at the start of the document, for the entire life of the document.

only wrapper routing is performed : no fragment keeps its own router. Two-level routing here, therefore, is not rejected as inferior, but was not necessary.

12.3 Topology of microfrontends

The following table lists the derived topology of seven packets (the wrapper and the six it registers).

Table 12-1Microfrontend topology in the derived system

Port	Package	Container name	Exposed modules	Frame
3001	home	home	— (pure host)	TypeScript , single-spa 6
3002	icd10	icd10	./ICD10	View 3
3003	here	here	./ Header , ./ Footer	custom elements
3004	examination	examination	./ Examination	Angular 22
3006	auth	auth	./Auth	React 19
3007	notifications	notifications	./ Notifications	custom elements
3009	calendar	calendar	./ Calendar	React 19

Remote modules are defined in the form name@address , for lazy loading purposes (packages / home /webpack.config.js, abbreviated) :

```
const REMOTES = {
 here : ' nav@http ://localhost:3003/remoteEntry.js' ,
 notifications : ' notifications@http ://localhost:3007/remoteEntry.js' ,
 icd10 : 'icd10@http://localhost:3002/remoteEntry.js' ,
 examination : ' examination@http ://localhost:3004/remoteEntry.js' ,
 calendar : ' calendar@http ://localhost:3009/remoteEntry.js' ,
 auth : ' auth@http ://localhost:3006/remoteEntry.js'
};

new ModuleFederationPlugin ( {
   name : 'home ' , library : { type : 'var' , name : 'home ' } ,
   filename : 'remoteEntry.js ' , remotes : REMOTES ,
   remoteType : 'script ' , shared : { ' single-spa ' : { singleton : true } } })
```

The name@address form is its own lazy build form: the container script is loaded on **first** import, and the remote module is only downloaded on the route that mounts it.

The image shows the loading of the main document, which pulls in the auth page as well as shared dependencies such as design-system, api -client, react, single-spa-react .

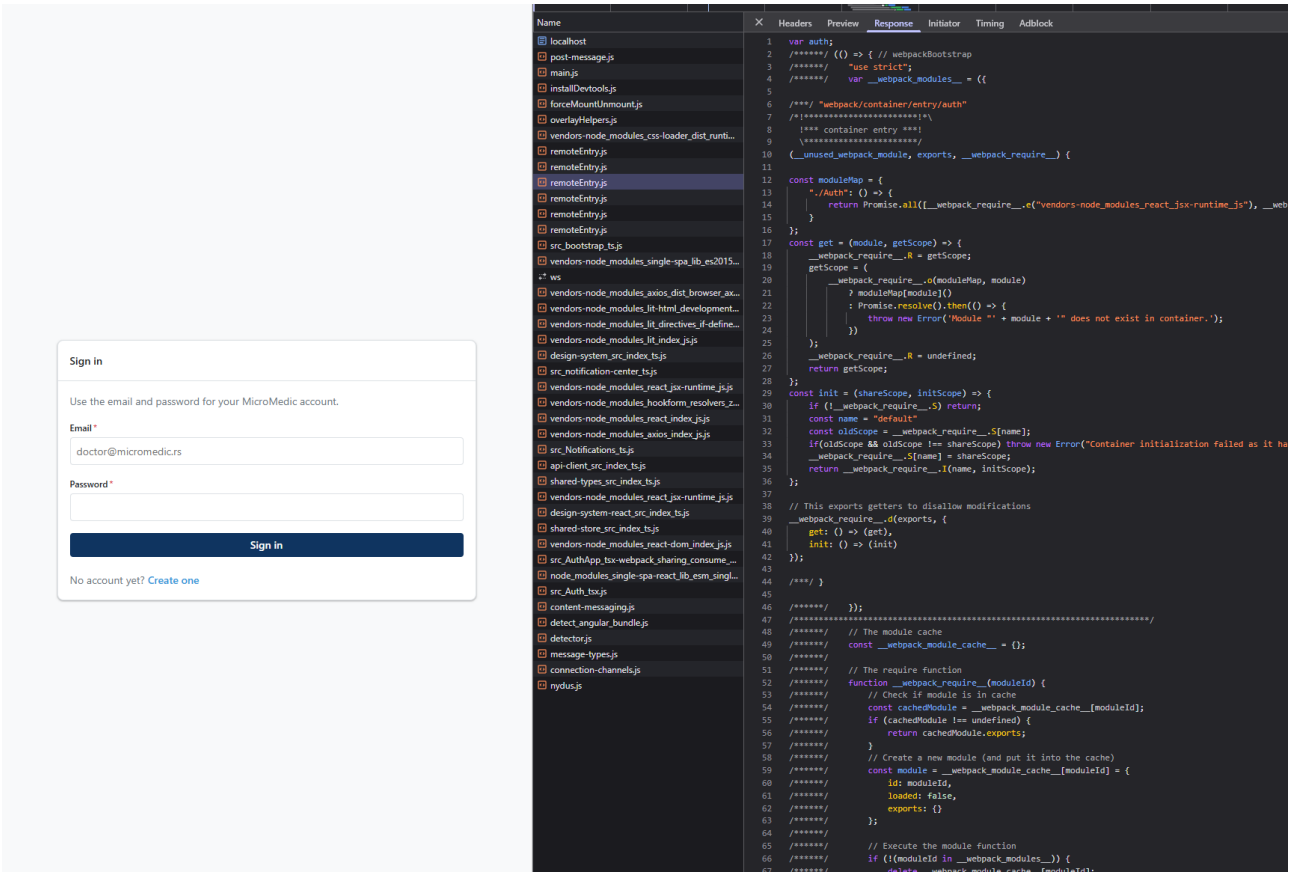


Figure 12-2 Initial pull of dependencies and auth packages

12.4 Vertical split: auth package

auth package is a derived example of vertical division. On the / login and / register routes, it **is not part of a composite page, but is a page** : this is the difference from the horizontal division in 12.5, where two fragments share a single viewport.

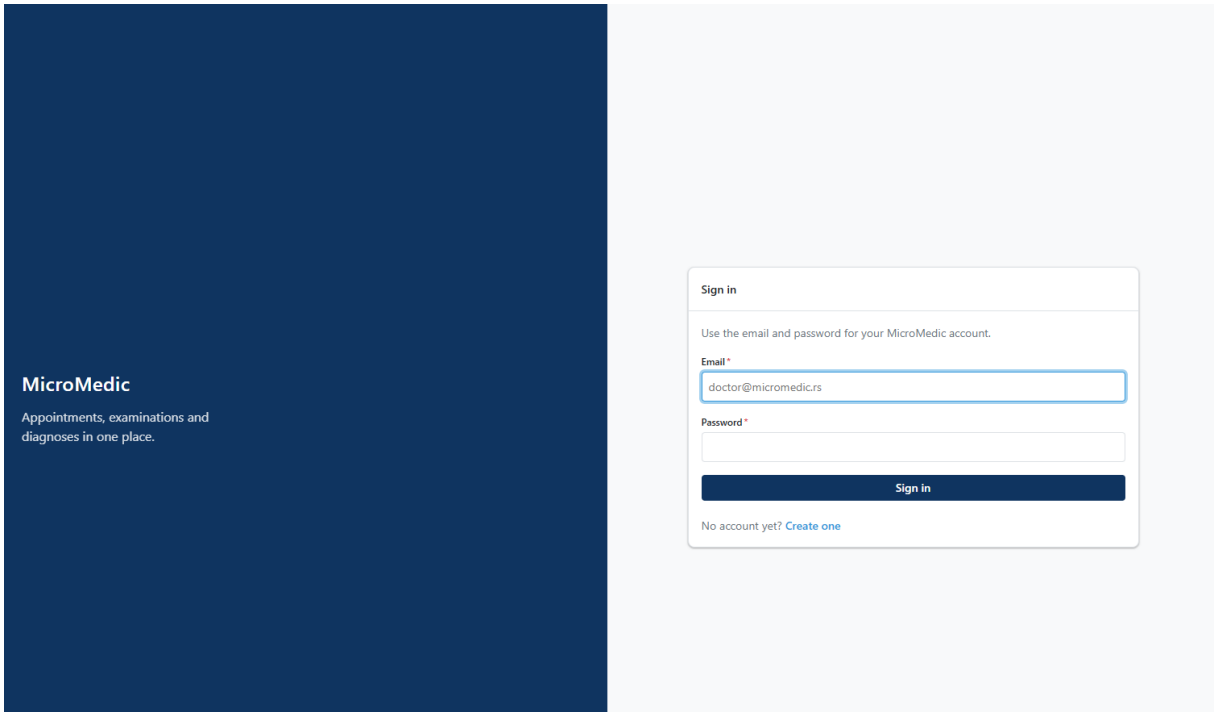


Figure 12-3Login page

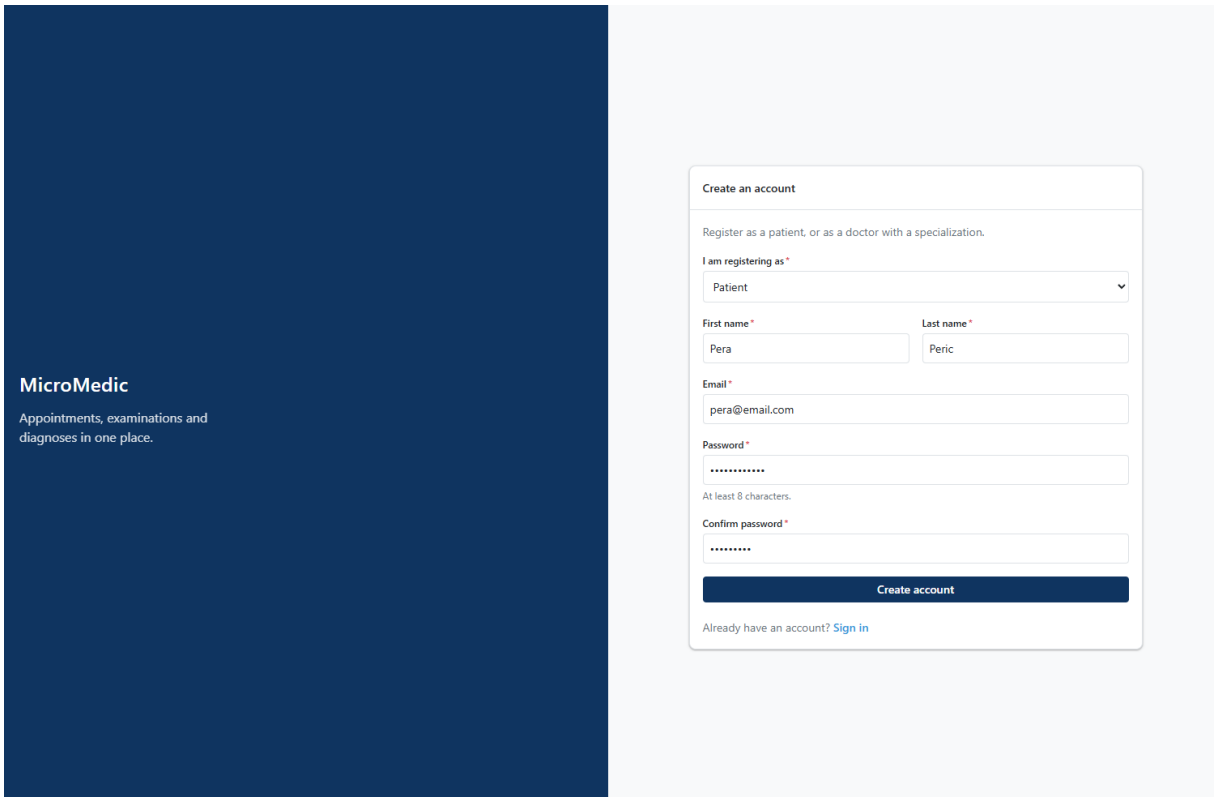


Figure 12-4Registration page

12.5 Horizontal division: examination with icd10

On the / examination route , the wrapper assembles **two independently delivered fragments into a single viewport** : the examination entry form, which is Angular 22 on port 3004, and the ICD-10 disease catalog, which is Vue 3 on port 3002.

The basis of the division is **domain-based, not technical** . The doctor writes a medical history during the examination, and along the way **consults** the diagnostic codebook . These are, therefore, two different jobs that take place at the same time on the same screen, and the division follows them.

Horizontal division in a wrapper document can be seen through two div tags located next to each other in the HTML definition (packages / home / public /index.html, for short):

```
< main id = "mm-main" tabindex = "-1" >
  < giant class = "mm-split mm-split --primary mm-region" >
    < giant id = " single-spa- application:examination " ></ div >
    < giant id = "single-spa- application:icd 10" ></ div >
  </div>
</main>
```

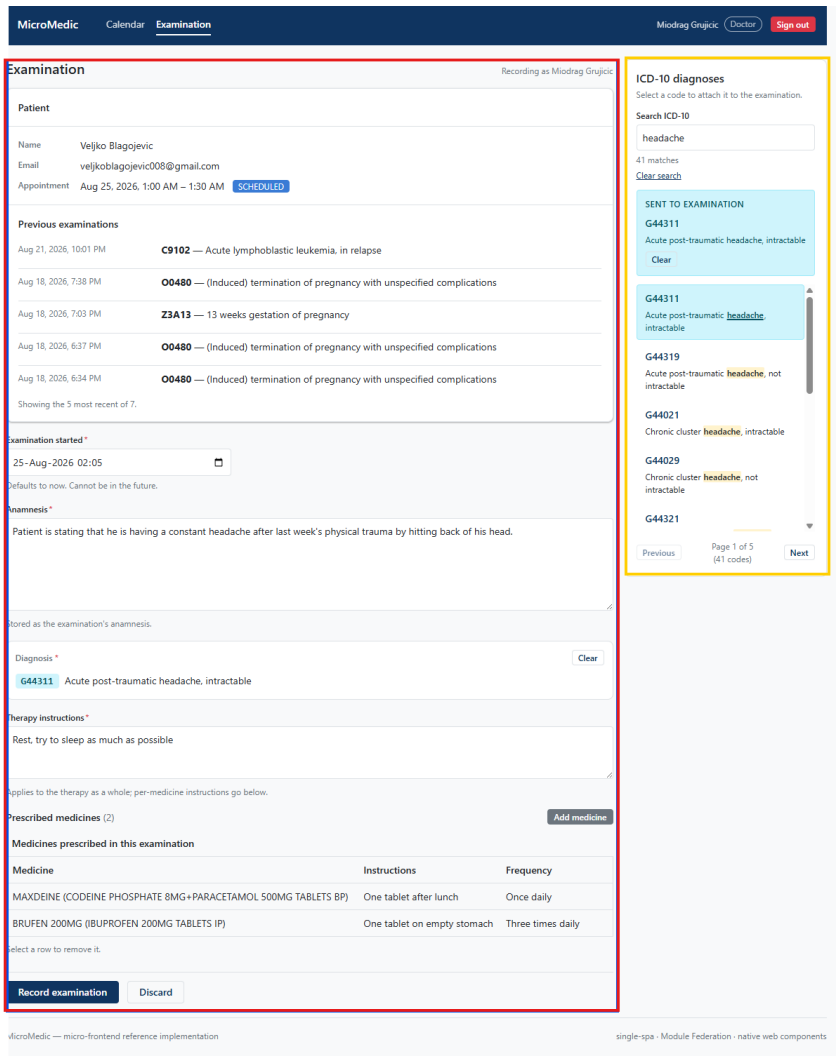


Figure 12-5Horizontal division of examination and icd10 fragments

In the image, the red frame on the left shows the examination fragment, while on the same screen on the right, framed in yellow , is a fragment called ICD10 (International Classification of Diseases-10) .

Neither fragment imports the other, and neither can. First, there is no import of the other in the source code of either package, the only coupling is the event bus and the shared elements of the design system. Second, and more importantly: an Angular component **cannot** import a Vue component, because they are two different display systems with different lifecycles. Here, the browser is the only possible point of integration; thus, the agreed-upon discipline is replaced by a physical barrier.

The contract between them is a single event: the catalog publishes the selected disease, and the form receives it and displays it in its diagnosis section.

12.6 Always mounted fragments: nav and notifications

Two fragments are not bound to any route: the header-footer, in the nav package on port 3003, and the notifications layer, in the notifications package on port 3007. Both are written as **custom elements in pure TypeScript , without any framework** .

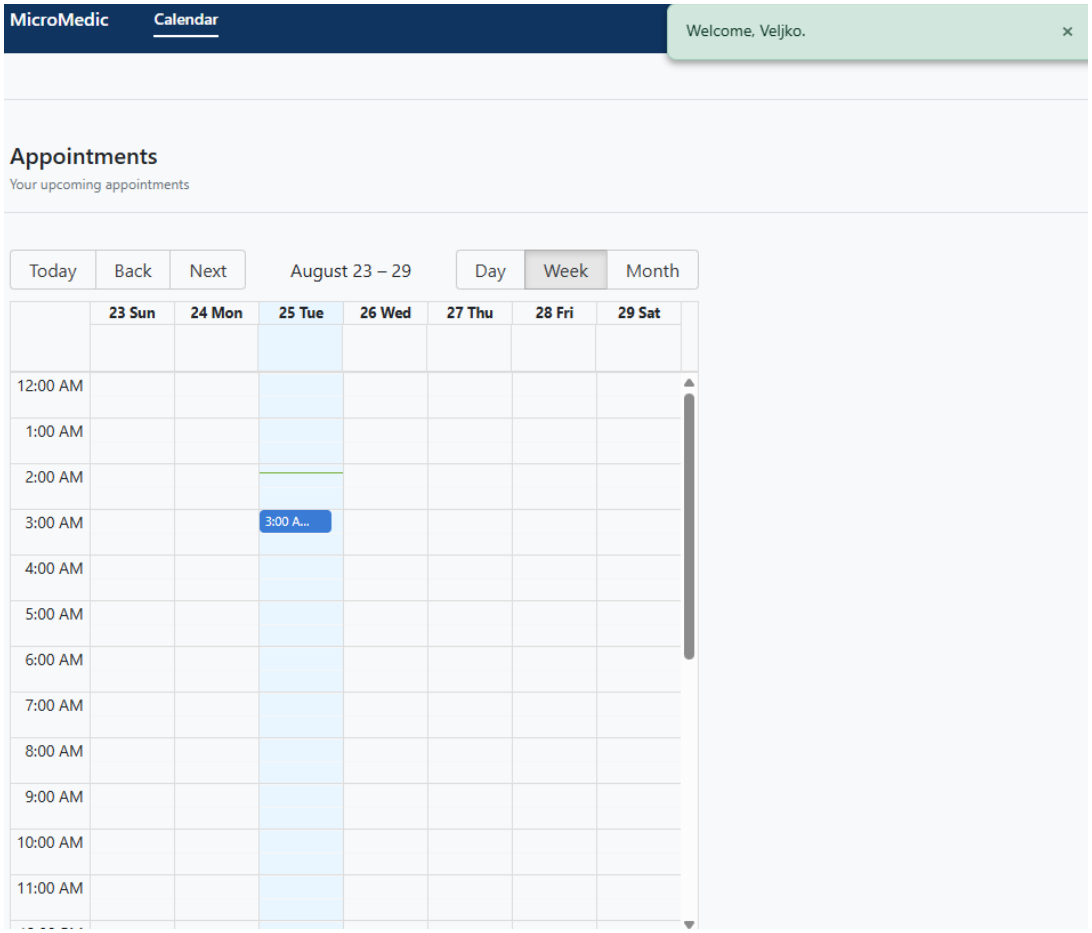


Figure 12-6Viewing nav and notifications fragments after login

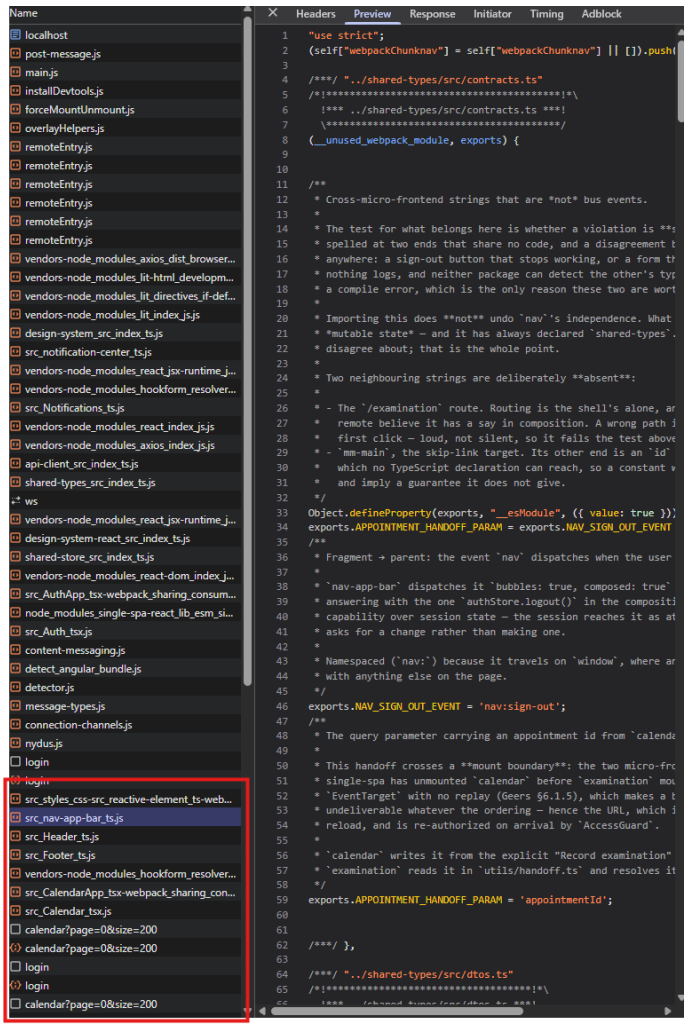


Figure 12-7 Loading calendar, nav, header and footer scripts only after user login

The navigation banner does not read shared storage. It receives the session state as element properties: whether the user is logged in, their name, and role, which are set by the application wrapper. This makes the fragment a pure function of its properties, and the session writer remains in one place.

The logout button on the navigation banner does not touch the session. It publishes an event that is raised, and the wrapper listens for it on the window and only then performs the logout on the shared storage. This way, the fragment communicates an intent, not a consequence, and remains without session write rights.

The notification layer has only two decisions of its own, and both are domain-specific rather than display-specific . The first is the duration by type, where **the error is persistent until it is resolved** , and the success message disappears after a few seconds. The second is that the entire notification sequence is **cleared on logout** , because the message may contain the patient name from the session that just ended, which is also a security decision.

12.7 Communication between fragments: derived mechanisms

The mechanisms are presented in 5.1–5.4, and selected ones in 11.4; only what is derived from them is stated here, where it is stated in the source code.

Table 12-2Derived communication mechanisms and the place of each in the artifact

Mechanism	Direction	A place in an artifact
Element properties	wrapper → fragment	session in header (nav / src / session-attributes.ts)
Raising custom event	fragment → envelope	sign out (nav:sign-out , handled in home / src / index.ts)
Event Highway	fragment → fragment neighbors on the same screen	diagnosis selection (icd10 → examination)
Unique resource indicator	across the mounting boundary i.e. using parameters from the address	submission of a scheduled examination (calendar → examination)
Broadcast Channel API	between cards of the same user	session modification (shared-store / src / session-channel.ts)

The first line represents the communication where the wrapper offers a session **to each** registered fragment, uniformly. The code for this mechanism is shown in the listing with `the projectSession` method. The next listing shows the translation of session state into element properties.

Session attributes are resolved in code (packages \ nav \ src \ session-attributes.ts , abbreviated):

```
export function toAttributes ( session : SessionSnapshot ) : AttributeMap {
  const authenticated = session .isAuthenticated && !! session .user ;
  const name = session . user ? ` ${ session . user . firstname } ${ session . user
. lastname } ` . trim ( ) : '' ;

  return {
    [ ATTR_AUTHENTICATED ]: authenticated ? ' ' : null ,
    [ ATTR_USER_NAME ]: authenticated && name ? name : null ,
    [ ATTR_USER_ROLE ]: authenticated ? ( session .role ?? session .user?. role ??
null ) : null ,
};
}

export const projectSession : AttributeProjector = ( props :
CustomElementMountProps ) : AttributeSource | null => {
  const session = props .session ;
  if ( session === undefined || session === null ) return null ;
  if ( ! isSessionContext ( session )) {
    return { get : () => toAttributes ( SIGNED_OUT ), subscribe : () => () => { } }
; }
  return {
    get : () => toAttributes ( session . getState ()),
    subscribe : ( onChange ) => session . subscribe ( onChange ),
}; } ;
```

The second line is shown in the code snippet that uses a custom named event (packages \ home \ src \index.js and packages \ nav \ src \ nav-app-bar.ts , abbreviated):

```
// navigation component
this . dispatchEvent ( new CustomEvent ( NAV_SIGN_OUT_EVENT , { bubbles : true ,
composed : true } ));

// index.ts
windows . addEventListener ( NAV_SIGN_OUT_EVENT , () => {
  authStore.logout ( );
} );
```

The third line is implemented using a service that manages the event bus (packages \ examination \ src \ state \ event-bus.service.ts).

```
@ Injectable ({ providedIn : 'root ' } )
export class EventBusService {
  private read-only destroyRef = inject ( DestroyRef );

  listen < T extends EventType >(
    event : T ,
    listener : ( payload : EventPayloadMap [ T ]) => void
) : void {
    const unsubscribe = eventBus. on ( event , listener );
    this . destroyRef . onDestroy ( unsubscribe );
}

  emit < T extends EventType >( event : T , ... args : EmitArgs < T >) : void {
    eventBus. emit ( event , ... args );
}

  notify ( type : ' success' | 'error' | 'info' | 'warning ' , message : string ) :
void {
    this . emit ( EventTypes. NOTIFICATION_SHOW , { message , type } );
} }

this . bus . listen ( EventTypes . ICD10_DISEASE_SELECTED , ({ disease }) => {
  this .diagnosisSignal. set ( disease );
  this .submitErrorSignal. set ( null );
} );
```

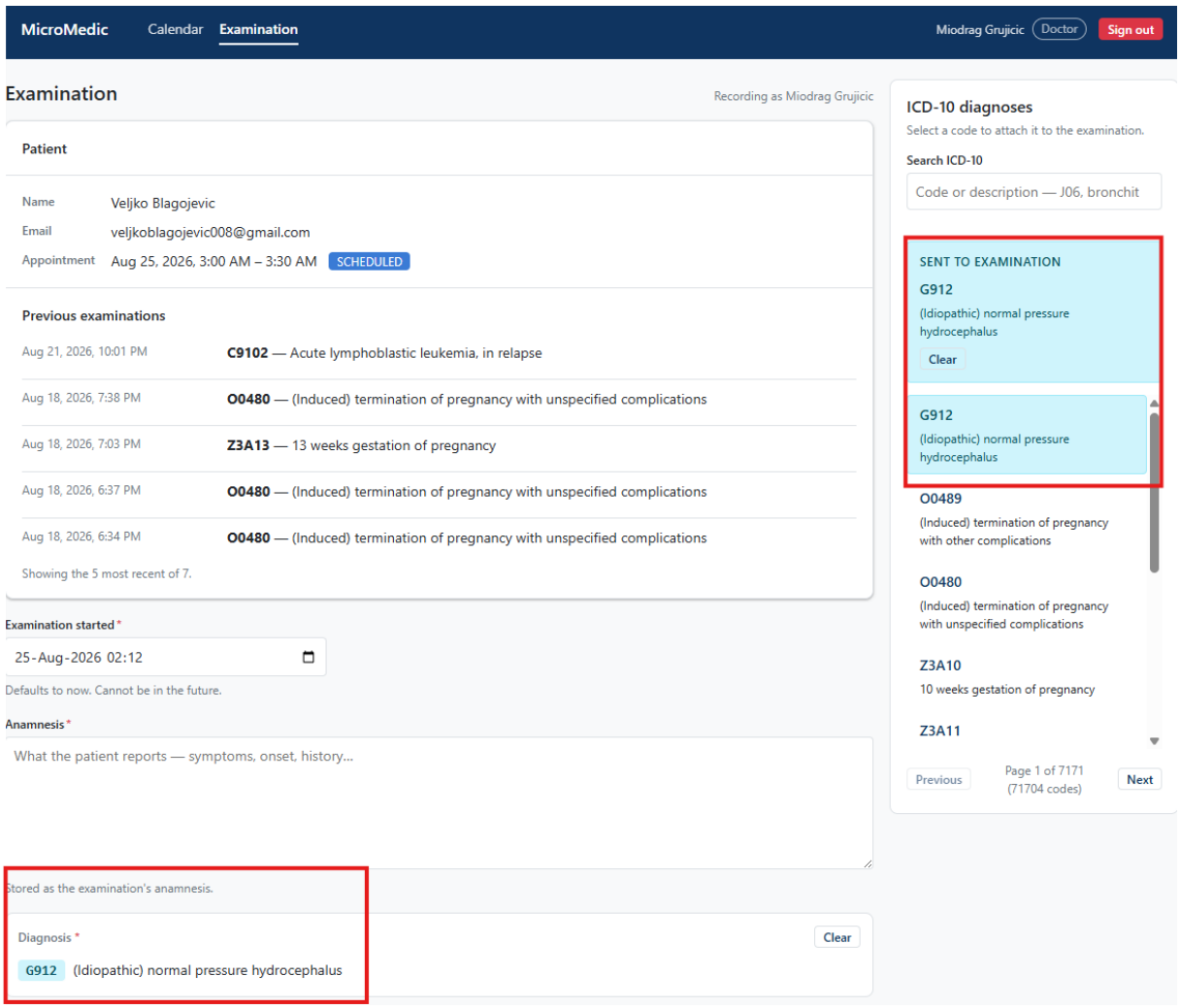


Figure 12-8 Selecting a diagnosis in the icd10 fragment triggers an event in the trunk that the examination fragment listens for and writes to the form.

The fourth line is shown by simply sending and reading the identifier as a parameter from the address (packages \ calendar \ src \ CalendarApp.tsx , packages \ examination \ src \ ExaminationApp.ts) :

```
const onRecordExamination = useCallback (
( appointment : ScheduledAppointmentDto ) => {
    navigate ( `/ examination? $ { APPOINTMENT_HANDOFF_ PARAM } = ${ appointment .
id } ` );
} ,
[ navigate ]
);
export function readAppointmentHandoff ( ) : number | null {
  if ( typeof window === 'undefined' ) return null ;

  const raw = new URLSearchParams ( window . location . search ). get (
APPOINTMENT_HANDOFF_PARAM );
  if ( raw === null || raw . trim () === '' ) return null ;
  const id = Number ( raw );
  if ( ! Number . isInteger ( id ) || id <= 0 ) return null ;
```

```
return id ; }
```

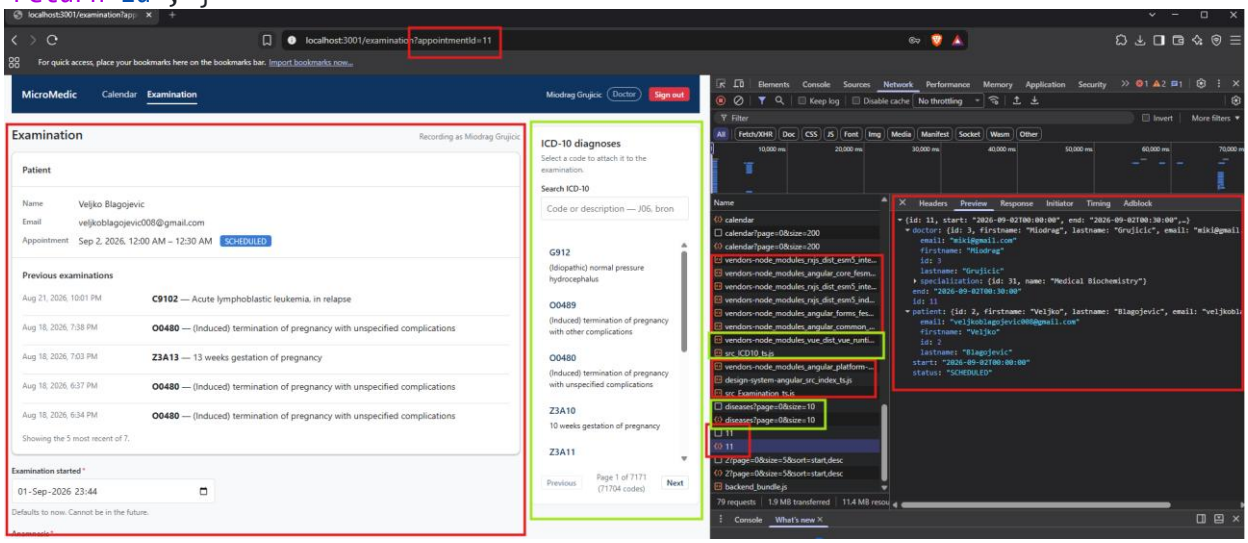


Figure 12-9 Parameter from the address as transmission data during transition and communication of calendar and examination fragments

Fifth row

```
export interface SessionMessage {
    type : 'signed-in' | 'signed-out' ;
}

export interface SessionChannel {
    publish ( message : SessionMessage ) : void ;
}

const CHANNEL_NAME = ' micro_medic :session ' ;

function isSessionMessage ( value : unknown ) : value with SessionMessage {
    if ( typeof value ! == 'object' || value === null ) return false ;
    const { type } = value ace { type ?: unknown } ;
    return type === 'signed-in' || type === 'signed-out' ;
}

export function openSessionChannel (
    onMessage : ( message : SessionMessage ) => void
) : SessionChannel | null {
    if ( typeof BroadcastChannel === 'undefined' ) return null ;

    let channel : BroadcastChannel ;
    try {
channel = new BroadcastChannel ( CHANNEL_NAME );
} catch (error) {
        console . warn ( '[ SessionChannel ] Could not open the session channel:' ,
error);
        return null ;
}
```

```
    channel. addEventListener ( 'message' , ( event : MessageEvent < unknown >) =>
{
        if ( isSessionMessage ( event . data )) onMessage ( event . data );
});

    return {
        publish ( message : SessionMessage ) : void {
            try {
                channel. postMessage ( message );
} catch (error) {
                console . warn ( '[ SessionChannel ] Could not publish a session
message:' , error);
}
},
};
}


export interface AuthState {
    token : string | null ;
    user : UserDto | null ;
    roles : Roles | null ;
    isAuthenticated : boolean ;
}

function readStoredUser ( ) : UserDto | null {
    flight raw : string | null = null ;
    try { raw = localStorage . getItem ( USER_KEY ) ; } catch { return null ; }
    if ( ! raw ) return null ;

    try {
        const parsed = JSON . parse (raw) as UserDto | null ;
        return parsed && typeof parsed === 'object ' ? parsed : null ;
} catch {
        console . warn ( '[ AuthStore ] Discarding unparseable persisted user.' );
        localStorage . removeItem ( USER_KEY );
        return null ;
}
}

function readStoredToken ( ) : string | null {
    try { return localStorage . getItem ( TOKEN_KEY ) ; }
    catch { return null ; }
}

function createAuthStore ( ) {
    flight token : string | null = readStoredToken ( );
    let user : UserDto | null = readStoredUser ( );
    flight isAuthenticated : boolean = !! token && !! user;
    const subscribers : Set < Subscriber > = new Set < Subscriber >( );
```

```
    flight sessionChannel : SessionChannel | null = null ;

    configureApiClient ( {
        getAuthToken : () => token,
        onUnauthorized : () => store . logout (),
});

    function getState ( ) : AuthState {
        return { token , user, role : user?. role ?? null , isAuthenticated } ;
}

    function notify ( ) : void {
        const snapshot = getState ( );
        subscribers . forEach (( subscriber ) => {
            try { subscriber ( snapshot ) ; }
            catch (error) { console . error ( '[ AuthStore ] Error in subscriber
callback:' , error) ; }
});
}

    function persist ( ) : void {
          if (token) localStorage . setItem ( TOKEN_KEY , token );
          else localStorage . removeItem ( TOKEN_KEY );

          if (user) localStorage . setItem ( USER_KEY , JSON . stringify ( user ));
          else localStorage . removeItem ( USER_KEY );
}
}

    const store = {
        getState ,

        login ( newToken : string , newUser : UserDto ) : void {
token = newToken ; user = newUser ; isAuthenticated = true ;
            persist ( ); notify ( );
            eventBus. emit ( EventTypes . AUTH_LOGIN , { user : newUser } );
            sessionChannel ?. publish ({ type : 'signed-in ' } );
} ,

        logout ( ) : void {
token = null ; user = null ; isAuthenticated = false ;
            persist ( ); notify ( );
            eventBus. emit ( EventTypes . AUTH_LOGOUT );
            sessionChannel ?. publish ({ type : 'signed-out ' } );
} ,

        subscribe ( subscriber : Subscriber ) : () => void {
            subscribers. add ( subscriber );
            return () => { subscribers . delete ( subscriber ) ; } ;
```

```
} ,
};

    function applyExternalSession ( nextToken : string | null , nextUser : UserDto
| null ) : void {
        const nextIsAuthenticated = !! nextToken && !! nextUser ;
        if ( nextToken === token && nextIsAuthenticated === isAuthenticated )
return ;

token = nextToken ; user = nextUser ; isAuthenticated = nextIsAuthenticated ;

        notify ( );

        if ( nextIsAuthenticated && nextUser ) eventBus. emit ( EventTypes .
AUTH_LOGIN , { user : nextUser } );
        else eventBus. emit ( EventTypes . AUTH_LOGOUT );
}

    if ( typeof window ! == 'undefined' ) {
        sessionChannel = openSessionChannel (( message ) => {
            if ( message .type === 'signed-out' ) applyExternalSession ( null ,
null );
            else applyExternalSession ( readStoredToken ( ), readStoredUser ( ));
});

        windows . addEventListener ( 'storage' , ( event ) => {
            if ( event . key ! == null && event . key ! == TOKEN_KEY && event . key
! == USER_KEY ) return ;
            applyExternalSession ( readStoredToken ( ), readStoredUser ( ));
});
}

    return store ;
}
```

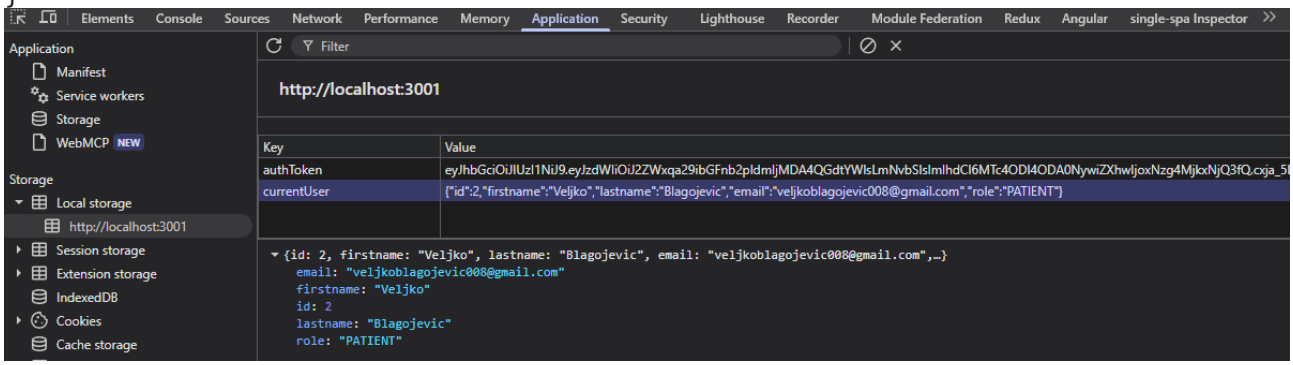


Figure 12-10Web storage with session token

12.8 Design system: one performance, four environments

The implementation of the shared display dictionary is **eleven custom elements** written with the Lit library (mm-button, mm-card, mm-empty-state, mm-error-state, mm-input, mm-modal, mm-select,

mm-spinner, mm-table, mm-toast, mm-toast-region), in a single package and **without any repetition per environment** . Their registration in the browser is a consequence of the package import itself , not a call that the consumer makes, which means that the fragment that wants the elements imports the package and that's all its work.

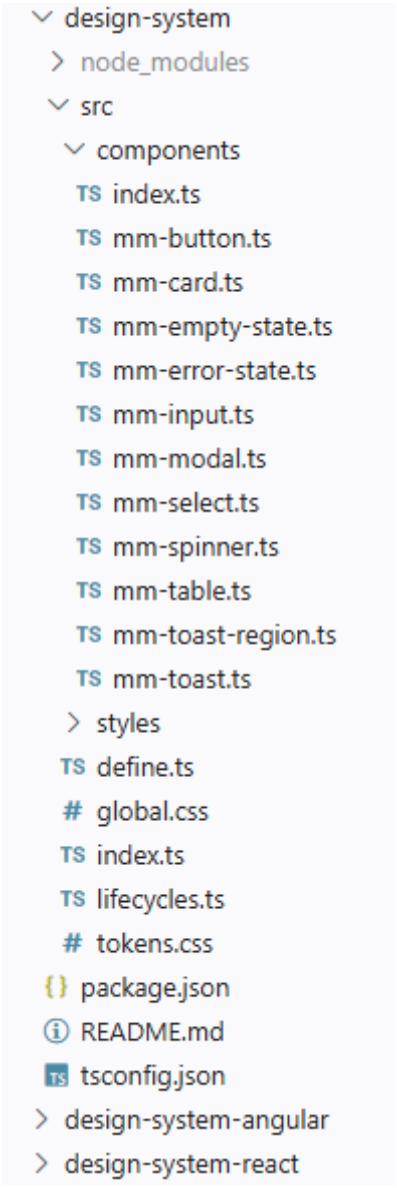


Figure 12-11 List of components within the design system

An example of a component, a button, is shown in the following listing. It needs to implement the render() method inherited from its parent LitElement classes. All other custom components follow a similar approach.

```
import { LitElement , html, css } from 'lit' ;

export type ButtonVariant = 'primary' | 'secondary' ;
export type ButtonSize = ' sm ' | 'md' | 'lg' ;
export type ButtonType = 'button' | 'submit' | 'reset' ;
```

```
export class MmButton extends LitElement {
    accessor variant : ButtonVariant = 'primary' ;
    accessor size : ButtonSize = 'md' ;
    accessor disabled = false ;
    accessor loading = false ;
    accessor type : ButtonType = 'button' ;
    accessor label = '' ;

    private handleClick ( event : Event ) {
        if ( this .disabled || this .loading ) {
            event . preventDefault ();
            event . stopImmediatePropagation ();
            return ;
}
        if ( this .type === 'submit' || this .type === 'reset' ) {
            const form = this . closest ( 'form' );
            if ( form ) {
                event . preventDefault ();
                if ( this .type === 'submit' ) {
                    form.requestSubmit ( ) ;
} else {
                    form.reset ( ) ;
}
}
}
}

    render ( ) {
        const busy = this .disabled || this .loading ;
        return html `
<button
class="variant- ${ this . variant } size- ${ this . size } "
type= ${ this . type }
?disabled= ${ busy }
aria-busy= ${ this . loading ? 'true ' : 'false' }
aria-label= ${ ifDefined ( this . label ?? undefined ) }
@click= ${ this . handleClick }
>
                ${ this . loading
? html `<span class="spinner" part="spinner"></span>`
: html `<slot> ${ this . label } </slot>` }
</button>
` ;
}
}

defineElement ( 'mm-button' , MmButton );
```

How are these elements written by Web Components standard, can be used in any framework but some adaptation is required. For the React framework, you need to use createComponent method.

However, so that every fragment written in the React framework does not have to make this adaptation, a middleware package has been created as a central place that all fragments implemented in the React framework can use. A code snippet demonstrating this adaptation is in the listing (packages / design-system-react / src / create-component.ts , abbreviated):

```
export function createComponent <
  TElement extends HTMLElement ,
  TEvents extends Record < string , EventName | string > = {},
>( tagName : string , elementClass : { new () : TElement ; prototype : TElement } ,
events ?: TEvents ) {
  return litCreateComponent ( {
react : React ,
    tagName ,
    elementClass ,
events,
});
}

export const MmButton = createComponent ( 'mm-button' , MmButtonElement );

export const MmInput = createComponent ( 'mm-input' , MmInputElement , {
    onInput : 'mm-input' ,
    onChange : 'mm-change' ,
    onBlur : 'mm-blur' ,
} as const );
```

A similar approach to implementing a proxy package was used for the Angular variant:

```
@ Directive ( )
export abstract class MmElementDirective < TElement extends HTMLElement >
implements OnChanges {
  protected read-only elementRef = inject < ElementRef < TElement >>( ElementRef );
  get nativeElement ( ) : TElement {
    return this . elementRef .nativeElement ;
}
  protected read-only nonForwardedInputs : ReadonlySet < string > = new Set
<string> ( ) ;
  ngOnChanges ( changes : SimpleChanges ) : void {
    const element = this . elementRef .nativeElement ace unknown ace Record <
string , unknown >;

    for ( const [ name , change ] of Object . entries ( changes )) {
      if ( this . nonForwardedInputs . has ( name )) continue ;
      if ( change .currentValue === undefined ) continue ;
      element [ name ] = change .currentValue ;
}
}
```

```
  protected forward < T >( emitter : EventEmitter < CustomEvent < T >>, event :
Event ) : void { emitter . emit ( event ace CustomEvent < T >);}
```

The Vue variant could have been implemented in an equivalent way, but as a demonstration example it was not. Vue is the control case here, it sets the non-primitive value as a property of an object in the document, and registers the event listener in the usual way, so it does not need an intermediary package and **design-system-vue is intentionally not present** . It only requires one rule at build time (packages /icd10/webpack.config.js, abbreviated):

```
compilerOptions : { isCustomElement : ( tag ) => tag . startsWith ( 'mm-' )}
```

The accumulation of design system packages and their use through fragments in the MicroMedic system can be seen formalized in Table 13'3.

Table 12-3Adaptation per consumer and the deficiency it compensates for

Consumer	Adaptation
nav , notifications (custom elements)	nothing
ICD10 (View 3)	one construction rule and one type declaration
calendar , auth (React 19)	Intermediate package design-system-react
examination (Angular 22)	Intermediate package design-system-angular

12.9 Shared state and shared access to services

Shared variable state is listed among the anti- patterns in 9.4 , so the derivation here is required to show under what conditions it is implemented. Three constraints are what separate this state from the anti-patterns in 9.4.

- First, **the content is session-only** , no domain data is inside the shared state, so there is nothing that two fragments can find in the state other than who is logged in, nor do they have any implicit communication over the shared state.
- Second, **access is intentionally locked down.** All fragments except writers get a "frozen copy" (via *JavaScript The Object.freeze ({object}))* method freezes the session state object, so it is a value that cannot be written to. Subscribing to changes and reading the current state are all their accesses.
- Third, **the writers** are named and **few in number** : the login fragment writes the session on successful login, the wrapper deletes it on logout, and the client to the server layer deletes it when the server layer rejects the request due to an invalid token. No other fragment can write, because it is handed a frozen copy without write capabilities.

12.10 Multiple frames as intentional observation

Section 9.3 calls multiple frameworks without management mechanisms anarchy . MicroMedic, a system that carries **three frameworks, one custom element library, and two framework-less fragments** , React 19, Angular 22, Vue 3, the Lit library , and, in two fragments, no framework, is required to demonstrate management, not just output.

There are four control mechanisms actually in place in the artifact, and each of them prevents a specific form of anarchy:

1. **One design system for all frameworks** (12.8). Without it, each framework brings its own collection of components, which is a form of anarchy that is first seen visually in the user interface.
2. **Contracts are defined using types** . Without it, each party assumes the form of what it receives, so inconsistencies only become visible at runtime.
3. **One router and one assembly point** (12.2). No remote module decides its own route.
4. **One mandatory check in delivery** (12.11), with one order for all packages.

The fourth item carries a finding that was found in performance and that, in managing multiple frameworks, is easy to miss: **the strength of management is the strength of the weakest check.**

Five frames in a single document are five runtime environments to load, and the repetition of shared dependencies among independently delivered content is a cost that sharing mitigates but does not eliminate. The claim of this chapter is not that This access is free and without consequences, but it is **monitored** .

12.11 Implemented security measures

Several security measures have been implemented, on both the client and server side, independently.

Several important measures have been implemented on the server side (11.6):

- CORS requests , reduced to fragment ports;

```
@Configuration
public class CorsConfig {

    @Value ( "${app.cors.allowed-origins}" )
    private String allowedOrigins ;

    @Bean
    public CorsConfigurationSource corsConfigurationSource () {
        org.springframework.web.cors.CorsConfiguration configuration = new
```

```
org.springframework.web.cors. CorsConfiguration ();
        List < String > parsedAllowedOrigins = Arrays . stream (
allowedOrigins .split ( " , " ))
.map( String ::trim)
.filter( origin -> ! origin .isBlank ())
. toList ();
        configuration .setAllowedOrigins ( parsedAllowedOrigins );
        configuration .setAllowedMethods ( java.util.List . of ( "GET" ,
"POST" , "PUT" , "DELETE" , "OPTIONS" ));
        configuration .setAllowedHeaders ( java.util.List . of ( "*" ));
        configuration .setAllowCredentials ( true );
        org.springframework.web.cors.UrlBasedCorsConfigurationSource
source = new org.springframework.web.cors. UrlBasedCorsConfigurationSource
();
        source .registerCorsConfiguration ( "/**" , configuration );
        return source ;
}}
```

- authorization on **service methods** and not on the controllers themselves;

```
@Slf4j
@Service@RequiredArgsConstructor
public class CalendarService {

    public static final int APPOINTMENT_MAX_DURATION_HOURS = 2 ;
    private final ScheduledAppointmentRepository
scheduledAppointmentRepository ;
    private final AccessGuard accessGuard ;


    @PreAuthorize ( "hasAuthority('ROLE_DOCTOR')" )
    @Transactional
    public ScheduledAppointment createAppointment ( AppointmentRequest
appointment ) {
        if ( appointment == null ) {
            throw new IllegalArgumentException ( "Appointment request
cannot be null" );
}

        if ( appointment . end (). isBefore ( appointment . start ())) {
            throw new IllegalArgumentException ( "Appointment end time
cannot be before start time" );
}

        if ( appointment .end().isAfter( appointment .start().plusHours(
APPOINTMENT_MAX_DURATION_HOURS ))) {
            throw new IllegalArgumentException ( "Appointment duration
cannot exceed " + APPOINTMENT_MAX_DURATION_HOURS + " hours" );
}

… return scheduledAppointmentRepository .save ( scheduledAppointment );
}
```

- data row ownership verification, which also creates **a record in the access trail for each access to medical data** ;

```
@Service
@RequiredArgsConstructor
public class AccessGuard {

    private final UserService userService ;
```

```
        private final MedicalAccessRecorder recorder ;

    …

        @Transactional
        public void requirePatientAccess ( Long patientId ) {
    User current = userService . getCurrentUser ();
            boolean isSelf = current instanceof Patient && current .getId
    ().equals( patientId );
            boolean isTreatingDoctor = current instanceof Doctor
    && scheduledAppointmentRepository.existsByDoctorIdAndPatientId( current
    .getId(), patientId );

            if ( isSelf || isTreatingDoctor || isAdmin ( current )) {
                recorder .record (
    MedicalAccessLog.AccessedResourceType.PATIENT , patientId , patientId ,
    current );
                return ;
    }

            throw new UnauthorizedActionException ( "User does not have access
    to patient with ID: " + patientId );
    }}
```

- refusing to launch the application if the token signing secret is invalid;

```
@Service
public class JwtSecretValidator {

    private static final int MIN_KEY_BYTES = 32 ; // H256
    @Value ( "${ app.jwt.secret :}" )
    private final String secretKey ;

…

    @PostConstruct
    void validate ( ) {


        if ( secretKey == null || secretKey .trim ( ). isEmpty ()) {
            fail( " app.jwt.secret is missing or blank. Set the JWT_SECRET env
variable to a Base64-encoded key of at least 256 bits" );
            return ;
    }

        int keyBytes ;
        try {
            keyBytes = java.util .Base64 . getDecoder ( ). decode ( secretKey ).
length
    } catch ( RuntimeException e ) {
            fail( " app.jwt.secret is not a valid Base64-encoded string" );
            return ;
    }
        if ( keyBytes < MIN_KEY_BYTES ) {
            fail( String . format ( " app.jwt.secret is too short. It must be at
least %d bytes (256 bits), but was %d bytes" , MIN_KEY_BYTES , keyBytes ));
    }
  }
}
```

- Limiting the number of requests on API access points.

```
@Component
public class RateLimitingFilter extends OncePerRequestFilter {

    private static final int CAPACITY = 20 ;
    private static final Duration WINDOW = Duration.ofMinutes ( 1 );
    private static final int MAX_TRACKED_CLIENTS = 10_000 ;

    private final Map< String , Bucket> buckets = new ConcurrentHashMap <>();

    private final ObjectMapper objectMapper ;
    @Value ( "${app.rate-limit.trust-forwarded-for:false}" )
    private final boolean trustForwardedFor ;

…

    @Override
    protected void doFilterInternal (
            @NonNull HttpServletRequest request ,
            @NonNull HttpServletResponse response ,
            @NonNull FilterChain filterChain )
            throws ServletException , IOException {

        String path = request.getRequestURI ( );
        if (! path .startsWith ( "/ api /auth" )) {
            filterChain .doFilter ( request , response );
            return ;
}

Bucket bucket = resolveBucket ( getClientIp ( request ));
        if ( bucket == null ) {
reject( response , WINDOW .toSeconds ());
            return ;
}

        ConsumptionProbe try = bucket .tryConsumeAndReturnRemaining ( 1 );
        if ( probe .isConsumed ()) {
            filterChain .doFilter ( request , response );
} else {
reject( response , secondsUntilRefill ( try ));
} }

    private void reject ( HttpServletResponse response , long retryAfterSeconds
) throws IOException {
        ApiError apiError = new ApiError (
                HttpStatus.TOO_MANY_REQUESTS.value (),
                "Too many requests. Please try again later." ,
                null ,
                LocalDateTime.now ()
);

        response .setStatus ( HttpStatus.TOO_MANY_REQUESTS.value ());
        response .setContentType ( MediaType.APPLICATION_JSON_VALUE );
        response .setHeader ( HttpHeaders.RETRY_AFTER , Long . toString (
retryAfterSeconds ));
        response .getWriter().write( objectMapper .writeValueAsString( apiError
));
}

    private @Nullable Bucket resolveBucket ( String clientIp ) {
Bucket existing = buckets .get ( clientIp );
        if ( existing != null ) {
            return existing ;
}
```

```
        if ( buckets .size () >= MAX_TRACKED_CLIENTS ) {
            evictRefilledBuckets ();
            if ( buckets .size () >= MAX_TRACKED_CLIENTS ) {
                return null ;
} }

        return buckets .computeIfAbsent ( clientIp , k -> createNewBucket ());
}

    private void evictRefilledBuckets () {
        buckets .values (). removeIf ( bucket -> bucket .getAvailableTokens ()
>= CAPACITY );
}

    private String getClientIp ( @NonNull HttpServletRequest request ) {
        if ( trustForwardedFor ) {
            String forwarded = request .getHeader ( "X-Forwarded-For" );
            if ( forwarded != null && ! forwarded .isBlank ()) {
                return forwarded .split ( " , " , 2 )[ 0 ].trim();
} } return request .getRemoteAddr ();
}}
```

Security measures implemented on the client side:

- **isolating styles** by display areas of design system elements;
- **text normalization over anything that crosses the trust boundary** in the header;
- **deleting the logout notification string** (12.6), as the message may contain the patient's name from the session that has ended;
- **session as a state with one writer and non-write access** for all others (12.9).

13 Conclusion

The master's thesis covers the principles, techniques, advantages and disadvantages of implementing microfrontend architecture. The concept of software architecture and architectural characteristics as a criterion by which architectural styles are compared, monolithic and distributed styles and the emergence of frontend monoliths as a consequence of the growth of single-page applications are presented. Then, the microfrontend architecture itself and its mechanisms are discussed: composition and routing techniques, communication methods between interface parts, system design as a carrier of appearance consistency, security boundaries and, ultimately, anti-patterns that such a division enables. Along with this theoretical part, a valued artifact was built: MicroMedic , an information system for scheduling and recording medical examinations, whose client layer is implemented as multiple independently deliverable microfrontends intentionally written in different frameworks.

The most controversial decision that microfrontend architecture allows is the use of multiple frameworks in a single user interface, which the literature often describes as an anti-pattern. Building and evaluating artifacts has shown that this assessment is not a property of diversity, but a consequence of the absence of a management mechanism. The diversity of frameworks in itself does not create a non-conforming appearance or a difficult maintenance ; they are created by the fact that no one has specified which decisions are common and how they are implemented. Once these decisions are specified - one implementation of each common component, one hierarchy of presentation layers, one way to address the server layer, one typed channel for exchange between fragments, and mandatory type checking in all packages - the use of multiple frameworks remains a controlled decision. What separates controlled multi - framework from anarchy is not the number of frameworks, but the existence of these decisions and the existence of a way to implement them.

The same finding also suggests that consistency of appearance can be transferred to other solutions. Consistency between parts of an interface written in different frameworks does not have to be maintained by the author's discipline, but can be ensured by construction. If common components are written once, as elements that the browser itself knows, and if they place their display in the DOM "under the shadow", external code cannot change its interior, not because it is forbidden but because it is technically impossible, which is evidence of a good infrastructure in place, where the system does not allow violations of established rules, even if there is such an intention. Any change in appearance then applies to all parts simultaneously, regardless of the framework in which the part was written. It also turned out that the connecting layer between such components and frameworks is not a cost that is paid for each framework, but rather a compensation for a precisely defined deficiency of a single framework: one of the frameworks used requires nothing but a single line of code. Thus,

the common claim that multi- framework necessarily multiplies the amount of code loses its generality.

The third finding concerns communication between interface parts. The way in which two parts communicate is not determined by which parts they are, but by their relationship in the composition of the screen. A part contained in another part receives data from properties set by its parent, and sends the notification back through an event that is raised to the parent components. Parts displayed next to each other on the same screen communicate over a common channel. When moving from one screen to another, however, the data is carried in the address, because the other part does not yet exist at the time of sending. It follows that reachability is a property of the channel, not a pair of participants: before asking how two parts will communicate, there is the question of whether they have ever been displayed simultaneously.

The fourth finding determines how far the microfrontend architecture goes in terms of security. It introduces real boundaries between areas of responsibility, but it cannot establish security boundaries on the client side. Every check in the wrapper or in a part of the interface is a navigation and serves to prevent the user from reaching a blank or meaningless screen, while granting access to data remains on the server layer and must be performed with each request. Isolation between parts, in the sense in which it is a property of separate processes, is not even a possibility, because all parts of a page share a single executable environment of a single document. For the domain in which medical data is processed, this is not a formality: confidentiality and completeness of the access trace cannot become a property of the interface division, but remain an obligation of the server layer, and the division of the interface must not make them less transparent.

The microfrontend architecture is paid for in the source code: one project becomes multiple projects, each shared dependency becomes a subject of negotiation, and the assembly of the whole is moved to the browser. What is gained in return is that one team publishes its part without asking the others, that different rhythms of change are separated and that the failure of one part does not bring down the rest of the system, is by nature tied to the delivery process and becomes visible only through it. Hence the scope of the evaluation in this paper: the properties expressed in the solution structure are verified on the built artifact, while those expressed in its delivery remain described.

The aim of the paper was not to present the microfrontend architecture as a better choice than the monolithic one. Its benefits are by nature delivery benefits, so the decision to implement it is made based on whether there are multiple teams that change the same interface at different rates and the need to publish their parts independently of each other. Where this is the case, everything that has been established in this paper about boundaries, appearance consistency, communication and security

boundaries is valid and applicable. Where this is not the case, the same boundaries between subdomains can be set within a single project, with fewer inconveniences that this architecture brings, and the decision on the division itself can wait. The growing use of the microfrontend architecture itself should not be a reason for its introduction in other projects.

Further research directions follow for the same reason. The first is to set up a process that builds and publishes only the changed part of the interface, thereby making measurable the issues that have only been described here: the amount of code that the browser downloads along a single route and throughout the user journey, the time from the change to its visibility to the user, and the behavior of the solution when one of its parts is not available. The second is the organizational side of the division, which no technical mechanism replaces; the distribution of ownership of areas, the agreement on shared dependencies, and the way to notice that the boundary between parts has begun to be violated. The first provides measures, the second provides the bearers of those measures, and together they fill the second half answers the question under which conditions the microfrontend architecture pays its price.

Microfrontend architecture is thus not a client-layer technique but a delivery technique, and is judged by that. MicroMedic has shown that its boundaries can be set and maintained well-separated, that consistency of appearance across different frameworks can be ensured by a good infrastructure, and that the use of multiple frameworks with certain management mechanisms remains a controlled decision, not an anti-pattern. What remains to be shown is not in the architecture, but around it.

Literature

Chen, KC (2021, August 24). *Micro Frontend Guide: Technical Integrations* . Retrieved September 1, 2026 from Trend Micro: https://www.trendmicro.com/de_de/research/21/h/micro-frontend-guide-technical-integrations.html

Clements, P., Bachmann, F., Bass, L., Garlan, D., Ivers, J., Little, R., . . . Stafford, J. (2010). *Documenting Software Architectures: Views and Beyond, Second Edition.* Boston: Addison-Wesley.

Collins, P., Gowans, A., Ackerman, JS, & Scruton, R. (2026, July 20). *architecture - Expression of technique* . Retrieved September 1, 2026 from Britannica: https://www.britannica.com/topic/architecture

Conway, ME (1968). *Conway's Law.*

Das, DK (2024, October 13). *Backend For Front End- Architectural Pattern* . Retrieved September 1, 2026 from Medium: https://medium.com/@dipakkrdas/backend-for-front-end-architectural-pattern-4500b262047e

Fowler, M. (2012, May 1). *Test Pyramid* . Retrieved September 1, 2026 from Martin Fowler: https://martinfowler.com/bliki/TestPyramid.html

Gandhi, R., Richards, M., & Ford, N. (2024). *Head First Software Architecture: A Learner's Guide to Architectural Thinking.* O'Reilly Media, Inc.

Garrett, JJ (2005). *Ajax: A new approach to web applications.* Washington.

Geers, M. (2020). *Micro Frontends in Action.* Manning Publications.

Glazkov, D. (2011, January 14). *What the Heck is Shadow DOM?* Retrieved May 11, 2025 from Glazkov: https://glazkov.com/2011/01/14/what-the-heck-is-shadow-dom/

Maruthi. (2022, May 27). *Planning for Blue-Green Deployments* . Retrieved September 1, 2026 from Medium: https://medium.com/@maruthijr/planning-for-blue-green-deployments-3a04f28c4a6d

Mezzalira, L. (2020, May). *Micro-frontends in context* . Retrieved September 1, 2026, from Increment: https://increment.com/frontend/micro-frontends-in-context/

Mezzalira, L. (2025). *Building Micro-Frontends* (Second ed.). O'Reilly Media, Inc.

Mikowski, MS, & Powell, JC (2013). *Single Page Web Applications.* Manning.

Mishra, M., Kunde, S., & Nambiar, M. (2018). *Cracking the monolith: Challenges in data transition to cloud native architectures.* New York, NY, USA: Association for Computing Machinery.

Monteiro, F. (2014). *Learning single-page web application development.* Packt Publishing Ltd.

Mosyan, D. (2023, September 23). *Claim-Check Design Pattern* . Retrieved September 1, 2026 from Medium: https://medium.com/@dmosyan/claim-check-design-pattern-603dc1f3796d

Mukhadin, B. (2026, May 11). *GraphQL protocol* . Retrieved September 1, 2026 from Wallarm: https://www.wallarm.com/what/what-is-graphql-definition-with-example

Münster, SA (2024). *Basics and Definitions. In Handbook of Digital 3D Reconstruction of Historical Architecture.* Cham.

Neill, CJ, Laplante, PA, & DeFranco, JF (2011). *Antipatterns.*

Newman, S. (2021). *Building Microservices, 2nd Edition* . O'Reilly Media Inc.

Nwamba, C. (2024, November 26). *The Ultimate Guide to the Broadcast Channel API* . Retrieved September 1, 2026 from Telerik & Kendo UI: https://www.telerik.com/blogs/ultimate-guide-broadcast-channel-api

Ottaviani Aragão, E. (2024, October 27). *Web Components — A Practical Perspective Using Custom Elements* . Retrieved September 1, 2026 from Medium: https://eduardo-ottaviani.medium.com/web-components-a-practical-perspective-using-custom-elements-7523a6462387

Ozkaya, M. (2021, September 7). *API Gateway Pattern* . Retrieved September 1, 2026 from Medium: https://medium.com/design-microservices-architecture-with-patterns/api-gateway-pattern-8ed0ddfce9df

Rapple, F. (2024). *The Art of Micro Frontends.*

Rappl, F. (2024, March 20). *Top 10 Micro Frontend Anti-Patterns* . Retrieved September 1, 2026 from DEV Community: https://dev.to/florianrappl/top-10-micro-frontend-anti-patterns-3809

Rastogi, A. (2022, February 10). *Canary Deployment Strategy: Benefits, Constraints And How It Can Be Used For Application Traffic Management* . Retrieved September 1, 2026 from

AppViewX: https://www.appviewx.com/blogs/canary-deployment-strategy-benefits-constraints-and-how-it-can-be-used-for-application-traffic-management/

Richards, M., & Ford, N. (2025). *Fundamentals of Software Architecture: An Engineering Approach.* O'Reilly Media, Inc.

Rufus, VJ (2023). *Building Micro Frontends with React 18.*

Vaishnavi, VK, & Kuechler, W. (2015). *Design Science Research Methods and Patterns.*

Villavicencio, N. (2023, August 29). *Strangler Fig Pattern: How to migrate from an old to a better architecture* . Retrieved September 1, 2026 from Samurai Developer: https://samurai-developer.com/strangler-fig-pattern/

Wright, G. (2022, April 8). *inline frame (iframe)* . Retrieved May 5, 2025 from TechTarget: https://www.techtarget.com/whatis/definition/IFrame-Inline-Frame#:~:text=An%20inline%20frame%20(iframe)%20is,webpage%20within%20the%20parent%20page.

Yadav, P. (2025, August 5). *What are Single Page Apps?* Retrieved September 4, 2025 from GeeksforGeeks: https://www.geeksforgeeks.org/what-are-single-page-apps/

Vlajić, S. (2020). *SOFTWARE DESIGN.* Belgrade: Golden Section.